GUESS

The Graph Exploration System — for the web

Eytan Adar

Manual version: November 2026

Contents

1. GUESS features **3**

2. Getting started **5**
2.1 Opening GUESS . . . 5
2.2 The window . . . 5
2.3 Finding help . . . 6

3. Tutorial **8**

4. Getting your data in **12**
4.1 Opening files . . . 12
4.2 The GUESS .gdf format . . . 12
4.3 Other formats . . . 14
4.4 Data from the web . . . 16
4.5 Building graphs in code . . . 16

5. Manipulating and querying nodes and edges **18**
5.1 Nodes, edges and fields . . . 18
5.2 Queries and sets . . . 18
5.3 Visibility . . . 20
5.4 The selection . . . 21
5.5 The data table . . . 21
5.6 SQL . . . 22

6. Laying out graphs **23**

7. Analysis commands **28**
7.1 Clustering and communities . . . 28
7.2 Visualizing fields . . . 28
7.3 Field, graph, node and edge statistics . . . 31
7.4 Paths and filters . . . 32
7.5 Charts . . . 33
7.6 Random graph generation . . . 35
7.7 Structure, mixing and comparing partitions . . . 35

7.8 networkx from the console . . . 36
7.9 igraph: faster statistics and more methods . . . 36

8. Modifying graphs **38**

9. Output commands **39**

10. Several graphs **40**

11. States and animations **41**
11.1 State-sensitive queries . . . 41
11.2 Ranges and time . . . 41

12. Legends, hulls and the display **43**
12.1 Legends . . . 43
12.2 Convex hulls . . . 43
12.3 The display . . . 44
12.4 Labels inside nodes, positions and sizes . . . 45

13. Direct manipulation **47**
13.1 Looking . . . 47
13.2 Selecting . . . 48
13.3 Dragging with meaning . . . 48
13.4 The click tools . . . 49
13.5 Responding to clicks in code . . . 50

14. The console, history and undo **51**

15. Modifying the interface / expanding GUESS **53**
15.1 Example 1: a simple button . . . 53
15.2 Example 2: a threshold slider . . . 53
15.3 Example 3: a network monitor . . . 54
15.4 Example 4: an app . . . 54
15.5 Code bits . . . 55

16. Sharing and deploying **56**
16.1 Sessions . . . 56
16.2 Links . . . 56
16.3 Notebooks and reports . . . 56
16.4 Deploying GUESS . . . 56
16.5 Working with an AI assistant . . . 56

Appendix A. Colors **58**

Appendix B. Gython and Python 3 **58**

Appendix C. Changes from the 2006 version **59**

Appendix D. Command reference **61**
Graph structure, loading and saving, mapping, view . . . 61

Undo, redo, history and logging . . . 73
Statistics, communities, filters and paths . . . 73
Clustering and generators . . . 75
More measures: Katz, prestige, bridging, cliques, MDS, k-brace; field tools and generators 76
More clustering: Chinese Whispers, MCL, DBSCAN, MCODE, overlapping communities . 78
Structure: bridges, cut nodes, backbones, directed graphs, mixing, comparing partitions . 80
networkx bridge and sample graphs . . . 82
igraph: communities, large-graph layouts, similarity, patterns, cuts and flows . . . 83
Graphviz layouts: dot, neato, sfdp, fdp, circo, twopi, osage; overlap removal . . . 86
Layouts . . . 87
Structure-aware layout (motifLayout, after LUMINA) . . . 92
More layouts: multipartite, concentric, ordered, circle packing, isometric layers . . . 92
Display: hulls, legends, background, edges, labels, selection, export . . . 93
Styles and visual mappings . . . 97
Pie nodes, borders and edge bundling . . . 98
Grouping: collapse and expand . . . 99
Several graphs: subgraphs and tabs . . . 100
Time: ranges, years and the timeline . . . 101
Presentations (slides) . . . 101
Data from the web: URLs, SPARQL, NDEx, Neo4j, packages . . . 101
Standalone web pages (exportHTML) . . . 103
Live graph streams (Gephi streaming protocol) . . . 103
AI assistants (MCP bridge) . . . 103
Cytoscape formats: SIF, NNF, Cytoscape.js JSON, CX/CX2, XGMML, .cys sessions . . . 104
Biology formats: PSI-MI (MITAB, XML), SBML, BioPAX, KEGG KGML, WikiPathways GPML 105
Import and export: Excel, RDF, Bluesky, JSON, KML, shapefiles, standalone HTML . . . 106
States and morphing . . . 107
Ranges (existence intervals) . . . 108
Charts . . . 109
Interface building (panels, widgets, timers, events) . . . 109
Sessions and apps . . . 110
SQL . . . 111
Colors . . . 111
Examples . . . 111
Fields . . . 123
Gython syntax . . . 124

1. GUESS features

GUESS is a tool for exploring graphs: social networks, citation networks, communication logs, biological pathways, anything with nodes and edges. You look at the graph, and you talk to it in a language built for the job. That was the idea in 2006 and it hasn't changed. What has changed is everything underneath: GUESS now runs in a web browser, there is nothing to install, and the language is Python 3.

This tool is/includes:

- **Gython**, Python 3 extended with operators for selecting and changing nodes and edges. (`freq`

`> 100).color = red` colours every edge whose `freq` field is over 100. `(dept == 'dept4') – (dept == 'dept9')` gives the edges between two departments. These are real Python expressions, so you can store them in variables, loop over them and write functions around them.

- **A zoomable view** of the graph. You can pan, zoom, hover for details, and select with clicks, boxes and lassos. Drags can mean something: after a circle layout, dragging a node reorders the ring.
- **A console made of cells.** Every command you run stays on the screen and can be edited, re-run, undone and redone. Output that mentions nodes or edges is linked back to the graph.
- **Fields and queries.** Nodes and edges carry properties (fields). You query them, map them to colour, size, shape and labels, chart them, and run SQL over them.
- **Analysis**: centrality measures computed on demand, communities (Louvain, edge betweenness, bicomponents), shortest paths, filters, and random graph generators.
- **Layouts**: ForceAtlas2, GEM, Fruchterman–Reingold, Kamada–Kawai, MDS, Sugiyama (layered), radial, circle, tree, hive plots, scatter plots by field, and overlap removal.
- **States and animation.** Save the graph as a named state, change things, and *morph* smoothly between states. Ranges and a timeline let you look at graphs that change over time.
- **Many formats in and out**: GDF (the GUESS format), GEXF (Gephi), GraphML, GML, Graphviz DOT, Pajek, UCINET DL, CSV, URLs, SPARQL endpoints (e.g. Wikidata), NDEx and Neo4j. Out: GDF, GEXF, GraphML, GML, CSV, PNG, JPG, GIF, SVG, PDF, PostScript/EPS, WebM movies, notebooks and HTML reports.
- **Interface building.** Panels, buttons, sliders and timers are a few lines of Gython. Package a graph and its interface as an *app*: a link that opens straight into your interactive figure.
- **Help everywhere**: a searchable command and example index, signature hints as you type, a guide to every mouse gesture, and error messages that point at the line and suggest the name you probably meant.

This tool isn't:

- A replacement for Gephi, Cytoscape, UCINET, Pajek or networkx. GUESS is for *exploring*: looking, asking, changing, and looking again. It does a fair amount of analysis, and when you need more it's still Python.
- A tool for enormous graphs. Tens of thousands of nodes are fine. Millions are not what it's built for.

2. Getting started

2.1 Opening GUESS

GUESS is a web page. Open its address in a current browser (Chrome, Edge, Firefox or Safari) and wait a few seconds. The first visit downloads the Python runtime, which takes a little while. After that it's cached, and GUESS also works offline. When it's ready, the sample graph appears and the console says `loaded 402 nodes, 725 edges`.

Nothing you load leaves your computer. Files you open are read by the page itself, and your work is autosaved in the browser. If you close the tab and come back, the graph, the history and the display are where you left them. Python variables you defined are not kept, but the commands that made them are in the history. *File* → *Start fresh* forgets all of it (the saved session, the history, your settings) and starts again with the sample graph.

The data files used by the GUESS cookbooks are there too: open them by name, e.g. `makeFromGDF("karate.gdf")`. Each is downloaded the first time it's used.

If you want to run your own copy, on a laptop with no network or on your group's web server, see Deploying GUESS.

2.2 The window

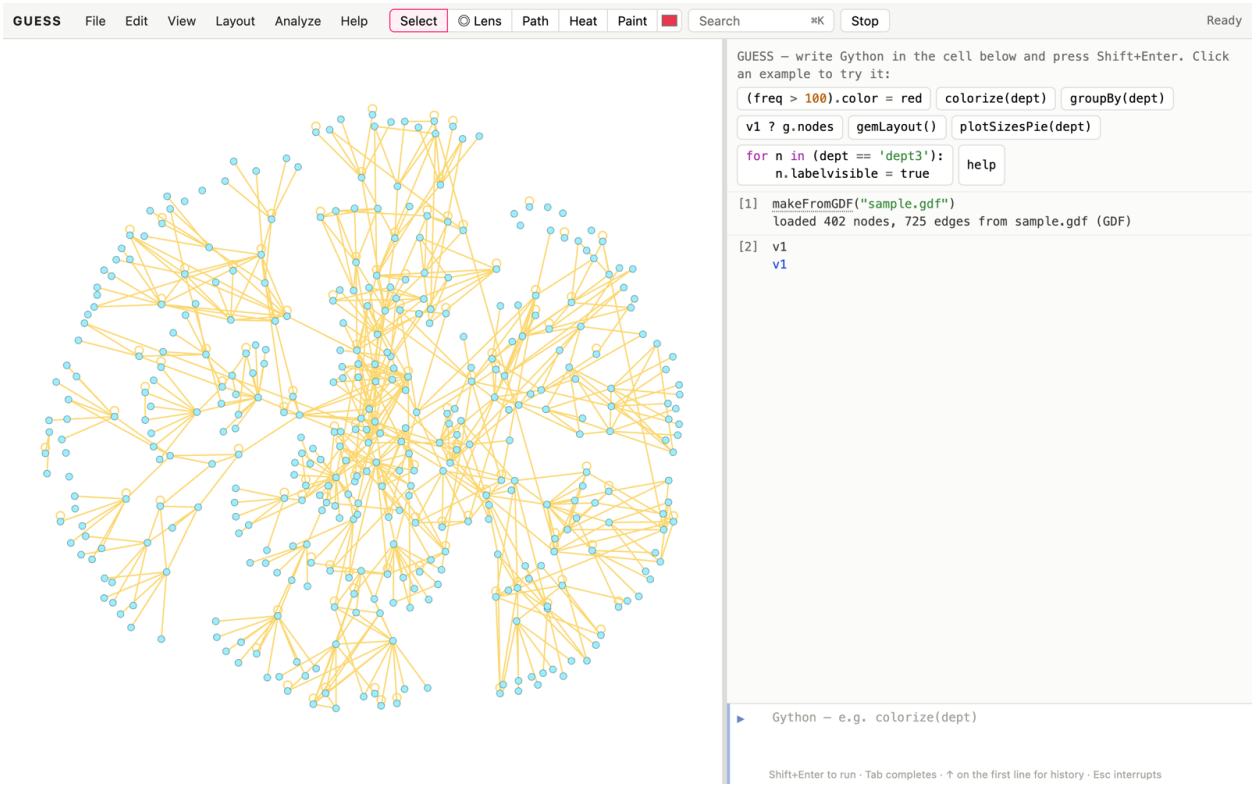


Figure 1: The GUESS window. The graph is on the left and the console on the right. The menus and tools are along the top.

The window has three parts:

- **The graph** (left). This is where you look, hover, click and drag.
- **The console** (right). Commands go in the working cell at the bottom. Each command you run becomes a numbered cell above it, followed by its output. You can drag the divider between the graph and the console. On a narrow screen the console moves below the graph.
- **The menu bar** (top): *File*, *Edit*, *View*, *Layout*, *Analyze* and *Help*, then the click tools (*Select*, *Lens*, *Path*, *Heat*, *Paint*, *Cluster* and a paint colour), a *Search* box, *Stop*, and a status line.

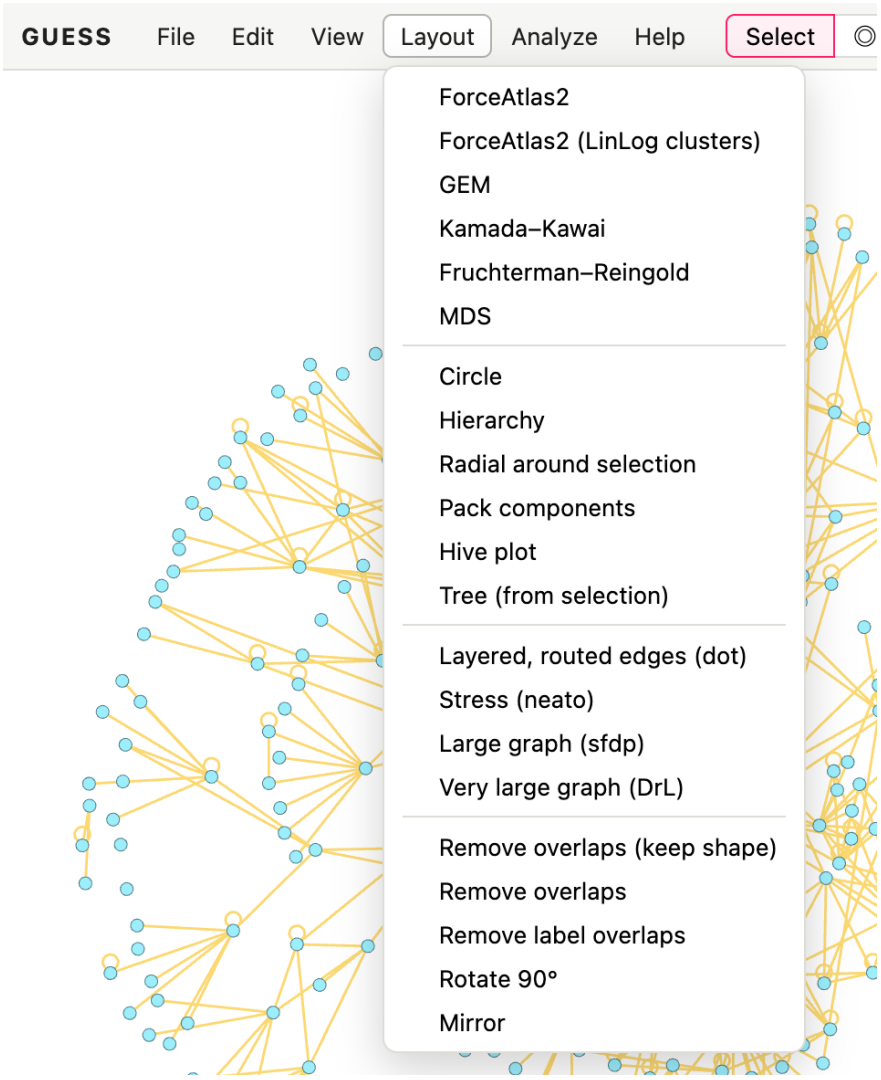


Figure 2: The menus collect the commands you'd otherwise type. Every menu item runs a command, and the command shows up in the console, so you can learn the language by using the menus.

Most menu items simply run a Gython command and show it in the console. This is deliberate. If you lay out the graph from the *Layout* menu, the console shows `forceAtlas2Layout()`, and the next time you can type it, change its options, or put it in a script.

2.3 Finding help

There are several ways in:

- **Help** → **Take the tour**, or `demo()` in the console, is a guided tour for newcomers. In 17 steps it goes from selecting nodes, through colouring, charts, centrality, communities and paths, to a what-if analysis, an Altair chart, a contagion model and a small app. Each step explains one idea and shows its code. *Run it* runs the code as a normal console cell (*Edit first* lets you change it), and a note then says what to look for. `demo(9)` starts at step 9; `demo("advanced")` at the advanced steps.
- **The code box's hint** suggests `demo()` on your first visit. After that it shows, in turn, short commands that work on the graph you have open, using its own fields and names: `colorize(club)` on the karate club, `plotHistogram(freq)` on the sample graph.
- **Help** → **Help and examples** (or **F1**) opens a panel with two tabs. *Commands & examples* is a searchable list of every command and a set of worked examples; click an example to put it in the console. *Mouse & gestures* explains, with pictures, everything the mouse does.
- **Search** (or **Cmd/Ctrl+K**) is a command palette. Type a few letters to find commands, examples, menu actions, nodes and fields.
- `help(colorize)` in the console prints the documentation of a command. `help()` alone opens the help panel.
- Hovering over a command name in the console shows its documentation. Typing `colorize(` shows its signature, and **Tab** completes names.
- When something goes wrong, the error names the line and, for misspelled names, the name you probably meant. Click *details* for the full Python traceback.
- **Help** → **Manual** opens this manual in a new tab, as a web page, PDF, EPUB or Word file. **Help** → **GitHub** opens the project's page, with the source code and issue tracker.

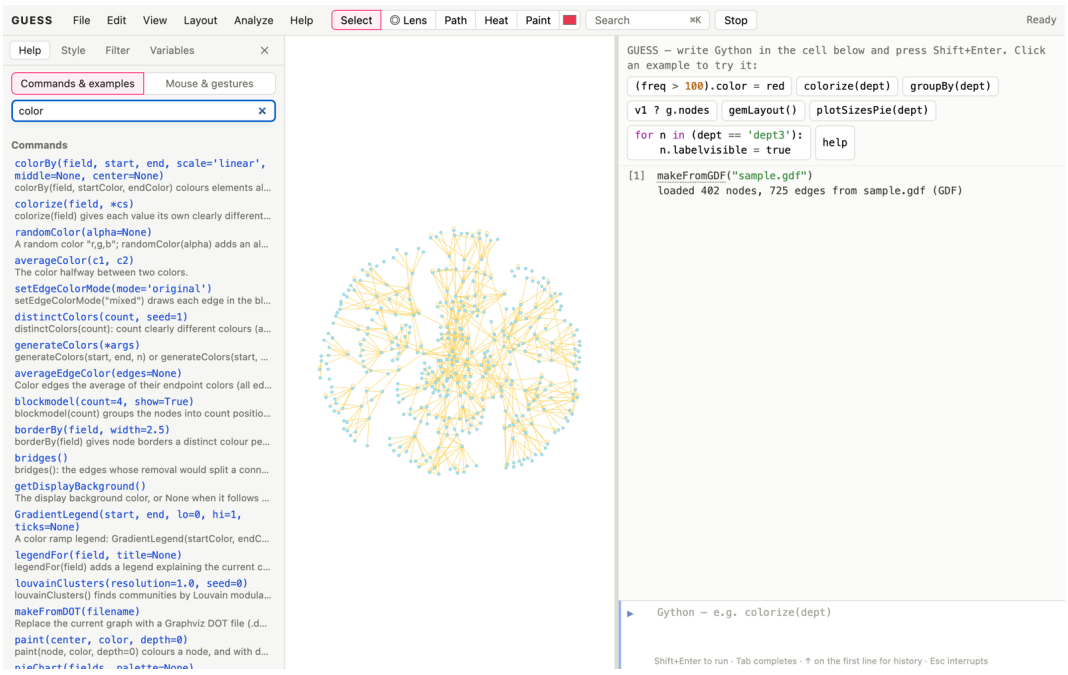


Figure 3: The help panel, searching for "color". Commands come first, then examples.

3. Tutorial

Let's start with a simple example. The sample graph that GUESS opens with (`sample.gdf`) has about 400 nodes and 700 edges. It represents a corporate communication network. Each node is an employee with a department (the `dept` field). Each edge is communication between two employees, and its `freq` field is the number of messages. If you've replaced the sample with something else, *File* → *Sample graph* brings it back.

First, some basics. Try moving around in this space:

- Drag the background to pan. Scroll or pinch to zoom. Double-click the background to fit the whole graph.
- Hover over a node or edge to see its fields. Hovering an edge also rings its two endpoints, so you can see where it goes.
- Click a node to focus on it and its neighbours. Click again, or press Esc, to clear. Double-click a node to zoom to it.
- Drag a node to move it.

Now click in the working cell at the bottom of the console. You're talking to a Python 3 interpreter. Type 2+2 and press **Shift+Enter** (or Cmd/Ctrl+Enter). It answers 4. Plain **Enter** adds a new line, so longer code is easy to write:

```
def fact(a):
    if a == 1:
        return 1
    else:
        return a * fact(a - 1)

fact(5)
```

Press Shift+Enter and you get 120. Through the console you can also control what you see. Type `center` and press Shift+Enter: the display zooms to fit all the nodes. Commands that take no arguments don't need the parentheses.

Nodes can be selected by name or by a query on their properties. For example, try `(v5, v6).color = red`. Nodes 5 and 6 turn red. Every node is a variable named after it, and assigning to a field of a group of nodes assigns it to each one. Nodes in the sample have a department, so

```
(dept == 'dept5').color = black
```

makes everyone in department 5 black. `dept == 'dept5'` is a query: it returns the set of nodes whose `dept` is `dept5`. Try typing just `dept == 'dept5'`. The console lists the nodes. Hover over one in the output and it lights up in the graph; click it and the view zooms to it.

Edges are accessed in a slightly different way. Edges are named by their end points. `(v67 – v76).color = red` changes the edge between person 67 and 76 to red. You can also find edges by query. Edges here have a `freq` field, so if we wanted to hide edges where the communication frequency was under 100 we would type:

```
(freq < 100).visible = false
```

The – operator means an edge in either direction. If the graph had directed edges (this one doesn't) you could also write `v67 -> v76` and `v67 <- v76`. `v67 <-> v76` means an undirected edge, and

`v67 ? v76` means any edge at all.

The last way to get at edges is through node sets. Let's say we only care about the communication between departments 4 and 9.

1. Let's hide everything: `g.nodes.visible = false`.
2. Then show only the nodes in departments 4 or 9: `((dept == 'dept4') | (dept == 'dept9')).visible = true`.
3. Colour the edges between the two departments: `((dept == 'dept4') - (dept == 'dept9')).color = red`. This finds all the nodes in department 4 and all the nodes in department 9, then all the edges between them (in this case only one).
4. Colour the edges inside department 9 blue: `((dept == 'dept9') - (dept == 'dept9')).color = blue`.

Because the language is Python, we could have made this simpler with some intermediate variables. The following gives the same result:

```
g.nodes.visible = false
dept4 = (dept == 'dept4')
dept9 = (dept == 'dept9')
(dept4, dept9).visible = true
(dept4 - dept9).color = red
(dept9 - dept9).color = blue
center()
```

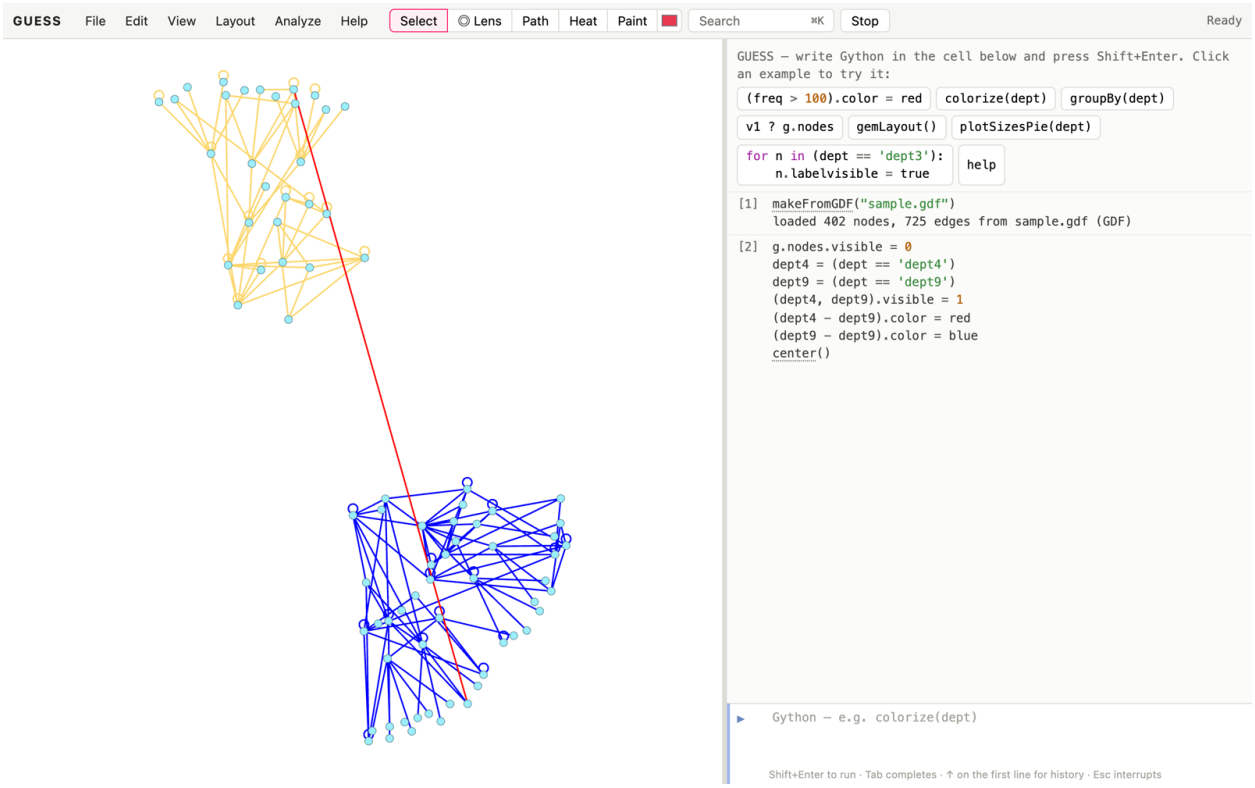


Figure 4: Departments 4 and 9. The one edge between them is red, and the edges inside department 9 are blue.

Made a mess? Press **Cmd/Ctrl+Z** (when the cursor isn't in the code cell), choose *Edit* → *Undo*, or type `undo()`. Each command is one step, and undone cells are struck through in the console. Redo brings them back.

GUESS also has analysis commands that simplify common tasks. First, let's put things back: `g.nodes.visible = true`, `g.edges.color = dandelion` and `g.nodes.color = cornflowerblue` (the defaults). Now type `density()`. This calculates the density of the graph (0.009).

Other commands do more interesting things. `colorize(dept)` gives each department its own colour, chosen to be as different from each other as possible. `colorBy(freq, blue, red)` colours the edges along a range from blue to red by their frequency.

Let's try something a little more interesting:

```
colorize(dept)
deptg = groupBy(dept)
xy = Legend()
for d in deptg: xy.add(d[0], d[0].dept)
```

`groupBy(dept)` returns a set of sets, one for each department. `Legend()` makes an empty legend. The last line goes through the departments and adds a "prototype" node from each, the first one, to the legend, along with its department's name. (The one-liner `legendFor(dept)` does the same thing.)

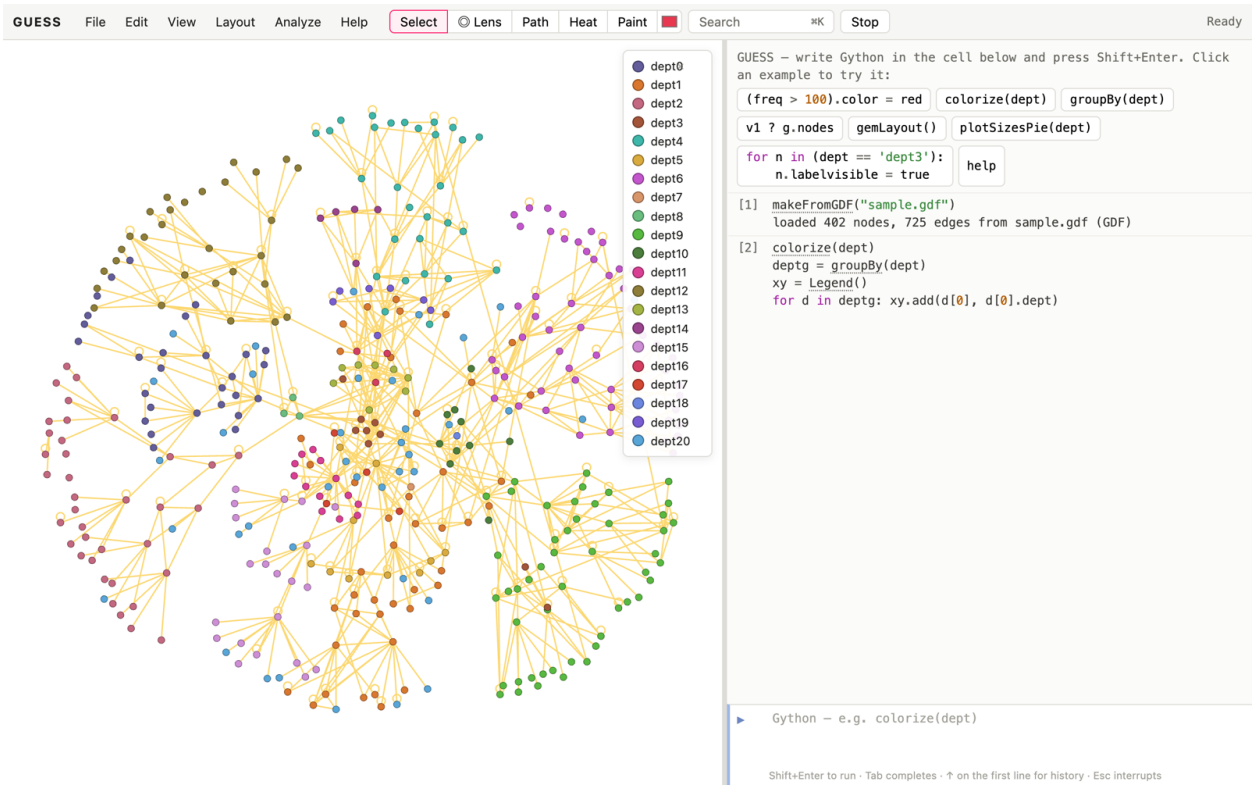


Figure 5: colorize(dept) and a legend built by hand.

This is standard Python, and any Python tutorial will help you with the general language. The rest

of this manual is about the parts that are particular to GUESS.

4. Getting your data in

There are three ways to get a graph into GUESS:

1. **Open a file.** Use *File* → *Open…*, or drag a file onto the window.
2. **Fetch it from the web.** Load a URL, query a SPARQL endpoint, or load an NDEx network or a Neo4j query.
3. **Build it in code.** Use `addNode`, `addEdge` and the random graph generators.

4.1 Opening files

File → *Open…* (or drag and drop) understands these formats:

extension	format	command
`.gdf`	GUESS data file (section 4.2)	`makeFromGDF("f.gdf")`
`.gexf`	Gephi	`makeFromGEXF("f.gexf")`
`.graphml`, `.xml`	GraphML	`makeFromGraphML("f.graphml")`
`.gml`	GML (networkx, igraph, yEd, Cytoscape)	`makeFromGML("f.gml")`
`.dot`, `.gv`	Graphviz DOT	`makeFromDOT("f.dot")`
`.net`	Pajek	`makeFromPajek("f.net")`
`.dl`	UCINET DL	`makeFromDL("f.dl")`
`.csv`, `.tsv`	edge table, node table, adjacency matrix or list (as Gephi reads them)	`makeFromCSV("edges.csv", "nodes.csv")`
`.guess`	a saved GUESS session	`makeFromSession("s.guess")`
`.py`	a Gython script (it runs)	`execfile("s.py")`
`.png`, `.jpg`, …	an image, for a background or node images	`setBackgroundImage("map.png")`

Opening a graph replaces the current one completely, its fields included, so nothing of the old graph lingers (`undo()` brings it back; to keep both, open the new one in another tab, section 10). Computed fields you were using, such as `pagerank`, are computed again for the new graph.

An opened file is also copied into GUESS's own file space, so the command form works on it afterwards. If you dropped `office.gdf` on the window, `makeFromGDF("office.gdf")` reloads it. *File* → *Paste data…* takes a table copied from a spreadsheet.

GML and DOT files bring their layout with them when they have one: GML `graphics [ x y w h fill type ]` and Graphviz `pos`, `width`, `height`, `color`/`fillcolor` and `shape` become positions, sizes, colours and shapes. Every other attribute becomes a field. Nodes in a Graphviz `cluster_*` subgraph get a `cluster` field, so `colorize(cluster)` shows the clusters. `makeFromGML` still opens GraphML too, as it did in GUESS 2006.

Loading a graph replaces the current one. To keep several graphs at once, see section 10.

4.2 The GUESS .gdf format

The GDF format is very simple. First we define the nodes and their properties, then the edges and theirs. The node section starts with a line like:

```
nodedef> name
```

The `nodedef>` line tells GUESS what is on each of the following lines. In the simplest case there is one column, the node's name. Names must be unique. GUESS makes a variable for every node, so names that are valid Python names (`v1`, `alice`, `gene_42`) are the most convenient. Other names still work, but you then reach the node with `g.getNodeByName("name with spaces")` instead.

The edge section starts with:

```
edgedef> node1,node2
```

and each line after it names the two endpoints. Here is a complete file:

```
nodedef> name
alice
bob
carol
edgedef> node1,node2
alice,bob
bob,carol
```

Now let's add some properties. Each column after the first is a field: a name, optionally followed by a type and a default value.

```
nodedef> name,dept VARCHAR(32),salary INT default 0,manager BOOLEAN
alice,sales,52000,true
bob,sales,48000,false
carol,engineering,,false
edgedef> node1,node2,freq INT,directed BOOLEAN
alice,bob,12,false
bob,carol,3,true
```

The types are `VARCHAR` (text, the default if you don't say), `INT`/`INTEGER` (also `TINYINT`, `SMALLINT`, `BIGINT`), `DOUBLE` (also `FLOAT`, `REAL`, `DECIMAL`) and `BOOLEAN` (also `BIT`). An empty value takes the default. Values containing commas go in single or double quotes.

Some field names are special. They control how nodes and edges look and behave, and you can set them in the file:

node field	meaning	default
`x`, `y`	position (if missing, GUESS lays out the graph)	
`color`	fill colour (a name or `"r,g,b"` / `"r,g,b,a"`)	cornflowerblue
`strokecolor`	outline colour	darker fill
`style`	shape: 1 rectangle, 2 ellipse, 3 rounded rectangle, 4–6 the same with the label inside, 7 image, 11 triangle, 12 diamond, 13 hexagon, 14 octagon	2
`width`, `height`	size	10
`label`, `labelvisible`, `labelcolor`	the label text, whether it's shown, its colour (inside a node: black or white for contrast, unless set)	name, false

node field	meaning	default
`labelpos`	where the label goes: `right`, `left`, `above`, `below`, `inside`, or `outside` (GUESS picks a free side)	right
`labelsize`	label size as a scale (1 = normal)	1
`image`	an image file or URL (with style 7)	
`visible`	whether it's drawn	true
`fixed`	layouts don't move it	false

edge field	meaning	default
`directed`	a directed edge (arrow)	false
`color`, `width`	colour and thickness	dandelion, 2
`weight`	used by weighted layouts and paths	1
`label`, `labelvisible`, `labelcolor`, `labelsize`	as for nodes; edge labels sit on a small background pill at the middle of the edge	
`visible`	whether it's drawn	true

GDF files embedded in old GUESS applets end each line with `;` and are sometimes written on one line. Both forms load.

4.3 Other formats

- **GEXF** (Gephi's format) keeps node attributes, edge attributes, colours, sizes and positions. `exportGEXF("g.gexf")` writes it back for Gephi.
- **GraphML** keeps typed attributes (`<key>`) with their defaults.
- **Pajek `.net`** reads vertices (with their coordinates, shapes and colours), arcs (directed) and edges (undirected), with weights.
- **UCINET DL** reads the edgelist and full-matrix forms, with labels.
- **GML** (networkx, igraph, yEd, Cytoscape) and **Graphviz DOT** keep their attributes and any layout they carry (section 4.1). `exportGML("g.gml")` and `exportDOT("g.dot")` write them. `exportDOT` groups nodes into Graphviz clusters by a field (`exportDOT("g.dot", clusters="louvain_class")`), and `layout=False` leaves positions out so Graphviz lays the graph out itself.
- **CSV** works as in Gephi. An edge table has `Source`, `Target`, and optionally `Type` (directed/undirected), `Weight` and `Label`; other columns become edge fields. A node table has `Id`, optionally `Label`, and other columns. GUESS also recognises adjacency matrices and adjacency lists. Opening only a node table adds its columns to the current graph's nodes.
- **Excel** workbooks (`.xlsx`) open the same way: `makeFromExcel("data.xlsx")` reads an edge sheet (or a matrix or list), and a node sheet with `Id` when there is one; `sheet=` and `nodes=` choose the sheets. `makeFromCSV` hands `.xlsx` files to it.
- **Lists in a column.** `makeFromCooccurrence("papers.csv", "authors", ";")` links the values that share a cell (co-authors, co-occurring keywords): nodes get `count`, edges `weight`. It reads CSV, TSV and Excel.

- **RDF** (Turtle, N-Triples, RDF/XML): `makeFromRDF("data.ttl")`. Resources become nodes, labelled by `rdfs:label` or `foaf:name`, with the IRI in `uri`; links between resources become edges with the property in `predicate`; literal values become node fields. Classes are nodes too (`types=false` keeps them as a `type` field).

As in Cytoscape's table import, any columns can hold the edges: `makeFromCSV("ppi.tsv", source="gene_a", target="gene_b", interaction="type", weight="score")` (by header, or by number from 0; `header=false` for a table without one). The interaction type goes in the edge field `interaction`.

```
makeFromExcel("network.xlsx", sheet="edges", nodes="people")
makeFromCooccurrence("papers.csv", "authors", ";")
makeFromRDF("foaf.ttl")
```

Cytoscape's formats. GUESS opens what Cytoscape opens:

format	command
SIF (simple interaction file)	`makeFromSIF`: the type goes in `interaction`
Cytoscape.js JSON (`.cyjs`, Cytoscape's web export)	`makeFromCYJS`; `exportCYJS` writes one for Cytoscape
CX and CX2 (NDEx)	`makeFromCX`
XGMML (Cytoscape's XML)	`makeFromXGMML`
a session (`.cys`)	`makeFromCYS(file)` opens its first network and lists the rest; `makeFromCYS(file, "all")` opens each as its own graph
NNF (nested networks)	`makeFromNNF`: `network` says where each node and edge belongs, `nested` which network a node stands for

Attributes become typed fields, and positions, colours, sizes and shapes come along; Cytoscape's style mappings don't. A name two nodes share gets the node's id added, and the original stays as the label.

Biological formats.

format	command
PSI-MI TAB (MITAB 2.5–2.7: IntAct, BioGRID, IMEx)	`makeFromMITAB`
PSI-MI XML 2.5 and 3.0	`makeFromPSIMI`: complexes are expanded around their bait (spoke), or to every pair
SBML (systems biology models)	`makeFromSBML`: species, and reactions as small grey nodes (`reactions="edges"` for edges instead)
BioPAX level 3 (Reactome, Pathway Commons)	`makeFromBioPAX`: a simple network of molecules, after Pathway Commons' rules (`mode="reactions"` keeps the reactions)
KEGG KGML, WikiPathways GPML	`makeFromKGML`, `makeFromGPML`: the map as drawn

Interaction files name proteins by gene name, with `ident`, `taxid` and `species`; repeated evidence for a pair becomes one edge with an `evidence` count, the methods and publications, and IntAct's `miscore` (`merge=false` keeps every record).

Any file. `makeFromFile(name)` picks the reader by extension, or by content when the extension doesn't say (`.xml`, `.txt`, `.json`), which is how *File → Open* and dropping a file work. `makeFromURL` does the same for downloads.

```
exportCYJS("sample.cyjs")
makeFromFile("sample.cyjs")
```

4.4 Data from the web

```
makeFromURL("https://example.org/network.gexf")         # any supported format
makeFromSPARQL("""
  SELECT ?item ?itemLabel ?linked ?linkedLabel WHERE {
    ?item wdt:P31 wd:Q5; wdt:P737 ?linked .               # people and who
 ↪  influenced them
    SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
  } LIMIT 300""")
makeFromNDEx("<network uuid>")                        # a network from ndexbio.org
makeFromCypher("MATCH (a)-[r]->(b) RETURN a, r, b LIMIT 500",
               url="http://localhost:7474", user="neo4j", password="...")
```

`makeFromSPARQL` turns each result row into an edge from the first column to the second (Wikidata is the default endpoint). Columns ending in `Label` name the nodes, and the remaining columns become fields. `sparql(query)` returns the rows without building a graph. A server must allow requests from web pages (CORS) for the browser to reach it; Wikidata and NDEx do.

Bluesky. `makeFromBluesky("handle")` builds the follow network of an account from Bluesky's public API: who it follows (`mode="followers"` or `"both"` for the other directions), and with `depth=2` who they follow in turn, up to `maxNodes` accounts. Nodes carry `handle`, `followers`, `follows`, `posts` and `description`.

Live streams. `streamFrom(url)` follows a graph that changes as you watch, in the format of Gephi's Graph Streaming plugin: one JSON event per line (over HTTP) or per message (over a `ws://` WebSocket), adding, changing and removing nodes and edges. Events are applied between commands, new nodes appear next to a neighbour, and streamed changes are not undo steps. `stopStream()` stops following.

```
makeFromBluesky("bsky.app", limit=50)
streamFrom("http://localhost:8080/workspace1?operation=getGraph")   # Gephi's
 ↪  streaming server
# each line: {"an": {"A": {"label": "Alice"}}}  {"ae": {"AB": {"source": "A",
 ↪  "target": "B"}}}
stopStream()
```

4.5 Building graphs in code

```
newGraph("scratch")              # an empty graph in a new tab (section 10)
addNode("ann", color=red)
addNode("bo")
```

```
addEdge(ann, bo, weight=3)
addDirectedEdge(bo, ann)
Node.score = 0.0                # a new node field (Edge.kind = "" for edges)
ann.score = 4.5
```

`addNode` makes a node and a variable with its name. `addEdge` (or `addUndirectedEdge`) and `addDirectedEdge` connect two nodes. Keyword arguments set fields. A new field must be made before you assign to it: `Node.score = 0.0` makes a number field whose default is 0.0 (`0` an integer field, `""` text, `false` true/false), as does `addNodeField("score", DOUBLE, 0.0)`. GUESS won't make a new field from a typo: `ann.scroe = 1` is an error that suggests `score`.

Classic networks from the literature load with `makeSample(name)`: `karate` (Zachary's karate club, with each member's `club`), `lesmis` (Les Misérables co-appearances), `florentine`, `davis` (Southern Women, two-mode), `petersen`, `kite`, `tutte`, `dodecahedron` and `caveman`. `makeSample()` lists them.

There are also random graph generators: `makeSimpleRandom(nodes, edges)`, `makeErdosRenyiRandom(n, p)`, `makeBarabasiAlbertRandom(start, add, steps)`, `makeKleinbergRandom(side, exponent)`, `makeLattice1DRandom(n, toroidal)`, `makeLattice2DRandom(side, toroidal)`, `makeEppsteinRandom(n, e, r)`, `makeSimpleDirectedRandom(n, e)`, `makeWattsStrogatz(n, k, p)` (small world), `makeBarabasiAlbertAging(n, m, aging)` (preferential attachment that fades with a node's age), `makeGeometricRandom(n, radius)` and `makePowerlawCluster(n, m, p)`. Each one replaces the current graph.

5. Manipulating and querying nodes and edges

5.1 Nodes, edges and fields

Every node is a variable named after it: `v1`, `alice`. The variable `g` is the graph. `g.nodes` and `g.edges` are all the nodes and all the edges. Edges are reached through their endpoints: `v1 - v2`, or `v1 -> v2` if direction matters.

A **field** is a property that every node (or every edge) has. Fields are named like variables. `dept` on its own is the department field, and `v1.dept` is v1's department. Fields come from the file you loaded, from the built-in set (section 4.2), from commands that add them (`louvainClusters()` adds `louvain_class`), and from `addNodeField/addEdgeField`. The node and edge fields are listed by `getNodeSchema()` and `getEdgeSchema()`. The *View* → *Data table* shows them all.

Fields know a little about themselves:

```
freq.min, freq.max, freq.avg, freq.sum, freq.median
max(freq)                       # a field is also its list of values
dept.distinct                   # every value, once each
dept.find('eng')                # the nodes whose dept is 'eng' (same as dept ==
 ↪  'eng')
highest(freq, 5)               # the five edges with the highest freq (lowest(...)
 ↪  too)
sortBy(freq, descending=true)  # every edge, highest freq first
groupBy(dept)                   # a set of sets, one per value
groupAndSortBy(dept)            # the same, ordered by value (dept0, dept1, ...)
```

Some node fields aren't stored but computed the first time you use them: `indegree`, `outdegree` and `totaldegree`, and the network statistics in section 7.3, such as `pagerank` and `betweenness`.

Names you've defined are yours, but GUESS protects its own names. You can't overwrite `g`, `dept`, `colorize` or a node's name by accident. Assigning to one gives an error that says so.

5.2 Queries and sets

A **query** compares a field with a value and returns the set of matching nodes or edges:

```
dept == 'dept3'                 # nodes
freq > 100                      # edges
totaldegree >= 10
name like 'v1%'                 # SQL-style pattern: % any text, _ one character
dept != 'dept3'
freq > freq.avg                 # compare with a statistic
```

Sets combine with `&` (intersection) and `|` (union), and with `.difference(...)`. `and`, `or` and `not` do the same on sets, a chained comparison is the intersection of its parts, and `in` asks for any of several values:

```
(dept == 'dept1') & (totaldegree > 3)
(dept == 'dept1') and (totaldegree > 3)  # the same
(dept == 'dept1') | (dept == 'dept2')
dept in ('dept1', 'dept2')              # the same
```

```
3 < totaldegree <= 6                    # (totaldegree > 3) & (totaldegree <= 6)
not (dept == 'dept1')                   # every other node
g.nodes.difference(dept == 'dept1')     # the same
```

Put each comparison in parentheses when you use & or |: `dept == 'dept1' & totaldegree > 3` means something else, and GUESS says so. In the test of an `if` or `while`, `and`, `or` and `not` keep their usual meaning (`if s and t:` asks whether both are non-empty).

A tuple or list of nodes also works as a set: `(v1, v2, v3).color = green`.

For a test no comparison can express, `where` keeps the members for which a function is true, and gives a set:

```
g.edges.where(lambda e: e.node1.dept != e.node2.dept)   # also interEdges(dept)
```

A query that can't match because of its type, such as `dept == 3` when `dept` holds text, prints a note saying so instead of quietly returning an empty set.

Broadcasting. Setting a field on a set sets it on every member. Reading a field from a set gives a list of values:

```
(dept == 'dept3').color = orange
(dept == 'dept3').size = 20          # size sets width and height together
(dept == 'dept3').labelvisible = true
(dept == 'dept3').totaldegree        # a list, one value per node
(dept == 'dept3').totaldegree.avg    # ... with .avg, .sum, .min, .max, .median,
↪  .distinct
```

Edges between sets. The edge operators work between nodes and between sets of nodes:

expression	gives
`a -> b`	edges directed from a to b
`a <- b`	edges directed from b to a
`a <-> b`, `a - b`	edges between a and b (undirected, or either direction)
`a ? b`	any edges between a and b

So `v1 ? g.nodes` is every edge touching v1, and `(dept == 'dept1') - (dept == 'dept3')` is every edge between the two departments.

From edges to nodes and back. `.nodes` on a set of edges gives their endpoints. `.edges` on a set of nodes gives the edges touching them:

```
(freq > 100).nodes.labelvisible = true    # label everyone in a busy conversation
(dept == 'dept3').edges.color = gray      # every edge touching department 3
v1.getNeighbors()                         # v1's neighbours
```

(`findNodes(set)` and `findEdges(set)` pick out just the nodes, or just the edges, from a mixed set.)

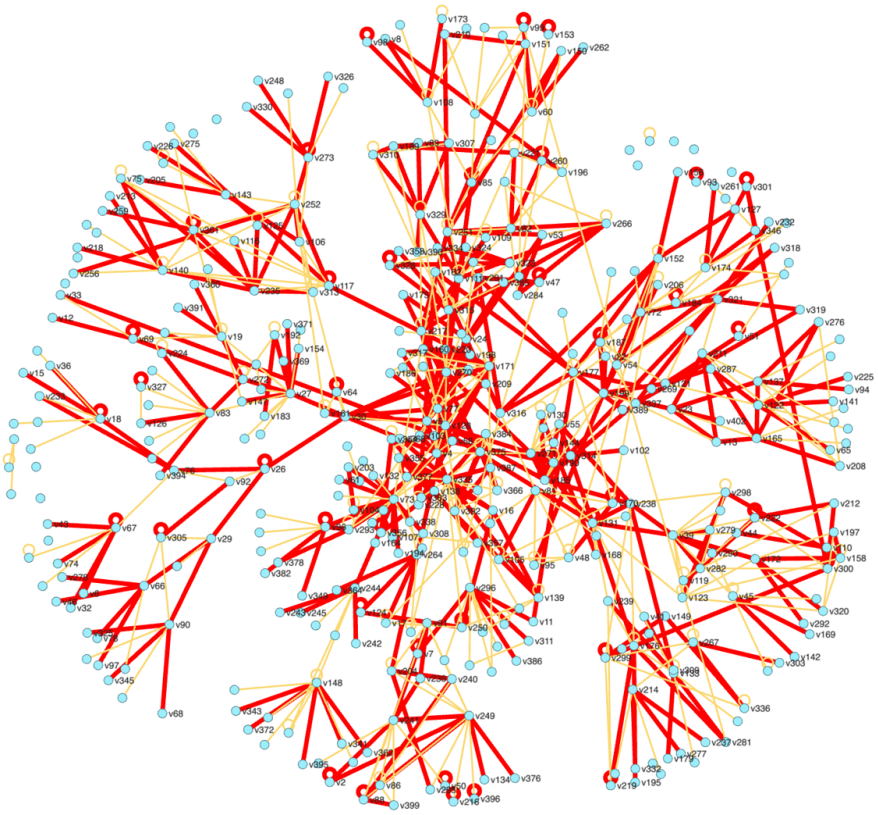


Figure 6: (freq > 100).color = red; (freq > 100).width = 5; (freq > 100).nodes.labelvisible = true

Sets are Python sequences. You can loop over them, take `len(...)`, index them and slice them:

```
for n in (dept == 'dept3'):
    n.label = n.name + " (" + n.dept + ")"
len(freq > 100)
```

5.3 Visibility

Hidden nodes and edges stay in the graph, so they still show up in queries. Two rules keep the picture consistent. Hiding a node hides its edges, and showing an edge shows its endpoints:

```
(dept == 'dept3').visible = false       # their edges disappear too
(freq > 150).visible = true             # these edges and their endpoints reappear
```

`showOnly(set)` hides everything except the set, and `showAll()` shows everything again. A set of nodes keeps the edges among them. A set that includes edges keeps just those edges, so `showOnly(freq > 100)` shows only the heavy ties and `showOnly(shortestPath(v1, v2))` only the path. `showOnly(set, induced=true)` keeps every edge among the shown nodes. To take nodes out of the graph rather than hide them, see section 8.

5.4 The selection

What you select with the mouse (section 13) is the variable `sel` in the console, and `select(...)` selects from code:

```
select(dept == 'dept2')
sel.color = orange
center(sel)
select()                        # clear it
```

5.5 The data table

View → *Data table* (or `table()`) opens a spreadsheet of the nodes or edges below the graph. `table(dept == 'dept3')` shows just those. You can sort by a column, filter, and double-click a cell to edit it. An edit is an ordinary command and can be undone. Selecting rows selects the nodes, and vice versa; in the edge table, the edges between selected nodes are marked. Hovering a node anywhere marks its row, and scrolls the table to it.

The filter box takes text or Gython. Text keeps the rows with a matching cell. A query keeps the rows it gives, and the box turns blue to say it was read as one:

```
degree > 5
dept == 'dept1' and degree > 3
sel                     # the selection
hubs                    # any variable holding nodes or edges
```

Queries are re-read as you type and whenever the graph changes. Expressions with calls, such as `highest(degree, 20)` or `(dept == 'dept1').neighbors`, run when you press Enter. They run on a scratch copy, so even `remove(...)` changes nothing. A query for edges typed in the node table (say `weight > 2`) says how many edges it found, so you can switch tabs.

Right-click a row for the same menu as on the graph: colour, labels, size, hide, *Select*, *Add to selection (sel)*, *Assign to variable…* and the rest. When the row is one of several selected rows, the menu acts on all of them. *Actions* ▽ opens the menu for the selected rows or, with none selected, for every row shown, so a filter followed by *Actions* acts on what it found.

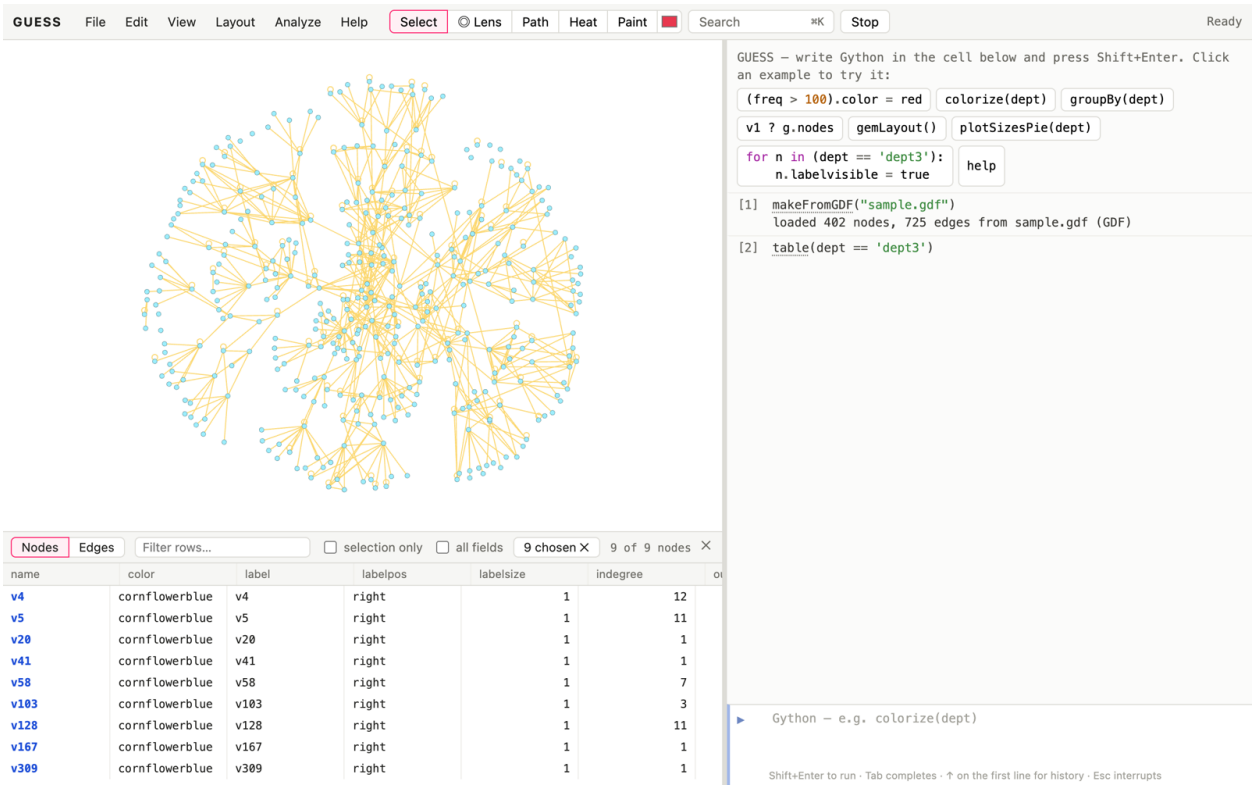


Figure 7: table(dept == 'dept3'): the data table below the graph.

5.6 SQL

If you'd rather ask in SQL, `sql(...)` puts the nodes and edges in two tables and runs your query:

```
sql("SELECT dept, count(*) FROM nodes GROUP BY dept ORDER BY 2 DESC")
sql("SELECT name FROM nodes WHERE totaldegree > 10")      # a set of nodes
sql("SELECT __edgeid FROM edges WHERE freq > 100")        # a set of edges
sql("UPDATE nodes SET color = 'red' WHERE dept = 'dept3'")
```

A query that returns only `name` (nodes) or `__edgeid` (edges) gives a set, so it combines with the rest of Gython. Other queries give rows. `UPDATE`s change the graph. Inserting and deleting rows isn't supported (use `addNode` and `remove`).

6. Laying out graphs

A layout command moves the nodes. Layouts animate while they run, and the *Stop* button or **Esc** stops them where they are. They're all under the *Layout* menu too, and every layout is one step for undo.

command	what it does
`forceAtlas2Layout()`	Gephi's force-directed layout. Options: `linLog=true` (tighter clusters), `preventOverlap=true`, `gravity=`, `scaling=`, `dissuadeHubs=true`, `edgeWeightInfluence=`, `iterations=`
`gemLayout()`	GEM, a fast force-directed layout
`frLayout()`	Fruchterman–Reingold
`kkLayout()`, `weightedKKLayout()`	Kamada–Kawai (spring distances follow graph distance)
`springLayout()`, `weightedSpringLayout(min, max)`, `isomLayout()`, `physicsLayout()`	other force and self-organising layouts
`mdsLayout()`	multidimensional scaling of the graph distances
`sugiyamaLayout()`, `dagLayout()`	layered drawing for hierarchies
`treeLayout(root)`	a tree (`horizontal=true` for left to right)
`radialLayout(v1)`	rings around a node by distance
`circleLayout()`	a circle; `circleLayout(dept)` makes one circle per group
`hiveLayout(axis, position)`	a hive plot (below)
`scatterLayout(x, y)`	a scatter plot of two fields
`randomLayout()`	random positions
`drlLayout()`, `lglLayout()`, `umapLayout()`	for large graphs (igraph, section 7.9): DrL (tens of thousands of nodes, clusters well apart), LGL (grown from a root), UMAP (tight islands)
`dhLayout()`	Davidson–Harel: even spacing, few crossings; small graphs
`dotLayout(rankdir="TB", clusters=None)`	Graphviz dot: layers with few crossings, long edges routed around nodes; `clusters=dept` keeps groups together
`neatoLayout()`	Graphviz neato (stress majorization): distances on screen follow distances in the graph; the best quality up to a few thousand nodes
`sfdpLayout()`, `fdpLayout(clusters=)`, `circoLayout()`, `twopiLayout(root)`, `osageLayout(field)`	other Graphviz engines: multilevel force (large graphs), force with cluster boxes, circles, rings around a root, packed groups
`graphvizLayout(engine, **attrs)`	any Graphviz engine with any graph attributes, e.g. `graphvizLayout("neato", mode="sgd")`
`bipartiteLayout(field)`	a two-mode network in two rows
`multipartiteLayout(field)`	one column per value of a field, each ordered to reduce crossings (`vertical=true` for rows)
`concentricLayout(field)`	rings by a field's value, the highest in the centre
`dualCircleLayout(count)`	the `count` busiest nodes on an inner circle, the rest outside, near the hubs they link to
`orderedLayout(field)`	one axis fixed by a field (a year, a step), ForceAtlas2 on the other
`circlePackLayout(field, …)`	nodes packed into nested, labelled circles by one or more fields

command	what it does
`isometricLayout(field)`	the current layout stacked into tilted layers by a field (the layer is stored in `zlayer`)
`hairballLayout()`	the Hairball Buster plot: nodes by degree rank, with a tick under each neighbour
`motifLayout()`	structure-aware: rings for cycles, stars for hubs with leaves, grids for nodes with the same neighbours (below)
`binPackLayout()`	packs the connected components side by side
`removeOverlap()`	separate overlapping nodes while keeping the layout's shape (Graphviz PRISM); works after any layout
`noverlapLayout()`, `labelAdjustLayout()`	nudge nodes until they (or their labels) don't overlap
`rotateLayout(90)`, `mirrorLayout()`, `rescaleLayout(0.5)`, `translateLayout(dx, dy)`, `moveLayout(x, y)`	transform the current layout; `rescaleLayout(nodes, 1.5)` spreads out just those nodes

The Graphviz and igraph layouts are loaded in the background a few seconds after GUESS starts (about 2 MB together). If you run one sooner, GUESS says so; run it again a moment later.

```
makeSimpleDirectedRandom(30, 45)
remove(feedbackEdges())                  # make it acyclic (section 7.7)
dotLayout()                              # layers, arrows pointing down
colorize(generation)
neatoLayout()
removeOverlap()
makeFromGDF("sample.gdf")                # back to the sample graph
```

Nodes whose `fixed` field is true stay where they are. So you can pin a few nodes and lay out the rest around them:

```
v1.fixed = true
gemLayout()
```

Use `centerAfterLayout(false)` if you'd rather the view didn't re-centre after every layout.

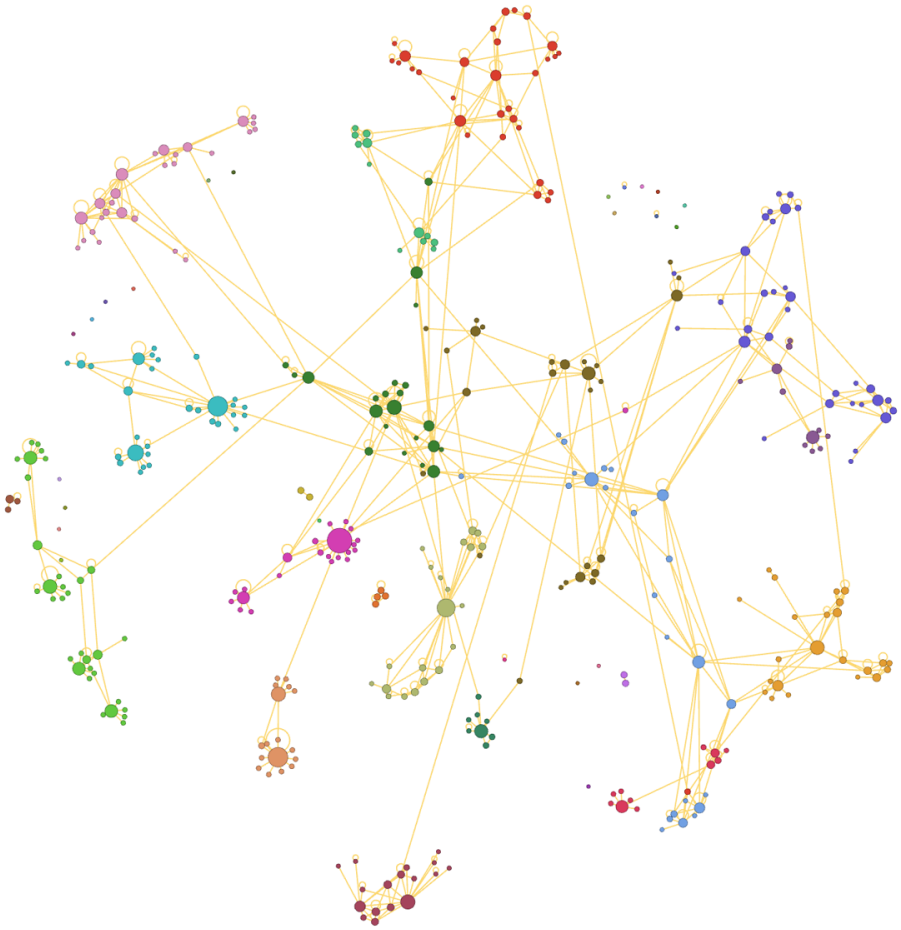


Figure 8: louvainClusters(); colorize(louvain_class); sizeBy(pagerank, 6, 40); forceAtlas2Layout(linLog=true, preventOverlap=true)

Hive plots. A hive plot puts each group on its own axis and places nodes along the axis by a value. Unlike force-directed layouts, the picture depends only on the data and not on chance, so two networks can be compared by their shapes.

```
hiveLayout(dept, totaldegree)              # an axis per department, ordered by
 ↪  degree
hiveLayout(dept, pagerank, axes=5)         # the 4 largest groups, plus "other"
```

The axes, their labels and the position note are drawn with the graph. They go away when you run another layout or load another graph.

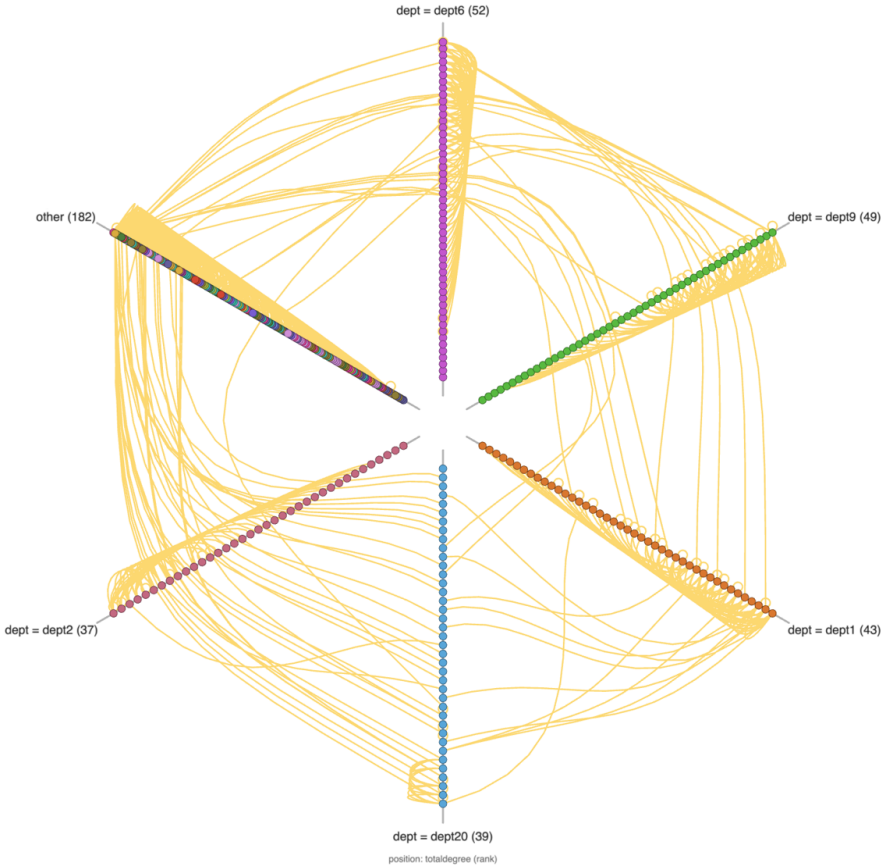


Figure 9: hiveLayout(dept, totaldegree)

Scatter plots. `scatterLayout(betweenness, pagerank)` places nodes by two fields, with axes. Dragging a node then edits those two fields (section 13).

Groups, rings and layers. These place nodes by a field, so the picture says what the field means:

```
multipartiteLayout(dept)                 # a column per department
concentricLayout(degree)                 # the best connected in the middle
circlePackLayout(dept)                   # departments as packed circles
gemLayout()
isometricLayout(dept)                    # the layout as stacked, tilted planes
orderedLayout(degree)                    # x from degree, y untangled by forces
forceAtlas2Layout(gravitySources=[v1, v73])   # two centres of gravity
```

Structure-aware layout. Protein interaction networks, and many others, are full of hubs with dozens of partners that touch nothing else. A force layout piles those partners into blobs. `motifLayout()` (a port of LUMINA, the IntAct team's layout) finds four shapes and gives each its own tidy arrangement: a node and its leaves become a *star* with the leaves in rings, nodes linked to exactly the same others form a *parallel* grid between them, *cycles* become circles and *chains* (paths ending in a leaf) small rings. Each shape is treated as one big node while the whole is laid out by

stress majorization, then the components are turned to fit and packed. The field `motif` says which shape each node is in, and `outline=true` draws an ellipse or box around every shape.

```
motifLayout()
colorize(motif)
motifLayout(outline=true, spacing=40)
removeConvexHulls()
```

`orderedLayout` suits anything with an order, such as citations by year or events by step, and works like Gephi's Ordered Layout and Event Graph Layout. `isometricLayout` stands in for 3D views such as Gephi's Network Splitter 3D: each value of the field gets its own plane. Running it again with another field starts from the flat layout.

Old names. The 2006 names still work. `random()` is the random layout, and `jfrLayout`, `jkkLayout1`, `jkkLayout2` and `jSpringLayout` are the JUNG versions of the force layouts. Every command is also a method of `g`, so `g.gemLayout()` works.

7. Analysis commands

7.1 Clustering and communities

Clustering commands return a set of sets (one per cluster) and, where it makes sense, add a field so you can colour by it:

```
louvainClusters()                    # communities; adds the field louvain_class
colorize(louvain_class)
modularity(louvain_class)       # how good the partition is
edgeBetweennessClusters(5)        # remove the 5 most "between" edges; the pieces
weakComponentClusters()            # connected components
biComponentClusters()              # biconnected components
groupsOfSize(dept, 40)             # groups with at least 40 members
```

More algorithms, each storing its result in a field named after it:

command	finds
`chineseWhispersClusters()`	fast, parameter-free communities: each node repeatedly takes its neighbours' most common class (`cw_class`)
`mclClusters(inflation=2)`	Markov clustering: random-walk flow sharpened until it stays in clusters; higher inflation, smaller clusters (`mcl_class`)
`dbscanClusters()`	nodes close together in the layout, or in any numeric fields, with the rest marked noise (-1) (`dbscan_class`)
`mcodeClusters()`	MCODE's dense complexes (Bader & Hogue), best first (`mcode_class`, `mcode_score`)

```
chineseWhispersClusters()
colorize(cw_class)
dbscanClusters((degree, pagerank), minPts=5)       # clusters in two fields, not
 ↪  positions
```

Overlapping communities. In real networks people belong to several groups at once. `cliquePercolationClusters(k)` chains cliques of k nodes that share k-1 of them; `linkCommunities()` clusters the edges instead of the nodes and cuts where partition density peaks. A node can then be in several communities: they're listed in a text field (`cpm_classes`, `link_classes`, as in `"0;3"`) with a count (`cpm_count`, `link_count`). `showOverlaps` colours each community and draws the nodes in several as pies, one slice per community; `removePies()` goes back to plain nodes.

```
cs = cliquePercolationClusters(3)
showOverlaps(cs, hulls=true)
cpm_count > 1                        # the people in more than one community
removePies()
```

7.2 Visualizing fields

The quickest way to see a field is to map it to colour, size, a label or a shape:

```
colorize(dept)                              # a distinct colour per value
colorBy(betweenness, yellow, red)           # a gradient over a number
colorBy(freq, yellow, red, 'log')           # scales: 'linear', 'log', 'sqrt', 'rank'
sizeBy(pagerank, 5, 40)                     # node size (or edge width) from a number
resizeLinear(totaldegree, 5, 30)            # the 2006 version
labelBy(dept)                               # labels from a field
shapeBy(dept)                               # a shape per value
polygonBy(dept, 3, 8)                     # a regular polygon per value (3 to 8 sides)
borderBy(dept)                              # a border colour per value
legendFor(dept)                             # a legend for the current colours
GradientLegend(yellow, red, betweenness.min, betweenness.max)
```

`polygon(sides)` is the shape code for any regular polygon, as in `(dept == 'dept1').style = polygon(5)`; `polygonBy(field)` takes the number of sides from a whole-number field, or gives each value of another field its own polygon.

`colorize` and `colorBy` work on node fields and on edge fields (they colour whichever the field belongs to). `colorize(freq, blue, red)` with two colours behaves like `colorBy`.

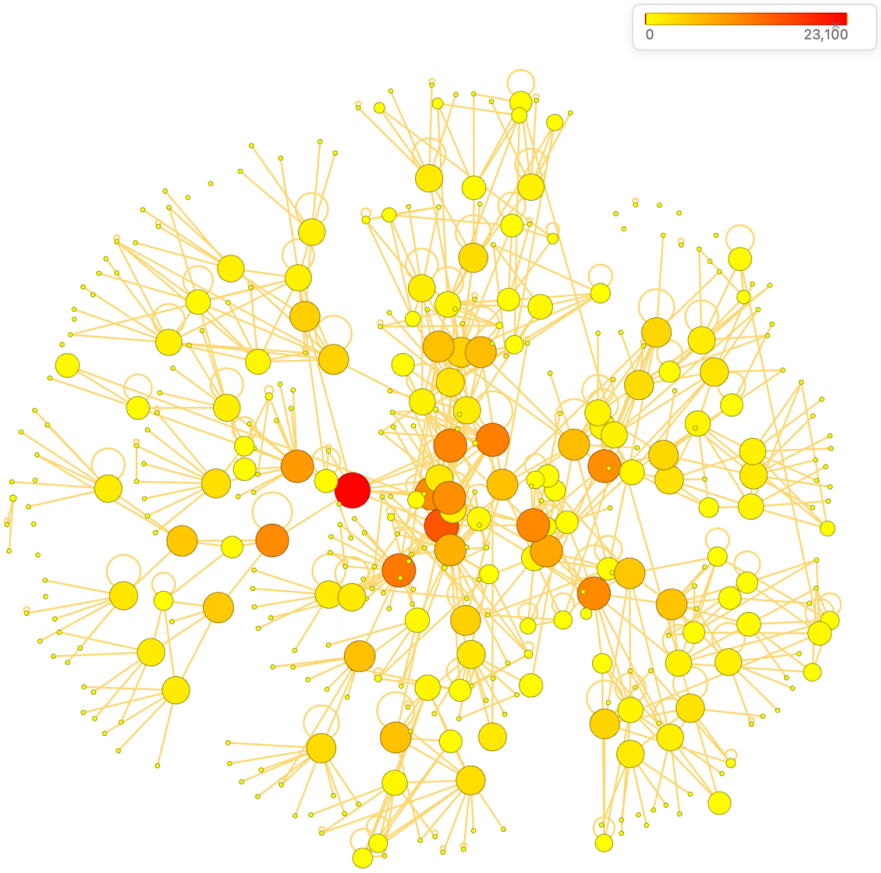


Figure 10: colorBy(betweenness, yellow, red); sizeBy(betweenness, 5, 40, 'log'); GradientLegend(…)

If you'd rather click than type, the **Style panel** (*View → Style panel*) does the same mappings with menus: pick nodes or edges, a property, a field and a mapping. It shows the command it runs. You can save a set of mappings as a named style, and `defineStyle`/`applyStyle` do the same from code:

```
defineStyle('by dept', ['colorize(dept)', 'legendFor(dept)'])
applyStyle('by dept')
```

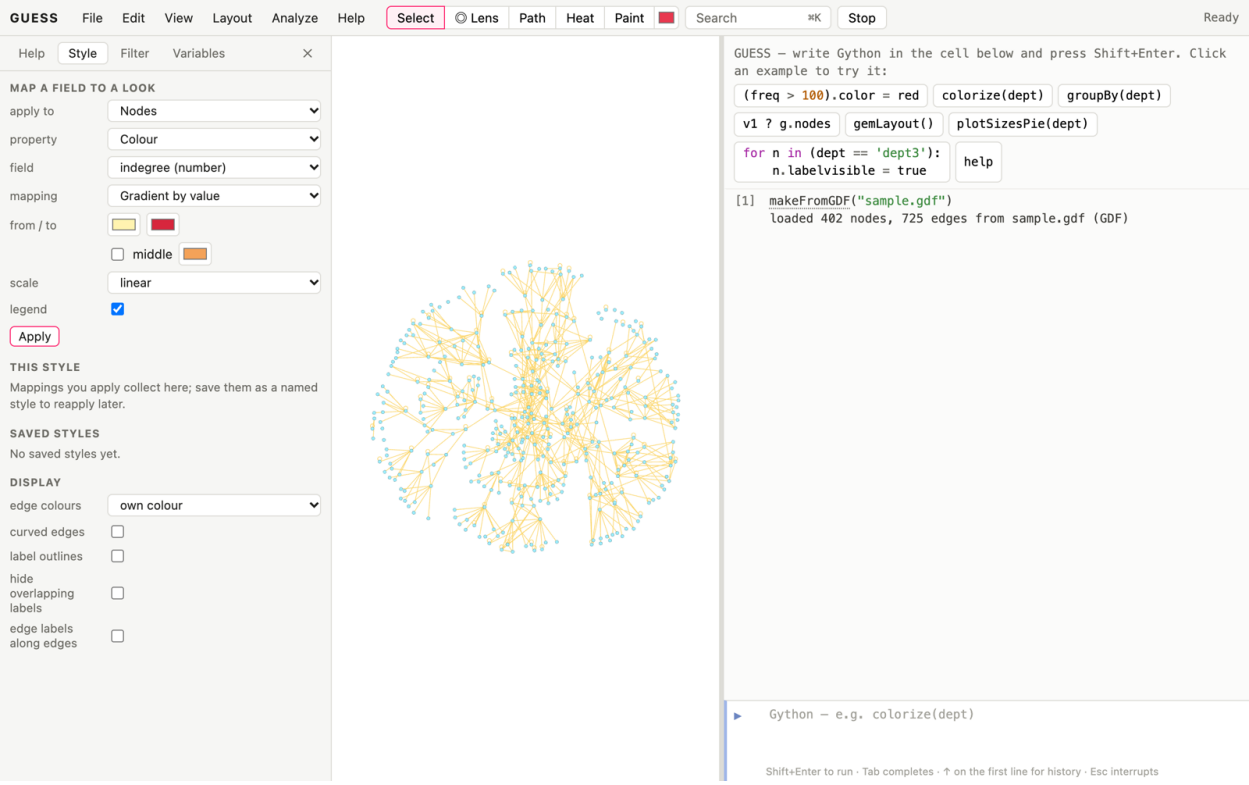


Figure 11: The Style panel.

A few more ways to show values on nodes:

```
pieChart([indegree, outdegree])     # each node becomes a pie of several fields
pieNeighbors(dept)                  # a pie of the neighbours' departments
removePies()
bundleEdges(dept)                 # bundle edges that run between the same groups
removeBundles()
```

Edge bundling also works without groups. `bundleEdges()` uses MINGLE (Gansner et al. 2011): edges that run alike merge, level by level, into bundles that share a trunk, saving "ink". It is fast enough for large graphs. `bundleEdges(method="fdeb")` is force-directed edge bundling (Holten and van Wijk 2009): edges attract compatible edges like elastic strands, giving smooth, organic curves; it suits up to a few thousand edges. Both work best when many long edges run between the same regions, and `strength=` (0 to 1) relaxes the bundles toward straight lines. The View menu has both.

```
bundleEdges()                          # MINGLE
bundleEdges(method="fdeb", strength=0.7)
removeBundles()
```

7.3 Field, graph, node and edge statistics

These node fields are computed the first time you use them. Use them like any other field:

field	meaning
`indegree`, `outdegree`, `totaldegree`	degree
`degrank`	degree as a fraction of the total degree
`strength`	weighted degree (sum of edge weights)
`pagerank`	PageRank
`hub`, `authority` (`hits`)	HITS scores
`eigenvector`	eigenvector centrality
`betweenness`	betweenness centrality (also an edge field)
`rwbetweenness`	random-walk (current-flow) betweenness
`closeness`, `harmonic`	closeness and harmonic centrality
`eccentricity`	the greatest distance to another node
`clustering`, `triangles`	local clustering coefficient, triangle count
`coreness`	k-core number
`component`, `strongcomponent`	which (strongly) connected component
`constraint`, `effsize`	Burt's structural holes: low constraint and a large effective size mark brokers
`generation`	the step in a directed hierarchy (0 = nothing points to it)
`katz`	Katz centrality: walks of every length, the longer ones counting less
`domain_prestige`, `proximity_prestige`, `rank_prestige`	prestige in directed networks: the share that can reach a node, also rewarding short distances, and being pointed at by prestigious nodes
`bridging_coef`, `bridging`	Hwang's bridging coefficient, and betweenness × bridging coefficient (bridging centrality)
`wclustering`	clustering weighted by edge weights

```
(pagerank > 0.01).labelvisible = true
sortBy(betweenness)[-10:]          # the ten most "between" nodes
```

These fields follow the graph: after you add or remove nodes or edges (or change edge weights), each is recalculated the next time it's read, once, however many edits came before. If you assign to one yourself (`v1.pagerank = 0`) it keeps your values until `recompute(pagerank)`.

Whole-graph measures are commands: `density()`, `diameter()`, `averagePathLength()`, `averageClustering()`, `transitivity()`, `assortativity()` (do hubs link to hubs?), `reciprocity()` (directed edges), `triadCensus()`, and `graphStats()` for a summary. `distance(v1, v2)` is the distance between two nodes on the screen.

A few more commands measure and store:

```
weightedBridging()                  # wbetweenness and wbridging, heavy edges as
 ↪  short ones
cliques(4)                          # every maximal clique of 4 or more
mds(3)                              # mds1, mds2, mds3: coordinates from graph
 ↪  distances
showOnly(kBrace(2))                 # edges whose ends share 2+ neighbours, after
 ↪  pruning
showAll()
```

`geoVectors()` (for nodes with latitude and longitude) stores each edge's `bearing` and length in `km` and summarises the edges' mean length and their overall direction.

7.4 Paths and filters

```
shortestPath(v1, v14).color = red       # nodes and edges of a shortest path
v1.unweightedShortestPath(v14)          # the path's edges (2006 form)
v1.dijkstraShortestPath(v14)            # weighted by the weight field
heatMap(v1)                             # colour everyone by distance from v1
showOnly(kCore(3))                      # the 3-core
showOnly(egoNetwork(v1, 2))             # everyone within 2 steps of v1
showOnly(giantComponent())
showOnly(intraEdges(dept))              # edges inside departments
interEdges(dept)                        # edges between departments
showAll()
```

The filters return sets, so they combine with queries: `showOnly(kCore(3) & (dept == 'dept1'))`. The **Filter panel** (*View → Filter panel*) stacks filters with sliders and checkboxes, if you'd rather build them by hand.

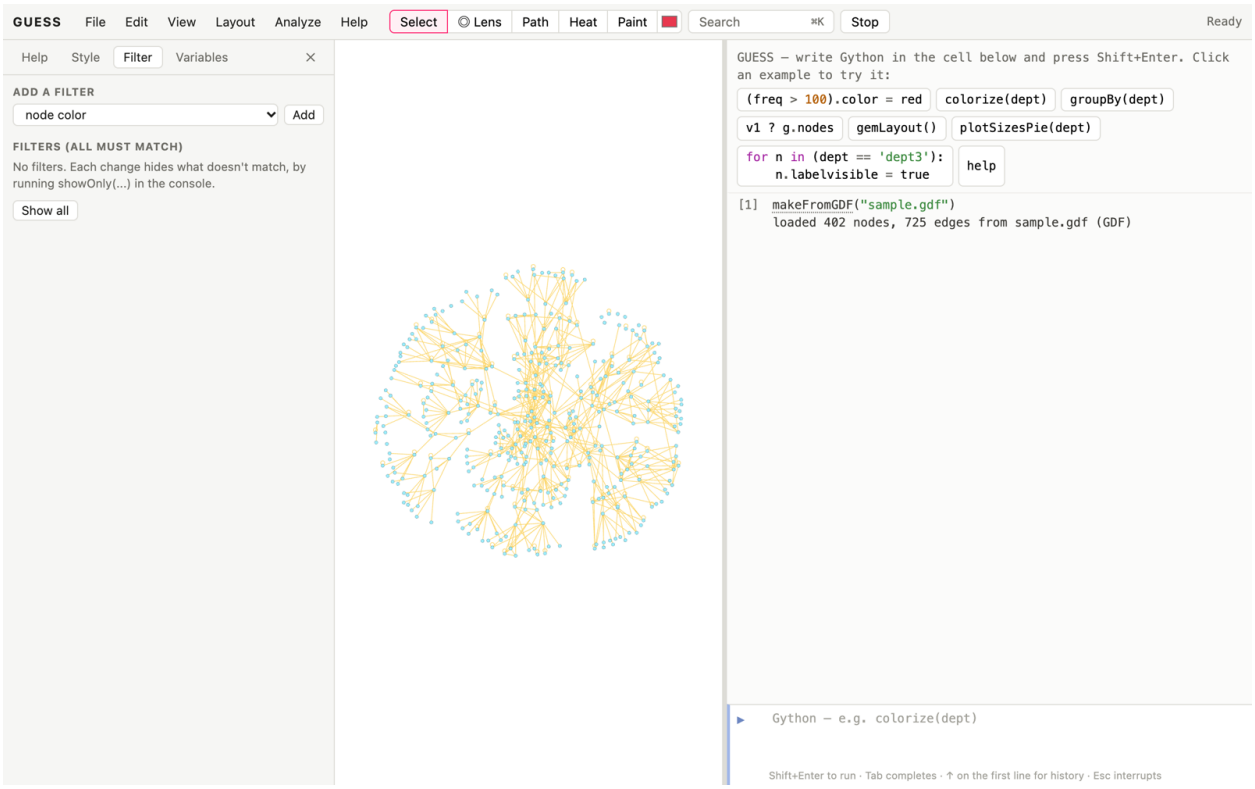


Figure 12: The Filter panel.

7.5 Charts

Charts are linked to the graph. Hover over a bar or a slice and its nodes light up, along with the console output and table rows that mention them; click and they're selected (shift-click adds). Drag across several bars of a bar chart or histogram to select them all at once. Right-click for the element menu: on a histogram bin or a bar it acts on the bin's query, such as `(degree >= 0) & (degree < 4.75)`. Hovering a bar also marks the bars of other charts that share its elements. On a line chart (`spread`, `robustness`, `extinctions`), pointing shows every series' value at that step. On the `spread` chart it also highlights who was reached by then, and a click selects them.

```
plotSizesPie(dept)          # how many in each department
plotSizes(dept)             # the same, as bars
plotHistogram(pagerank)     # distribution of a number
plotDistrib(totaldegree)    # each value in rank order
```

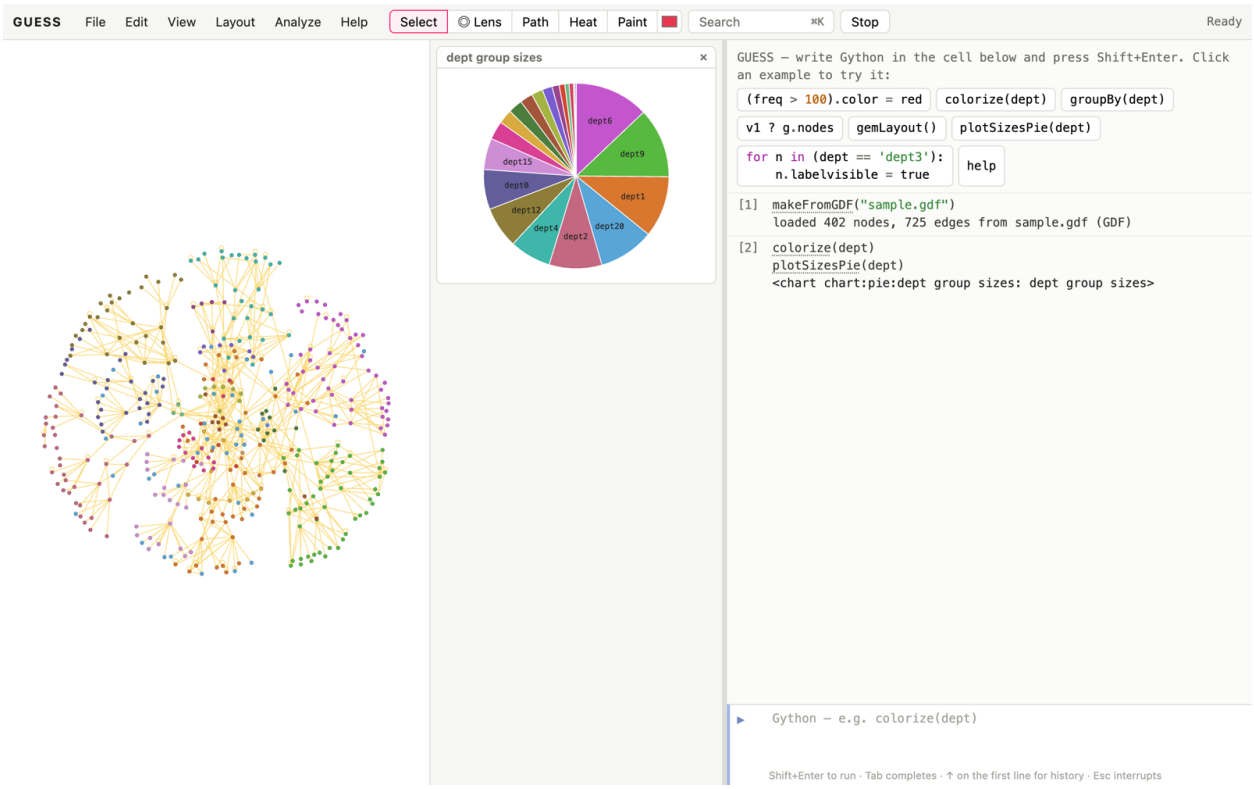


Figure 13: plotSizesPie(dept)

Live charts. A chart shows the graph as it was when you made it. Click ● *Live* in its header and it is recomputed after every command that changes the graph or the display, including undo and the history scrubber. `liveCharts()` makes every chart live, now and from then on; `liveCharts(False)` stops them all. Recomputing only redraws the chart: a command like `spread` that colours nodes when you run it doesn't recolour them when its chart updates. A chart that can no longer be made (its field has no numbers left, say) stops being live and says so. Legends made by `colorize` and `legendFor` follow the colours by themselves.

Altair charts. For any other chart, use Altair. `nodeData(...)` gives one row per node with the fields you name (all data fields when you name none), and `edgeData(...)` one row per edge. A cell whose value is a chart shows it in the dock, like the charts above. `alt` is imported the first time you use it, which takes a few seconds once.

```
alt.Chart(nodeData("degree", "pagerank", "dept")).mark_point().encode(
    x="degree", y="pagerank", color="dept", tooltip="name")

alt.Chart(nodeData("dept")).mark_bar().encode(x="dept", y="count()")

alt.Chart(nodeData(dept == "dept3", "degree")).mark_bar().encode(
    alt.X("degree", bin=True), y="count()")
```

Field types come from the graph, so `x="degree"` is a number and `x="dept"` a category, without `:Q` or `:N`. A chart that uses a field missing from its data says how to add it.

Altair charts are linked like the others. Each row carries its element's id, so a point for one node

highlights that node, clicking selects it, and right-clicking opens its menu. Hovering nodes anywhere else fades the other points. A mark that summarises several nodes stands for the query it was grouped by: a `count()` bar for `dept == 'dept1'`, or a bin for `(degree >= 4) & (degree < 6)`. Selections you add with Altair link too: dragging an `alt.selection_interval()` brush highlights the nodes inside it, and selects them when you stop. A point selection on a field (a clickable legend) does the same for the values chosen.

`showChart(chart, title)` shows a chart from inside a script or function. Running the same cell again redraws its chart in place. Altair charts can be live too: their `nodeData` and `edgeData` rows are rebuilt from the graph each time. The … menu on a chart saves it as SVG or PNG. Data frames work too (`import pandas`), but only `nodeData` and `edgeData` rows link to the graph.

Word clouds. `wordCloud(field)` shows the most frequent words of a text field, sized by how often they occur, with common English words left out (`stopwords=[...]` adds more). Hovering a word lights up the nodes whose text contains it; clicking selects them. Like the other charts it can be live.

```
wordCloud(dept)
```

7.6 Random graph generation

See section 4.5 for the generators. A typical use is to compare a network with random networks of the same size:

```
makeErdosRenyiRandom(400, 0.009)
averageClustering()
```

7.7 Structure, mixing and comparing partitions

These commands return sets you can colour, hide or show, or a single number:

```
makeFromGDF("sample.gdf")                  # back to the sample graph
bridges().color = red                      # edges whose removal splits the graph
cutNodes().size = 30                       # nodes whose removal splits it
showOnly(spanningTree(maximum=True))       # the backbone of strongest ties
showAll()
sortBy(constraint)[:5]                     # brokers (structural holes)
assortativity(dept)                       # 1: ties stay in a department; 0: random
mixingMatrix(dept)                         # edges between each pair of departments
louvainClusters()
compareClusters(louvain_class, dept)   # do the communities match departments?
```

`compareClusters` gives normalized mutual information by default (0 to 1); `method="ari"` (adjusted Rand), `"rand"` and `"vi"` are the alternatives.

For directed graphs such as citations or dependencies, `ancestors(node)` and `descendants(node)` follow the arrows, the `generation` field gives each node's step in the hierarchy, `transitiveReduction()` returns the edges that longer paths already imply (hide them to see the backbone), and `feedbackEdges()` returns edges that close cycles.

7.8 networkx from the console

networkx is loaded with GUESS, so any of its algorithms is one line away. `toNetworkX()` gives the graph with its fields as attributes (node keys are node names). `setField` stores a result as a field: a node dictionary, an edge dictionary, or a list of communities.

```
import networkx as nx
G = toNetworkX()
setField(nx.average_neighbor_degree(G), "nbrdeg")
setField(nx.community.greedy_modularity_communities(G), "greedy")
colorize(greedy)
```

`fromNetworkX` goes the other way. It turns attributes into fields and uses `pos` (or `x` and `y`) for positions, with a spring layout when there are none. Given a name, it opens the graph in a new tab and switches to it; the tabs above the graph switch back.

```
fromNetworkX(nx.karate_club_graph(), "karate")
colorize(club)
use("main")                                  # back to the first graph
```

7.9 igraph: faster statistics and more methods

GUESS also loads igraph, a graph library written in C, a few seconds after it starts. Once it is there, betweenness, closeness, harmonic, eccentricity, diameter, clustering, triangles, coreness and components use it. The numbers are the same and large graphs are many times faster. `useIgraph(false)` switches back to the networkx and numpy versions, for comparison.

igraph also adds community methods. Each stores its result in a field (`leiden_class`, `infomap_class`, …; 0 is the largest community) and prints the modularity:

command	method
`leidenClusters(resolution=1)`	Leiden: like Louvain, but every community is connected; `objective="CPM"` avoids modularity's resolution limit
`infomapClusters()`	Infomap: random-walk flow, best for directed graphs
`walktrapClusters(count)`, `fastGreedyClusters(count)`	hierarchies; `cutDendrogram(n)` re-cuts them into n groups
`labelPropagationClusters(field)`	very fast; given a field, spreads its values to the nodes without one
`eigenvectorClusters()`, `fluidClusters(count)`, `voronoiClusters()`	spectral; exactly `count` groups; around central nodes
`spinglassClusters()`, `optimalClusters()`, `cohesiveBlocks()`	small graphs: signed networks; the exact optimum; nested cohesive groups

```
makeFromGDF("sample.gdf")
leidenClusters()
colorize(leiden_class)
compareClusters(leiden_class, dept)
similarTo(v1)                                # nodes whose neighbours overlap v1's
```

```
predictLinks(5)                              # likely missing edges
evaluateLinks("all")                       # how well each method finds hidden edges
findPattern([("a", "b"), ("b", "c"), ("c", "a")])   # every triangle
minCut(v1, v14)                              # fewest edges separating two nodes
personalizedPagerank(v1)                     # relevance to v1, in the field ppr
largestCliques()
```

Also: `maxFlow(a, b)` (writes an edge field `flow`), `dominatorTree(root)`, `maxMatching()`, the node field `knn` (average degree of the neighbours), and two commands that don't need igraph: `simplify()` merges parallel edges, and `rewire()` makes a degree-preserving random copy in a new tab, to check whether a pattern is more than chance.

8. Modifying graphs

Let's take a few nodes out of the graph:

```
remove(totaldegree == 0)       # remove the isolated nodes
remove(v4)                     # a node takes its edges with it
remove(freq < 10)              # edges can be removed too
```

Removed elements go to `g.deleted`, so `add(g.deleted)` puts everything back. As in the 2006 version, a removed node keeps its name, so this works:

```
removeNode(v10)
addNode(v10)
```

`removeComplete(x)` removes something for good (it doesn't go to `g.deleted`). Undo works either way.

Three field tools: `castField(score, INTEGER)` changes a field's type and converts its values; `showOnly(readNames("keep.txt"))` keeps the nodes listed in a file (one name per line); `copyColors("other")` copies node colours, matched by name, from another graph of the session (section 10).

Other structural commands:

```
copyNode(v128, 'v128copy', color=red)    # a copy with the same fields
removeDisconnected()         # nodes with no edges
removeLeaves()               # nodes with one edge
removeSelfLoops()
shortcutNodes()              # replace nodes of degree 2 by a single edge
condenseGraph()              # remove leaves and shortcut nodes until nothing
 ↪  changes
```

Fields. Declare a new field before you use it. It then has a value on every node (or edge):

```
addNodeField("seniority", INTEGER, 0)
addEdgeField("kind", VARCHAR, "email")
v128.seniority = 10
```

The types are `INTEGER`, `DOUBLE`, `VARCHAR` and `BOOLEAN`, as in GDF.

Grouping. You can collapse a set of nodes into one *meta-node* and expand it again. This is useful for showing the structure between groups:

```
collapseBy(dept)           # one node per department, edges merged
expandAll()
collapse(dept == 'dept3', 'dept3group')
expand(dept3group)
```

A meta-node's size reflects how many nodes it holds, and double-clicking it expands it.

9. Output commands

command	output
`exportPNG("g.png")`, `exportJPG(...)`, `exportGIF(...)`	an image of the view
`exportPDF("g.pdf")`, `exportPS(...)`, `exportEPS(...)`	an image of the view in a PDF / PostScript page
`exportSVG("g.svg")`	the whole graph as vector graphics
`exportGDF("g.gdf")`, `exportGEXF(...)`, `exportGraphML(...)`, `exportGML(...)`, `exportDOT(...)`, `exportCSV(...)`	the graph and its fields
`exportHTML("g.html")`	one self-contained, interactive web page (below)
`exportJSON("g.json")`	graphology JSON for sigma.js and other web libraries (`format="sigma"` for the older sigma format)
`exportKML("g.kml")`, `exportSHP("g.zip")`	for Google Earth, and point and line shapefiles for QGIS or ArcGIS, from latitude and longitude fields
`saveSession("s.guess")`	everything: graph, fields, states, styles, legends, hulls, view, slides
`saveHistory("work.py")`	every command you ran, as a script
`startMovie(30, "m.webm")` … `stopMovie()`	a movie of the display

In a browser, "saving a file" means downloading it, so these files go to your downloads folder. The *File* menu has the common ones, plus:

- **Export notebook** writes the session as a Jupyter notebook (one cell per command).
- **Export report** writes an HTML page with the commands, their output, and a picture of the graph.
- **Share link** makes a URL that contains the whole session. Anyone who opens it sees your graph as you left it. **Share as viewer** does the same, but hides the console.

A page anyone can open. `exportHTML("graph.html")` writes the graph as one web page with nothing else to install: open it from disk, email it or put it on any web server. It shows the graph as drawn here (colours, sizes, shapes, labels, legends, images) and lets the reader pan and zoom, search for a node, hover to read its fields and click to see its neighbours. It follows the reader's light or dark setting. `fields=false` leaves the data fields out; `hidden=true` includes hidden nodes and edges.

```
exportHTML("organisation.html", title="Who works with whom")
```

`history()` lists every command you've run this session. `log("work.py")` starts writing commands to a file as you go, and `stoplog()` stops.

10. Several graphs

Sometimes you want to look at part of a graph on its own, or compare two graphs. Each graph lives in a tab above the display:

```
subgraph(dept == 'dept3', 'dept3')        # a copy of these nodes and their
 ↪  edges, in a new tab
subgraph(freq > 100, 'heavy')             # just these edges and the nodes they
 ↪  join
use('main')                               # switch back to the first graph
renameGraph('office')                     # rename the current graph
newGraph('scratch')                       # an empty graph
graphNames()
use('office')
closeGraph('scratch')                     # (not the current one)
```

Clicking a tab switches to it as well. Commands always apply to the current graph, and `g` always refers to it.

11. States and animations

A **state** is a snapshot of the graph: which nodes and edges exist, and every field value. Save one with `ss`, change things, and load it again with `ls` or animate back to it with `morph`:

```
ss('before')
colorize(dept)
circleLayout()
morph('before', 2000)       # animate back over 2 seconds
ls('before')                # jump back
getStates()
```

During a morph, nodes and edges that only exist in one of the two states fade in or out. `setNodeDisappearBy(.2)`, `setNodeAppearAfter(.8)`, `setEdgeDisappearBy(…)` and `setEdgeAppearAfter(…)` say when, as a fraction of the morph.

States can be saved under numbers too (`ss(1999)`, `ss(2000)`), which suits a series of snapshots over time.

To try something and put everything back, use `with scratch():`:

```
with scratch():
    remove(dept == 'dept1')
    print(len(g.nodes), graphStats())
```

When the block ends, the graph is as it was: the same nodes, edges and values, and fields made inside the block are gone. Print or keep in variables what you want to keep.

11.1 State-sensitive queries

A field subscripted with a state reads the values in that state:

```
ss('before')
(freq > 100).color = red
freq['before'] > 100              # the edges that had freq > 100 in 'before'
v1['before'].color                # v1's colour then
g.deleted                         # what's been removed
weight['_deleted'] > 5            # the 2006 form: deleted edges with weight > 5
```

Saved states can't be changed: `v1['before'].x = 20` is an error.

11.2 Ranges and time

A state is a whole snapshot. When nodes and edges simply come and go over time, **ranges** are lighter. Give each element the times it exists:

```
addRange(v1, "1–5")
addRange(v2, "3,7–9")
Node rcontains 4                # nodes whose range contains 4
Node roverlaps (2, 5)           # nodes whose range overlaps 2..5
Node rcontained (0, 10)         # nodes whose range lies inside 0..10
getRange(v1)
```

Ranges are closed intervals. “3-5” includes 3 and 5. If your nodes have a text field called `range` holding ranges like “1-5” (or another text field you name with `Node.setRangeField(when)`), the range operators use it for nodes without `addRange`.

Many graphs instead have a number for when each node appeared, such as a year. `showTime` shows the graph at a moment. Let’s give the sample graph a (made-up) year for each person:

```
addNodeField('year', INTEGER, 0)
for n in g.nodes: n.year = 1990 + int(n.name[1:]) % 20
showTime(1995, year)              # nodes with year <= 1995 (and their edges)
showTime(1995, year, 5)           # only the last five years
showTime(3)                       # by ranges: only what exists at time 3
```

The **timeline** (*View* → *Timeline*) gives you a slider and a play button for the same thing. It finds number fields that look like time and plays through them.

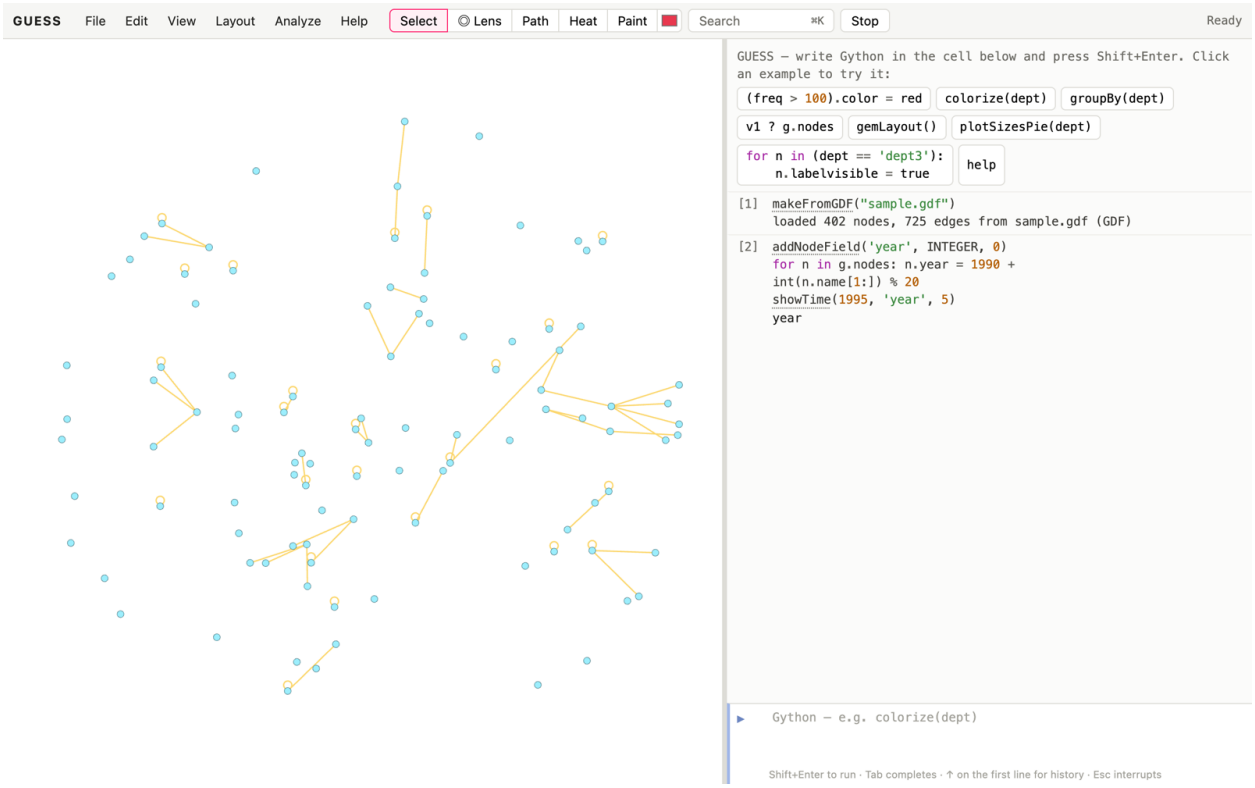


Figure 14: showTime(1995, ‘year’, 5): only nodes from 1991–1995 are shown.

12. Legends, hulls and the display

12.1 Legends

`legendFor(field)` makes a legend for the current colours of a field. `GradientLegend(start, end, min, max)` makes a colour ramp, and a fifth argument adds tick marks. For full control, build one yourself:

```
lg = Legend()
lg.add(v1, "a node")            # a node (as a prototype), a label
lg.add(sortBy(freq)[-1], "busiest")   # edges work too
lg.add(red, "important")        # or just a colour
```

Legends appear at the top right of the display, and they're included in exports and reports.

12.2 Convex hulls

A convex hull shades the area around a group of nodes:

```
h = createConvexHull(dept == 'dept4', "200,200,255,90")
for d in ('dept4', 'dept6', 'dept9'):
    grp = dept == d
    createConvexHull(grp, averageColor(grp[0].color, white))
removeConvexHull(h)
removeConvexHulls()
```

Hulls follow their nodes when you move them or run a layout.

A group can also be marked by a box or an ellipse: `shape="box"` or `"ellipse"`, upright or, with `rotate=true`, turned to fit the group (the smallest box at any angle, and the ellipse around it). `encircle` does this with a turned ellipse and a colour of its own; `encircleBy` draws one shape per value of a field. The shape, rotation and padding can be changed later:

```
e = encircle(dept == 'dept4')                 # a turned ellipse
encircle(dept == 'dept6', "box")              # a turned box
createConvexHull(dept == 'dept9', shape="box", rotate=false)   # an upright box
e.shape = "box"
e.padding = 20
removeConvexHulls()
encircleBy(dept)                              # one ellipse per department
removeConvexHulls()
```

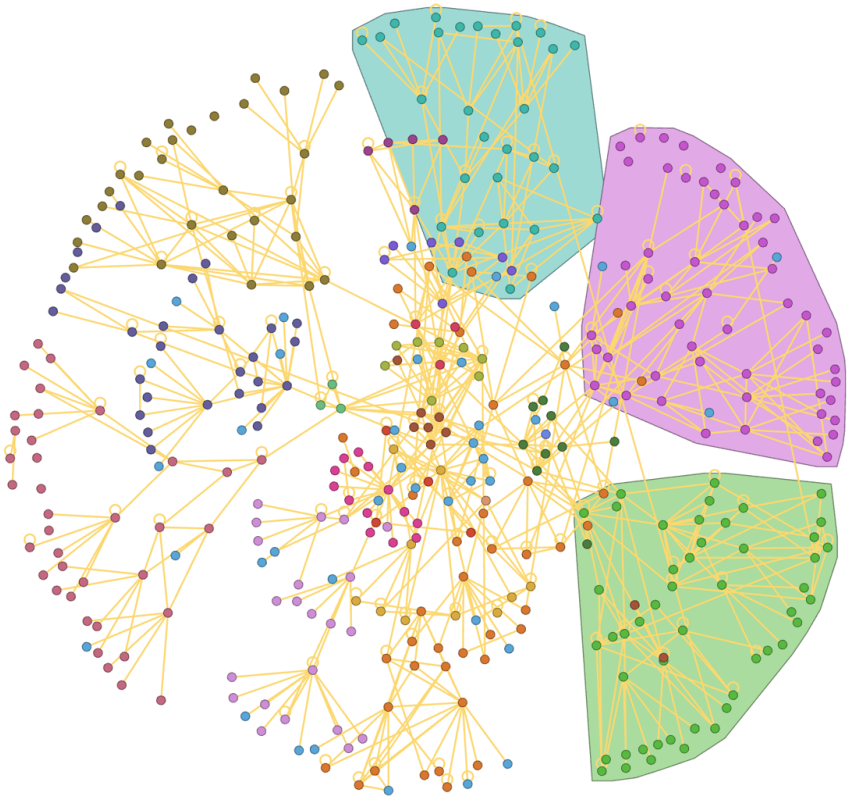


Figure 15: Hulls around three departments.

12.3 The display

- **Theme**: GUESS starts in *Dark*. *View* → *Theme* switches to *Light* or *Automatic* (follows your system), and remembers the choice. The graph's background follows the theme, unless you set one with `setDisplayBackground(black)`. `setDisplayBackground(None)` follows the theme again.
- **Edges**: *View* → *Curved edges* or `curvedEdges()`. `setEdgeColorMode("mixed")` draws each edge in the colours of its two ends. Parallel edges between the same two nodes are always drawn as separate curves.
- **Labels**: `labelOutlines()` draws labels with a halo so they read on any background. `hideOverlappingLabels()` drops labels that would collide, keeping the bigger nodes' labels. `edgeLabelsAlong()` turns edge labels to run along their edges.
- **Drawing order**: a node or edge whose colour, size, shape, label or visibility changes is drawn on top of the others (edges stay under nodes), so what you just changed isn't hidden. Moving a node doesn't raise it. `raiseChanged(false)` or *View* → *Bring changed elements to the front* keeps the order fixed.
- **Background image**: `setBackgroundImage("map.png", x, y)` puts an image under the graph (a map, say), and `removeBackgroundImage()` takes it away.

- **Arrows**: `overrideArrowLength(12)`, `overrideArrowWidth(6)`.

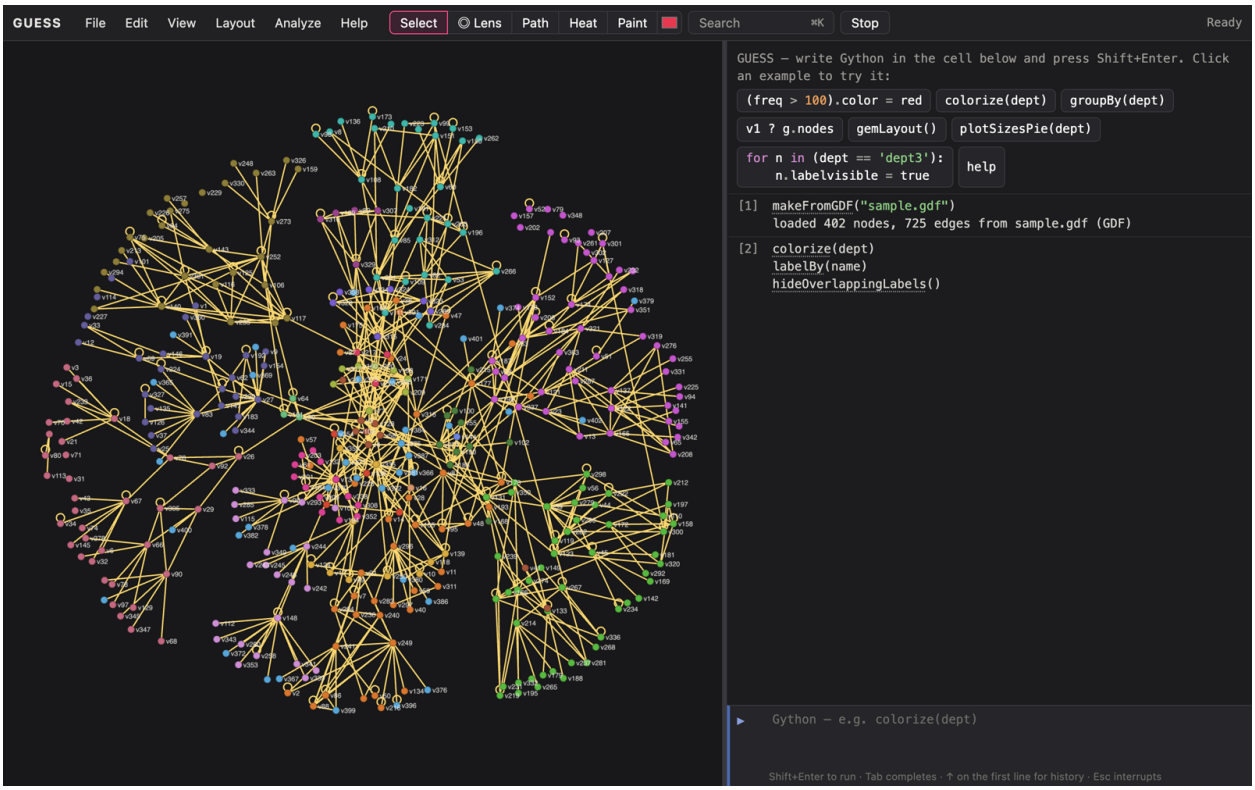


Figure 16: Dark theme, with colorize(dept), labelBy(name) and hideOverlappingLabels().

12.4 Labels inside nodes, positions and sizes

A node's `labelpos` says where its label goes. `inside` fits the label into the node: it wraps long labels onto several lines when the node is tall enough, shrinks the text down to about half size, and finally shortens it with "…". The text is black or white, whichever reads better on the fill (unless `labelcolor` is set). Styles 4–6 always put the label inside. `outside` lets GUESS choose a side for each label, bigger nodes first, avoiding other nodes and labels.

```
g.nodes.labelvisible = true
g.nodes.labelpos = "outside"                # a free side for each label
(dept == "dept3").labelpos = "inside"
fitNodesToLabels(dept == "dept3")           # or: grow nodes to fit their labels
labelSizeBy(pagerank, 0.8, 2)               # bigger labels for important nodes
for e in (freq > 150): e.label = str(e.freq)
(freq > 150).labelvisible = true
edgeLabelsAlong()
```

The Style panel sets the same things: map *Label* with a position, or *Label size* to a number.

Finding room for labels. Labels placed `outside`, and edge labels (which slide along their edge), are placed to avoid other labels, nodes and edges:

- `labelPlacement("candidates")` (the default) tries 16 positions around each node, biggest nodes first, then moves the labels that still collide.

- `labelPlacement("particles")` also searches further out, ring by ring, and draws a thin leader line to labels it places away from their node. It keeps labels off edges best.
- `optimizeLabels(2)` polishes the positions by simulated annealing for two seconds and keeps the result (until the graph is replaced or the method changes). The status bar reports how much it improved. Use it before exporting a figure.
- `constantLabelSize()` keeps labels the same size on screen as you zoom. Each label appears once there is room for it and stays as you zoom further in, so zooming in reveals more labels.

On the sample graph with every label shown, placing them `outside` cuts label overlaps by about two thirds and edges crossing labels by about the same; particles all but removes the edge crossings.

13. Direct manipulation

The mouse does more than pan and zoom. *Help* → *Mouse and gestures* shows all of this with pictures, and a tip appears the first time a gesture becomes useful.

13.1 Looking

gesture	does
drag the background	pan
wheel or pinch	zoom
double-click the background	fit the graph
hover a node or edge	its label at once, then (after a moment) all its fields; an edge rings its two endpoints
click a node	focus on it and its neighbours (click again or Esc to clear)
double-click a node or edge	zoom to it
hover a node or edge in the console output	highlight it in the graph; click to select and zoom to it
hover a node, a table row, a legend row or a chart bar	mark everywhere else it appears: console output, table rows, legend rows and chart bars

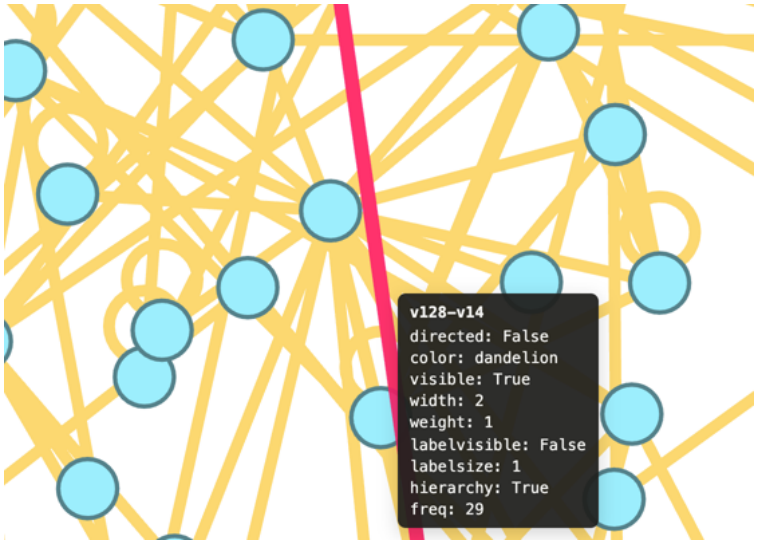


Figure 17: Hovering an edge rings both endpoints.

The tooltip comes in two stages: the label straight away, so it doesn't get in the way as you sweep across the graph, and every field (with the code that last changed the element) once you stay on it for a moment. To see some fields at once, pick them in *View* → *Hover fields*… or name them:

```
hoverFields(dept, degree)                # node fields; edge fields go to edges
hoverFields(nodes=["dept"], edges=["weight"])
hoverFields()                            # back to the label only
```

Hovering a hidden node or edge somewhere else (the console output, the table, a chart) draws a ghost of it in the graph, a dashed outline, so you can see where it is.

13.2 Selecting

gesture	does
Shift-drag	box select
Alt/Option-drag	lasso select
Cmd/Ctrl with a box or lasso	add to the selection
Shift-click a node	add it to (or remove it from) the selection
Esc, or click the background	clear the selection
right-click	a menu for the node, the edge, or the whole selection
Delete or Backspace	remove the selected nodes (also *Edit* → *Delete selected nodes*, or *Delete* in the right-click menu); undo brings them back

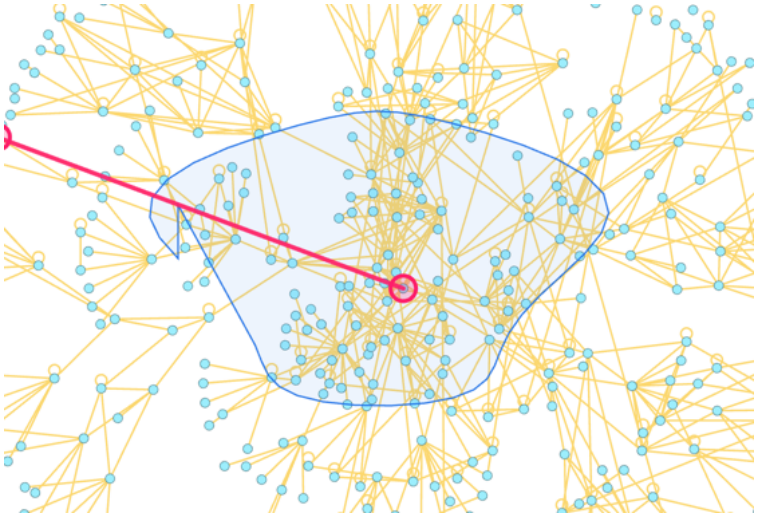


Figure 18: A lasso selection (Alt/Option-drag).

The selection is `sel` in the console (section 5.4). The right-click menu colours, sizes, labels, hides and removes, and *Assign to variable…* gives the selection a name you can use in code.

13.3 Dragging with meaning

Dragging a node moves it, and the selection moves with it if the node is selected. Each drag is recorded as an ordinary command, such as `v12.x = 310.5; v12.y = -42`. It is one undo step, and `history()`, `saveHistory()`, `log()` and notebook export include it, so a saved script puts the nodes back where you left them. To keep the console readable, these commands are hidden. *View* → *Show node moves in the console* shows them.

After some commands, a drag also *means* something. GUESS knows what the arrangement on the screen stands for, so it can turn your drag into the change you meant:

- **Reordering a circle.** After `circleLayout()`, nodes slide around the circle and the others make room. Dropping runs `ringMove(node, position)`.
- **Adding an edge.** Alt/Option-drag from one node to another adds an edge.

- **Editing values.** After `scatterLayout(x, y)`, dragging a node changes its two fields.

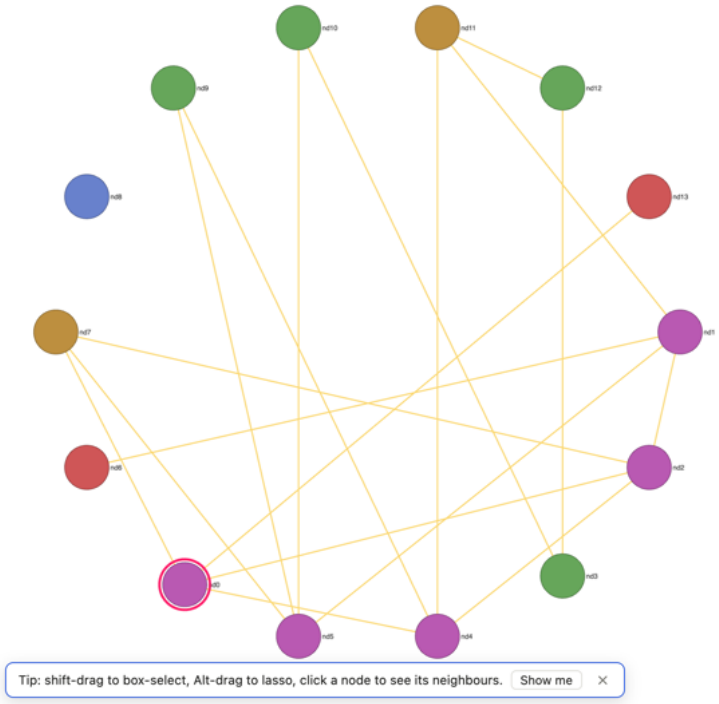


Figure 19: After circleLayout(), a dragged node slides around the ring and the others make room.

Each of these drops appears in the console as a visible command (for example `ringMove(v128, 4)`). So you can see what happened, undo it, or copy it into a script.

13.4 The click tools

The tools next to the menus turn clicks into commands:

- **Select** (the default) is everything above.
- **Path**: click two nodes to colour a shortest path between them.
- **Heat**: click a node for a heat map of the distance from it: red near it, through violet, to blue for the farthest (grey: unreachable).
- **Paint**: click nodes to paint them with the colour next to the tools. Shift-click paints the neighbours too.
- ◎ **Lens** (or press **L**, which switches between Select and Lens): an excentric labelling lens. A circle follows the pointer, and the labels of the nodes under it are listed beside it, each joined to its node by a line, in their nodes' colours (Fekete and Plaisant 1999, extended by Bertini, Rigamonti and Lalanne 2009). The wheel resizes it and Shift+wheel scrolls long lists. A bar chart under it counts the nodes by colour. Click to pin it: then click a label to select its node (Shift-click adds or removes), click the bars to show only some colours, and *Select N* selects everything shown. Esc releases it. Hover popups are off while the lens is on. `excentricLens(radius=80, autoSize=true)` turns it on from code; with `autoSize` it grows over sparse areas and shrinks over dense ones.

- **Cluster**: label clusters by hand. It first asks which field to use: `cluster`, or `cluster1`, `cluster2`, … when that is taken. The field is made if it doesn't exist. If you name one that already has values, it asks whether to clear it (`g.nodes.cluster = None`, undoable) or keep the values and add to them. Then every lasso you draw puts the nodes it catches in the next cluster: 1, then 2, and so on. Type in *next id* to choose a number or a name yourself (`a7` is followed by `a8`). Plain drag draws a lasso, shift-drag a box, and Space-drag pans. While the tool is on, labelled nodes show in their cluster's colour and the rest are grey. The bar counts the clusters, the labelled nodes and those left: hover *left* to highlight them, click it to zoom to them. A new lasso moves the nodes it catches to the new cluster, even if they had one. Untick *relabel* to make it skip labelled nodes, so quick lassos can overlap. Cmd/Ctrl-drag, or clicking a node, adds to the last cluster. *Colour by it* runs `colorize(cluster)`; *Done* or Esc ends the tool.

Each click runs a command in the console (`shortestPath(v1, v14)`, `heatMap(v3)`, `paint(v5, red)`), so you can edit it and run it again. The Cluster tool's lassos run `labelCluster(["v1", "v7", …], 3, "cluster")` as hidden cells, like node drags: they can be undone, and `history()` and `saveHistory()` include them.

13.5 Responding to clicks in code

You can also attach your own code to the mouse:

```
graphevents.clickNode = lambda n: center([n] + n.getNeighbors())
graphevents.mouseEnterNode = lambda n: setattr(n, 'labelvisible', true)
graphevents.mouseLeaveNode = lambda n: setattr(n, 'labelvisible', false)
```

The events are `clickNode`, `clickEdge`, `shiftClickNode`, `shiftClickEdge`, `mouseEnterNode`, `mouseLeaveNode`, `mouseEnterEdge` and `mouseLeaveEdge`. Setting one to `None` turns it off. `vf.defaultNodeHighlights(false)`, `vf.defaultNodeZooming(false)`, `vf.defaultEdgeHighlights(false)` and `vf.defaultEdgeZooming(false)` turn off the built-in hover highlight and double-click zoom. Do this when your own handlers should be the only thing that happens. `lastClick()` gives the last place clicked, as `(x, y)` in graph coordinates.

You can add items to the right-click menu, too:

```
def paintRed(n):
    n.color = red

item = NodeEditorPopup.addItem("Paint red")
item.menuEvent = paintRed
```

`EdgeEditorPopup` adds to the edge menu and `GraphElementPopup` to both. When several things are selected, the function receives the selection.

14. The console, history and undo

key / action	does
Shift+Enter, Cmd/Ctrl+Enter	run the cell
Enter	a new line
Tab	complete commands, fields, nodes and attributes
↑ / ↓	move between the lines of the code; ↑ on the first line recalls the previous command, ↓ on the last line goes forward again, back to your unsaved code (kept, as are edits to recalled commands)
Esc, or *Stop*	interrupt running code
Ctrl+L	clear the output
Shift+Cmd/Ctrl+F	find in the console output
Edit on a cell (or double-click its code)	copy it into the working cell
Run again on a cell	run it again, as a new cell at the bottom
Watch on a cell	re-run it whenever the graph changes, or when you select nodes on the display (if the code uses `sel`)
Pin on a watched cell	show its live output in a side panel, which stays in view as the console scrolls; close the panel to put it back
`[n]`	fold or unfold a cell's output; hover it to highlight everything that cell changed
`;` at the end of a line	don't print its value (`louvainClusters();`)

View → Vim keys in the code cell switches the editor to vim bindings.

Long results print compactly: a set of many groups (`groupBy(dept)`, the communities from `louvainClusters()`) shows as `21 groups (402 elements): [dept0: 28] [dept1: 43] …`, each entry linked to its members.

Code and graph, linked. The console and the graph point at each other:

- **Output.** Nodes, edges, sets, dicts, f-strings and `dept.distinct` values in the output highlight what they stand for. Hover `'dept1'` in `dept.distinct` to see the nodes in dept1. An edge prints as `v1–v2` (`v1→v2` when directed): hover either name for that node, or the dash or arrow between them for the edge.
- **Executed code.** Hover a name or a query in code you've run to see what it means *now*: a variable's members, the matches of `degree > 5` (or of the whole of `3 < degree < 6`), a node, or a value. Hovering a field shows its values and previews colouring by it; nothing is changed. Commands show their documentation.
- **Reverse highlighting.** Hovering a node in the graph, a table row, a legend row or a chart bar marks the output that mentions it.
- **Which code changed it.** A node's tooltip lists the cells that last changed it (`color ← [12] colorize(dept)`). Its right-click menu has *Show the code that changed it*.
- **Into your code.** Alt-click a node in the graph, a table row, a legend row, a chart bar or anything in the output to insert it at the cursor: its name, a query, `sel`, or a short tuple. A larger set is bound to a new variable, `picked1`. The right-click menu has *Insert as code*.
- **Earlier graphs.** Output from an earlier graph (before a new file was opened, or another graph tab) is greyed out and no longer linked, since its elements are gone.

Undo. Every command is one step: a layout, a file load, `colorize`, a node drag, a table edit. *Edit*

→ *Undo*, Cmd/Ctrl+Z (when the code cell isn't focused) or `undo()` reverses the last step, up to 50 steps back. Redo is Shift+Cmd/Ctrl+Z, Ctrl+Y or `redo()`. Undone cells are struck through until you redo them. Undo covers the graph, the fields, states, ranges, hulls, legends, styles and the display. It doesn't cover Python variables you made, or files you've downloaded.

Edit → *History…* opens the undo history as a list with a slider. Drag the slider, or click a step, to undo or redo to that point; the graph changes as you go, and each step you cross is highlighted. Hovering a step highlights the nodes and edges it changed, and its `[n]` scrolls the console to the cell. Running a new command after going back discards the steps after it, as with `undo()`.

History. The cells on the screen are the session's history, in the order you ran them. `history()` lists them and `saveHistory("work.py")` downloads them as a script that rebuilds your work. Errors have a red bar, and the line that failed is underlined. Nodes and edges an error names are links, like printed output: hover to see them, click to select them.

15. Modifying the interface / expanding GUESS

GUESS can be extended in Gython itself: new commands, panels of controls, and handlers for the mouse. In 2006 this meant Java Swing. Now it's a few small Python classes.

15.1 Example 1: a simple button

```
def bigger():
    g.nodes.size = 20

b = Button("Bigger nodes", onClick=bigger)
ui.dock(Panel("Tools", b))
```

`Panel(title, *widgets)` makes a panel, and `ui.dock(panel)` docks it below the graph (`position="right"` docks it to the right). The widgets are `Button`, `Label`, `Slider`, `Checkbox`, `Dropdown` and `TextField`.

15.2 Example 2: a threshold slider

This slider hides edges whose `freq` is below the threshold, and a label counts what's left:

```
lab = Label('')
def hideshow(val):
    g.edges.visible = true
    (freq < val).visible = false
    lab.text = '%d edges visible' % len(Edge.visible == true)

s = Slider(freq.min, freq.max, value=100, label='Minimum frequency',
 ↪  onChange=hideshow)
ui.dock(Panel('Threshold', s, lab))
hideshow(100)
```

`onChange` runs when you let go of the slider. `onSlide` runs continuously while you drag. (`Edge.visible == true` is a query on the edge field `visible`. `Node` and `Edge` pick which kind of field when the name is shared.)

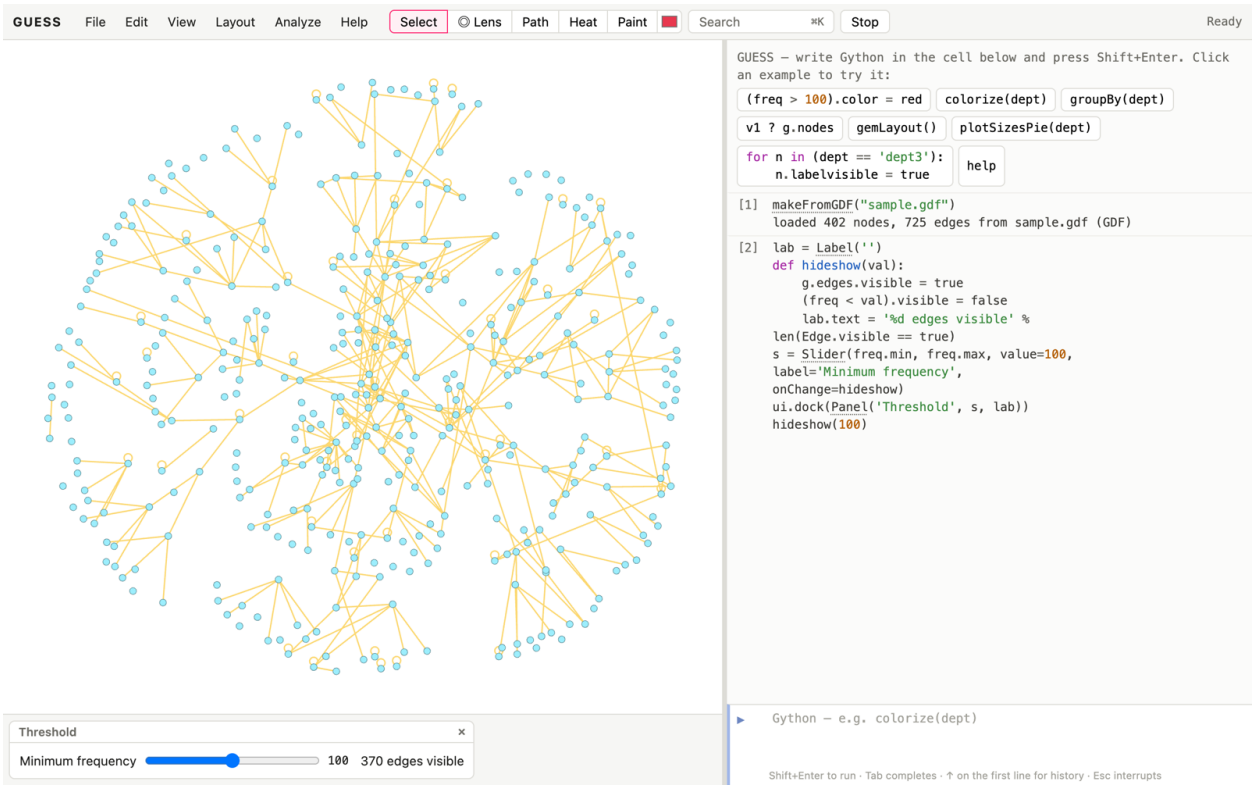


Figure 20: The threshold slider docked below the graph.

15.3 Example 3: a network monitor

A timer runs code every few seconds, for instance to poll for new data:

```
def tick():
    v = randomColor()
    (dept == 'dept3').color = v

t = every(2, tick)        # every 2 seconds
t.stop()                  # or stopTimers() for all of them
after(5, gemLayout)       # once, in 5 seconds
```

A timer's code runs between your commands, so it never interrupts one. A handler that takes longer than its interval doesn't pile up behind itself.

15.4 Example 4: an app

`exportApp` packages the current session with a script that builds its interface:

```
exportApp("threshold.guess", "ui.py")
```

Opening `index.html?load=threshold.guess&app=1` (with the file on the same server) shows the graph and its panels without the console. *File → Share as viewer* does the same thing as a single link. This replaces the applets of 2006.

15.5 Code bits

- `execfile("file.py")` runs a script you've opened.
- `openURL(url)` opens a web page.
- `installPackage("name")` installs a pure-Python package from PyPI into the page, so you can `import` it.
- Presentations: `addSlide("note")` saves the current view as a slide, and *View* → *Present* steps through them with the arrow keys. `showSlide(2)`, `moveSlide(…)`, `noteSlide(…)` and `removeSlide(…)` edit them.

16. Sharing and deploying

16.1 Sessions

`saveSession("work.guess")` (*File* → *Save*) saves everything about the graph: the nodes, edges and fields, the states, styles, legends, hulls, slides, groups, the other graph tabs and the view. Opening the file restores it. Python variables aren't saved. If you need them, save your history as a script.

16.2 Links

File → *Share link* puts the whole session, compressed, into a URL. Nothing is uploaded: the graph travels in the link itself, so links to big graphs are long. *Share as viewer* adds `&app=1`, which hides the console.

16.3 Notebooks and reports

File → *Export notebook* writes a Jupyter notebook of your commands. *File* → *Export report* writes a self-contained HTML page with the commands, their output and a picture of the graph. Use it to send someone what you found.

16.4 Deploying GUESS

GUESS is a set of static files, so any web server can host it. From a copy of the source:

```
cd web
npm install
npm run bundle              # web/bundle/guess-web-<version>/ and .zip, .tar.gz
npm run bundle -- --offline   # the same, with the Python runtime included
```

Copy the contents of the `guess-web-<version>` folder (or unzip the archive) to the server. The site includes this manual under `manuals/`, which *Help* → *Manual* opens. A normal bundle loads the Python runtime from a CDN the first time it runs. An `--offline` bundle carries it (about 25 MB more), so it works with no internet access. The server must send two headers (`Cross-Origin-Opener-Policy` and `Cross-Origin-Embedder-Policy`) for the *Stop* button to interrupt running code. The bundle includes a fallback for servers that can't. `docs/DEPLOY.md` has ready-made settings for Apache, nginx, Caddy, Netlify and GitHub Pages.

To open a graph straight away, link to `index.html?load=mygraph.gdf`, with the file next to `index.html`. Add `&app=1` to hide the console.

16.5 Working with an AI assistant

An AI assistant that speaks the Model Context Protocol (Claude Desktop, Claude Code and others) can work in GUESS with you. It runs Gython in your session, reads a summary of the graph and the command help, and looks at the view. Everything it runs appears in the console as a cell marked **assistant**, so you see each step and can undo it.

The bridge is `mcp/guess-mcp.mjs` in the GUESS source, a small Node program with nothing to install. Add it to your assistant's MCP settings, for example for Claude Desktop (`claude_desktop_config.json`):

```
{"mcpServers": {"guess": {"command": "node", "args":
 ↪  ["/path/to/guess/mcp/guess-mcp.mjs"]}}}
```

or for Claude Code: `claude mcp add guess -- node /path/to/guess/mcp/guess-mcp.mjs`. Then, in GUESS, choose *File* → *Connect AI assistant* or run `connectAssistant()`; ● *assistant* in the toolbar shows the connection, and clicking it (or `disconnectAssistant()`) ends it.

The bridge listens only on your own computer (127.0.0.1, port 7777, or `GUESS_MCP_PORT`) and accepts only GUESS pages: from `localhost` by default, or the addresses you list in `GUESS_ORIGINS` (for a GUESS you host, `GUESS_ORIGINS=https://guess.example.org`). The assistant's own service sees what its tools return: command output, the graph summary and screenshots.

Appendix A. Colors

Colours can be given by name (`red`, `cornflowerblue`), as `"r,g,b"` or `"r,g,b,a"` strings (0–255), or as `"#rrggbb"`. The names are variables, so `v1.color = wildstrawberry` needs no quotes. `averageColor(c1, c2)`, `generateColors(start, end, n)`, `distinctColors(count)` and `randomColor()` make new colours, and every colour has variations: `red.alpha(80)` (or `red.alpha(0.3)`) is partly transparent, `red.lighter(0.3)` is 30% of the way to white, and `red.darker(0.3)` 30% of the way to black. Colours compare by the colour they mean: `v1.color == red` holds whether the field says `red`, `"255,0,0"` or `"#ff0000"`, so `color == red` finds every red node.

The named colours from 2006 are below (case doesn't matter). The standard web colour names work as well.

name	r, g, b	name	r, g, b	name	r, g, b
apricot	251, 213, 184	lavender	251, 190, 254	processblue	46, 253, 253
aquamarine	115, 253, 217	lfadedgreen	241, 253, 230	purple	191, 106, 253
bittersweet	223, 45, 2	lightcyan	227, 253, 254	rawsienna	194, 1, 1
black	0, 0, 0	lightgray	192, 192, 192	red	255, 0, 0
blue	0, 0, 255	lightgreen	228, 253, 216	redorange	252, 130, 101
bluegreen	106, 253, 212	lightmagenta	251, 229, 254	redviolet	197, 13, 211
blueviolet	79, 66, 249	lightorange	252, 229, 216	rhodamine	250, 118, 253
brickred	218, 1, 2	lightpurple	227, 229, 253	royalblue	0, 187, 253
brown	168, 0, 1	lightyellow	252, 252, 202	royalpurple	130, 91, 253
burntorange	253, 184, 7	limegreen	188, 252, 9	rubinered	250, 14, 239
cadetblue	161, 174, 226	lskyblue	234, 247, 254	salmon	252, 181, 205
canary	251, 252, 187	magenta	255, 0, 255	seagreen	150, 253, 187
carnationpink	250, 163, 253	mahogany	209, 0, 2	sepia	147, 0, 1
cerulean	61, 240, 253	maroon	213, 0, 31	skyblue	162, 253, 240
cornflowerblue	156, 238, 253	melon	252, 192, 187	springgreen	221, 252, 134
cyan	0, 255, 255	midnightblue	0, 176, 198	tan	236, 198, 176
dandelion	252, 216, 112	mulberry	204, 82, 251	tealblue	95, 250, 208
darkgray	64, 64, 64	navyblue	57, 180, 254	thistle	237, 170, 253
darkorchid	199, 124, 230	olivegreen	13, 201, 7	turquoise	104, 253, 230
emerald	0, 253, 187	orange	255, 200, 0	violet	119, 99, 253
forestgreen	39, 239, 34	orangered	251, 11, 187	violetred	250, 121, 253
fuchsia	174, 47, 244	orchid	210, 161, 253	white	255, 255, 255
goldenrod	253, 241, 112	peach	252, 186, 148	wildstrawberry	251, 59, 203
gray	128, 128, 128	periwinkle	170, 178, 254	yellow	255, 255, 0
green	0, 255, 0	pinegreen	8, 222, 111	yellowgreen	196, 252, 139
greenyellow	236, 252, 151	pink	255, 175, 175	yelloworange	253, 198, 7
junglegreen	23, 253, 184	plum	182, 18, 253		

Appendix B. Gython and Python 3

Gython is Python 3 with these additions:

addition	meaning
`field == value` (and `!=`, `<`, `>`, `<=`, `>=`, `like`) `3 < field < 6`, `field in (a, b)`	a query: the set of matching nodes or edges the matching set (Python would give a bool or the wrong set)

addition	meaning
`set.field = value`	sets the field on every member
`set.field.avg` (`.sum`, `.min`, `.max`, `.median`)	summaries of the members' values
`a & b`, `a | b`; `a and b`, `a or b`, `not a`	intersection, union and complement of sets (in an `if` test, `and`/`or`/`not` are plain Python)
`print(f"... {node} ...")`	the nodes and edges in an f-string stay linked in the output
`->`, `<-`, `<->`, `-`, `?`	edges between nodes or sets
`Node rcontains x`, `roverlaps`, `rcontained`	range queries
`field['state']`	values in a saved state
`true`, `false`	the same as `True`, `False`
a command name on its own (`center`)	calls it
node names, field names, commands, colours	predefined, protected variables

Command conventions. - **Fields.** An argument that takes a field also takes its name: `sizeBy(degree)` and `sizeBy("degree")` are the same. A command with one field argument calls it `field`; one with several names them by role (`scatterLayout(x, y)`). - **Ranges.** Sizes run from `smallest` to `largest`, colours from `start` to `end`. - **Counts.** A number of things to make or return is `count`, and a node argument is `node`. - **Switching things off.** A display mode takes `on=` (`curvedEdges(false)`); something drawn on top is removed with a matching `remove…` command (`removePies()`, `removeBundles()`, `removeConvexHulls()`, `removeLegends()`, `removeMap()`). - **`name=`** always names what the command creates: a field, a graph or a node.

Names. GUESS's own names (commands, colours, fields, node names, `g`) are protected, and assigning to one is an error that says what the name is. Three exceptions: - an explicit import replaces a command for the session (`from math import log`; `del log` brings the command back); - a `for` loop variable may reuse a node's name for that cell; - nodes and fields named after a Python builtin (`len`, `type`) don't hide it: use `g.getNodeByName('len')` or `Node.type`.

`a - b` between node sets is the 2006 edge operator (the edges between them, like `a <-> b`), and `a -= b` does the same. For the nodes in `a` but not in `b`, use `a.difference(b)` or `a & (not b)`.

Things to know if you're coming from the 2006 version (Jython, i.e. Python 2):

- `print` is a function: `print(v1)`, not `print v1`.
- `/` always divides exactly (`7/2` is 3.5). `//` is integer division.
- `except (A, B) as e`, not `except (A, B), e`.
- Strings are Unicode. There's no `unicode()`, and `str()` is all you need.
- Java isn't there: `import javax.swing` and friends don't exist (see section 15 for the widgets that replace them). Neither does the R interface.

Appendix C. Changes from the 2006 version

Nearly everything in the 2006 manual still works, and its examples run unchanged except where Python 3 differs. The main differences:

- **Platform.** A web page instead of a Java application. No installation, no database files, no start-up dialogs. Work is autosaved in the browser, and `.guess` files hold sessions.
- **The console** is made of cells, with history, completion, undo and help.
- **Mouse.** Clicking a node focuses its neighbourhood instead of centring on it. Shift-drag selects instead of zooming. Zoom is the wheel or a pinch, not a right-drag. The five tool buttons

(node, edge, hull and annotation editing) are replaced by the click tools and the drag gestures of section 13.

- **Output.** SWF, Java, CGM and EMF export are gone. Use SVG or PDF. Movies are WebM instead of QuickTime.
- **Java-only names are gone (2026).** `Types.INTEGER` and `Integer(0)`: use `INTEGER` and `0`. `Morpher.setNodeDisappearBy(…)`: use `setNodeDisappearBy(…)`. `InfoWindow.details(x)` and `infowindow()`: use `table(x)` and `table()`. `getAppletContext().showDocument(url)`: use `openURL(url)`. The display object `v`: use `setBackgroundImage(…)`, `center(…)` and `lastClick()` (for `v.getLastClickedPosition()`); `v.repaint()` isn't needed. `readjustEdges()` and `setSynchronous()` did nothing and are gone too.
- **Interface building** uses `Panel`, `Button`, `Slider` and timers instead of Java Swing classes, threads and sockets.
- **Not available**: the R interface, applet signing and `jythonc`, the Prefuse and TouchGraph front ends, and command-line options. These are replaced by links and apps (section 16).
- **Multiple edges.** Any number of edges is allowed between two nodes, not just three.
- **Defaults.** Nodes are cornflowerblue ellipses and edges are dandelion, as the 2006 code had them (the 2006 manual says blue and green).
- **Names made consistent (2026).**
  - `pieOff()` is now `removePies()`, and `unbundleEdges()` is `removeBundles()`. `biComponentCluster()` is gone: use `biComponentClusters()`.
  - `addNodeField(name, type, default)` and `addEdgeField(...)` (were `fieldName`, `sqlType`, `defaultValue`).
  - `resizeLinear(field, smallest, largest)` and `resizeRandom(...)` (were `start`, `end`).
  - `springLayout(iterations=...)` and the other force layouts (was `max`).
  - `moveLayout(x, y)` and `translateLayout(dx, dy)` (were `newX`, `newY`).
  - Keyword arguments named `fld`, `kindField`, `typeField`, `seedField` and `lenField` are now `field`, `scatterLayout(xfield, yfield)` is `scatterLayout(x, y)`, `minimum=` (motifs, `collapseBy`) is `smallest=`, and `k` or `n` (a number of groups or colours) is `count`.

  These only matter when an argument is given by name, as in `resizeLinear(degree, start=5, end=30)`.
- **Louvain's field** is `louvain_class`, like `leiden_class` and the other methods' (it was `modularity_class`, Gephi's name).
- **Language.** `3 < degree < 6`, `field in (...)`, and `and`, `or` and `not` on sets now do what they say (Appendix B). A node or field named after a Python builtin (`len`, `type`) no longer hides it.

`docs/MANUAL_COMPARISON.md` goes through the 2006 manual section by section.

Appendix D. Command reference

Every command, generated from the code's own documentation (the same text as `help(name)` and the Help panel).

Graph structure, loading and saving, mapping, view

`add(seq)`

Re-add removed nodes/edges, e.g. add(g.deleted).

`addDirectedEdge(node1, node2, **attrs)`

Add a directed edge from node1 to node2.

`addEdge(node1, node2, **attrs)`

Add an undirected edge.

`addEdgeField(name, type='VARCHAR', default=None)`

Create an edge field, e.g. addEdgeField("kind", VARCHAR, "").

`addNode(name, **attrs)`

Add a node (or return the existing one with that name). addNode(vx) with a removed node puts it back, with its field values.

`addNodeField(name, type='VARCHAR', default=None)`

Create a node field, e.g. addNodeField("score", DOUBLE, 0).

`addUndirectedEdge(node1, node2, **attrs)`

Add an undirected edge between two nodes (by node or name).

`averageEdgeColor(edges=None)`

Color edges the average of their endpoint colors (all edges by default).

`center(*d)`

center() fits the whole graph; center(seq) zooms to the given elements.

`collapseMotifs(kind='fan', smallest=3)`

collapseMotifs("fan") collapses every motif found by findMotifs into one meta-node (labelled with the kind and its size), which simplifies the drawing without losing anything: expand(node) or expandAll() bring the members back. Returns the meta-nodes.

colorBy(field, start, end, scale='linear', middle=None, center=None)

colorBy(field, startColor, endColor) colours elements along a gradient in proportion to a numeric field's value; scale may be "linear", "log", "sqrt" or "rank"; middle=color adds a midpoint colour. center=value makes a diverging gradient: that value gets the middle colour (white unless given) and the ends are equally far from it, so equal changes up and down look equally strong, e.g. colorBy(lfc, blue, red, center=0) for fold changes. Compare colorize, which spaces distinct values evenly.

colorize(field, *cs)

colorize(field) gives each value its own clearly different colour (the largest group first; see distinctColors); colorize(field, start, end) or colorize(field, start, middle, end) a gradient over the sorted values.

complement(seq)

Everything (nodes if seq has nodes, edges if it has edges) not in seq.

condenseGraph()

removeDisconnected, removeSelfLoops, then alternate removeLeaves and shortcutNodes until nothing changes. Returns [removed, added].

copyNode(template, name, **extra)

Create node `name` with all field values copied from `template`.

demo(step=1)

demo() starts a guided tour of GUESS for newcomers, from selecting nodes to communities, what-if analysis, Altair charts, a contagion model and a small app. Each step explains an idea and runs its code as a normal console cell. demo(9) starts at step 9; demo("advanced") at the first step of that level (basic, intermediate, advanced).

density()

Edges divided by the number of possible undirected node pairs.

distance(node1, node2)

Euclidean distance between two nodes' positions.

edgeData(*args)

edgeData(field, …) or edgeData(edges, field, …): one row per edge (`name` is "a-b"), with `_edge`, the edge's id, for linking.

effectiveDistance(origin, weight='weight', name='eff_distance')

effectiveDistance(origin) measures how 'close' every node is to the origin through a travel network, as an epidemic would see it (Brockmann and Helbing, Science 2013). From node m, the share of m's traffic (weight) that goes to n is P; the step m -> n costs 1 - ln P, so busy routes are short and a route among many is long. Stores the shortest total in `eff_distance` (name=...; unreachable nodes get None) and marks the edges of the shortest-path tree in the edge field `eff_tree`. Arrival times of real epidemics grow almost linearly with it. Returns the field.

einverse(seq)

All edges not in seq.

embed(dims=8, walks=10, length=40, window=5, p=1.0, q=1.0, seed=0, name='emb')

embed(dims=8) gives every node coordinates in `dims` dimensions such that nodes in similar network positions are close (node2vec, Grover and Leskovec 2016): random walks from every node (`walks` of `length` steps; p and q bias them towards returning or exploring), then a factorisation of how often nodes co-occur within `window` steps. The coordinates are stored in fields emb0, emb1, ... (name=...), ready for scatterLayout(emb0, emb1), similarity searches or a classifier (toNetworkX(), or export the table). Fine up to a few thousand nodes.

enrichment(seq, sets, background=None, top=10, minSize=2, show=True)

enrichment(nodes, sets) tests which sets (from readSets, or any {name: [node names]}) are over-represented among the nodes, against the background (default: every node in the graph that is in at least one set). For each set it gives the overlap k of n nodes, the set's size K among the N background nodes, the fold enrichment (k/n)/(K/N), the hypergeometric p-value and the Benjamini-Hochberg q-value (the false discovery rate over all sets tested). Sets with fewer than minSize background members are skipped. Prints the `top` rows by p-value and returns all rows as dicts, best first.

execfile(filename)

Run a Gython script file in the current namespace.

exportCSV(filename='edges.csv', table='edges')

exportCSV("edges.csv") writes the edge table (Source, Target, Type, Weight, Label and edge fields); exportCSV("nodes.csv", "nodes") the node table.

exportDOT(filename='graph.dot', clusters='cluster', layout=True)

exportDOT("graph.dot") writes the graph for Graphviz: positions (fixed nodes pinned with !), sizes, shapes, colours, labels, hidden elements and every user field; nodes are grouped into cluster subgraphs by the field named by clusters= (default `cluster`, e.g. clusters="louvain_class"). layout=False leaves positions out so Graphviz lays the graph out itself. makeFromDOT reads it back. Offered as a download.

`exportGDF(filename)`

Write the graph to a GDF file (offered as a download in the browser).

`exportGEXF(filename='graph.gexf')`

Write the graph as GEXF 1.3 for Gephi (offered as a download).

`exportGML(filename)`

Write the graph as GML (networkx, igraph, yEd, Cytoscape and Gephi read it): fields, positions, colours and sizes. Offered as a download.

`exportGraphML(filename)`

Write the graph as GraphML (offered as a download in the browser).

`extinctions(orders=('random', 'degree'), trials=10, seed=0, show=True)`

extinctions() simulates species loss in a food web: species are removed one at a time ("primary" extinctions) and any species left with nothing to eat dies out too ("secondary" extinctions; species that eat nothing in the web, the basal ones, never starve). orders: "random" (averaged over `trials`), "degree" (most connected first), "least" (least connected first), "basal" (producers first) or a node field (highest first). Plots the share of species surviving against the share removed, and prints R50, the share of primary removals that brings total losses to half the species (higher = more robust). The graph isn't changed. Returns {order: [(removed, surviving)…]}.

`findCycles(maxLength=5, among=None, limit=1000)`

findCycles(maxLength=5) finds directed cycles (A -> B -> … -> A) of up to maxLength nodes, e.g. money sent round a ring of accounts back to where it started. among=nodes restricts the search. Each cycle is a set of its nodes and edges (shortest first); at most `limit` are returned. Undirected edges count both ways.

`findDuplicates(field=None, threshold=0.85, among=None, top=50, show=True)`

findDuplicates(name) lists pairs of nodes whose values of a text field look like the same entity: compared after normalising (case, accents, punctuation, word order, company suffixes and titles like Ltd, Inc, Mr), scored 0..1 by string similarity, best first, at least `threshold`. among=nodes restricts the search (e.g. one kind of entity). Default field: label. Returns (node, node, score) for review and mergeNodes; it never merges anything itself.

`findEdges(seq)`

The edges among the members of a (possibly mixed) set.

findMotifs(kind='fan', smallest=3, show=True)

findMotifs("fan") finds NodeXL-style motifs, the small repeated shapes that clutter big drawings: "fan" several leaves (nodes with one neighbour) on the same node "connector" several nodes linked to exactly the same 2-4 others and nothing else (e.g. people who all know the same two) "clique" groups where everyone is linked to everyone (each node in at most one, largest first) with at least `smallest` members. Returns the groups (members only); collapseMotifs(kind) replaces each with one meta-node.

findNodes(seq)

The nodes among the members of a (possibly mixed) set.

fitNodesToLabels(seq=None, padding=4)

fitNodesToLabels() puts each node's label inside it (labelpos = "inside") and resizes the node to fit the label, as in a flowchart or a Graphviz drawing; fitNodesToLabels(dept == 'dept3') does some nodes. padding is the space around the text. Labels with line breaks (`\n`) make taller nodes.

floatRange(a, b, inc)

Like range() but for floats.

geoDistance(name='km', lat=None, lon=None)

geoDistance() stores each edge's great-circle length in kilometres (from its ends' latitude and longitude fields, as geoLayout finds them) in the edge field `km` (or name=...). Use it as a cost: for e in g.edges: e.weight = e.km, then shortestPath(a, b, weighted=true) is the shortest route by distance. Returns the field.

geoLayout(lat=None, lon=None, projection='mercator', map='auto', width=1000, land=None, borders=None, parallels=(29.5, 45.5), center=(-96.0, 37.5))

geoLayout() places nodes by their latitude and longitude fields (found by name: latitude/lat and longitude/lon/lng, or pass them: geoLayout(lat, lon)) and draws a map underneath. projection: "mercator" (default; shapes kept, high latitudes enlarged), "equirectangular" (longitude and latitude as x and y) or "albers" (equal-area conic; with parallels= and center=, US by default, which suits the lower 48 states). map: "world" (land outlines), "usa" (the states), None for no map, or "auto" (usa when every node is in the US, else world). land= and borders= set the map's colours (default: light or dark grey to suit the theme). The layout fits nodes into `width` units across; nodes without coordinates are set in a row below. Outlines: Natural Earth 1:110m, public domain.

getEdgeSchema()

The edge fields.

getNodeSchema()

The node fields.

groupAndSortBy(*d)

Like groupBy, with the groups ordered by value.

groupBy(*d)

groupBy(field) or groupBy(seq, field): groups of elements sharing a value.

groupStats(field, top=12, show=True)

groupStats(louvain_class) profiles each group of nodes sharing a value of a field: size, internal edges, density (internal edges / possible pairs), boundary edges (to the rest), conductance (boundary / all edges touching the group: low = well separated), and the group's most connected member. Prints the largest `top` groups and returns a row per group, largest first; a dense, isolated group of entities that should be unrelated is a classic red flag.

help(obj=None)

help() opens the help browser (searchable commands and examples; also F1 or Cmd/Ctrl+K for the command palette); help(command) prints that command's documentation and shows it in the browser.

hideDisconnected()

Hide nodes with no edges other than self loops.

highest(field, count=10, seq=None)

highest(degree) is the 10 nodes (or edges) with the largest degree, largest first; highest(degree, 5), or highest(degree, 5, seq) within a set. Elements without a value are left out.

labelCluster(nodes, value, field='cluster', relabel=True)

labelCluster(nodes, value, field="cluster") puts `value` in `field` for each of the nodes (a set, or a list of node names), making the field if needed: INTEGER for a number, VARCHAR for text; nodes not labelled hold None. relabel=False leaves nodes that already have a value alone. The Cluster tool writes one of these per selection. Returns the nodes labelled.

labelSizeBy(field, smallest=0.8, largest=2.0, scale='linear')

labelSizeBy(field, 0.8, 2) scales labels (of nodes, or of edges for an edge field) in proportion to a numeric field: 1 is the normal size. Sets the labelsize field; scale as in sizeBy ("linear", "log", "sqrt", "rank").

lastClick()

lastClick() is where the display was last clicked, as (x, y) in graph coordinates, or None before the first click.

lowest(field, count=10, seq=None)

lowest(degree, 5): the 5 with the smallest degree, smallest first (see highest).

makeFromCalls(target)

makeFromCalls("package.module") builds a call graph of a module (a module name or a .py file): a node per function and method, an edge from each to the functions it calls. Calls are matched by name (f(), self.f(), cls.f(), Class.f()), so it's an approximation: calls through variables, other modules or dynamic dispatch are missed. Nodes get `function` (Class.name or name), `cls`, `lines`, `params` and `public` (no leading underscore); edges get `weight`, the number of call sites.

makeFromCSV(filename, nodes=None, directed=False, source=None, target=None, interaction=None, weight=None, header=True)

makeFromCSV("edges.csv") replaces the graph with a spreadsheet table as Gephi reads it: an edge table (Source, Target[, Type, Weight, Label, …]), an adjacency matrix or an adjacency list. makeFromCSV("edges.csv", "nodes.csv") also reads a node table (Id[, Label, …]). Opening just a node table adds its columns to the current graph's nodes (matched by Id). Other columns become fields; directed=true makes edges without a Type directed.

As in Cytoscape, any columns can hold the edges: source= and target= name them (by header, or number from 0), interaction= the type of interaction (stored in the edge field `interaction`) and weight= the weight: makeFromCSV("ppi.tsv", source="gene_a", target="gene_b", interaction="type"). header=false reads a table without a header row (columns col1, col2, …; source=0, target=1 pick them).

makeFromDL(filename)

Replace the current graph with a UCINET DL file (fullmatrix, edgelist1, nodelist1).

makeFromDOT(filename)

Replace the current graph with a Graphviz DOT file (.dot, .gv): node and edge statements, subgraphs (cluster_* members get a `cluster` field), default attributes; pos, color/fillcolor, label, width/height, shape, penwidth and weight map to GUESS, other attributes become fields.

makeFromGDF(filename)

Replace the current graph with the contents of a GDF file.

makeFromGEXF(filename)

Replace the current graph with a GEXF file (Gephi's format): attributes, colours, positions, sizes, edge weights and thickness.

makeFromGloBI(taxa, interaction='eats', limit=500, server='https://api.globalbioticinteraction

makeFromGloBI(["Vulpes vulpes", "Lynx lynx"]) opens observed species interactions from the Global Biotic Interactions database (GloBI): for each taxon, what it `interactions` (eats, preysOn,

pollinates, parasiteOf, visitsFlowersOf, …), up to `limit` records each. Feeding links run from food to eater, other kinds from the taxon to its partner. Edges get `weight` (records) and `interaction`; nodes `taxon` (the full name) and `query` (true for the taxa you asked for). GloBI aggregates many studies with their own licences: cite GloBI and the sources you use.

`makeFromGML(filename)`

Replace the current graph with a GML file (Graph Modelling Language, as written by networkx, igraph, yEd and Cytoscape: labels, graphics x/y, fill colour, size and shape, and every other attribute as a field). For compatibility with GUESS 2006, a GraphML (XML) file works too.

`makeFromGraphML(filename)`

Replace the current graph with a GraphML file.

`makeFromImports(target, external=False, tests=False, level=None)`

makeFromImports("networkx") builds the import graph of a Python package (an installed package's name, or a folder or file of source): a node per module, an edge from each module to each module of the package it imports (importer -> imported). Nodes get `module` (the dotted name), `package` (its parent package), `depth`, `lines`, `functions` and `classes`; edges get `weight` (import statements) and `toplevel` (false when every import between the two happens inside a function: a deferred import, often used to dodge an import cycle). external=true also adds the outside packages imported (`external` true). level=k merges modules below depth k into their parent (e.g. level=2 gives one node per subpackage). Test modules are skipped unless tests=true. The code is parsed, never run.

`makeFromPajek(filename)`

Replace the current graph with a Pajek .net file (nodes are named pajek).

`makeFromSimilarity(filename, k=5, threshold=None, method='cosine', id=None, columns=None, features=None, values=None, mutual=False)`

makeFromSimilarity("profiles.csv") links each row of a feature table to the k rows most similar to it (k=5), or, with threshold=t, every pair at least t similar. The table is wide (an id column, here the first or id=…, then numeric columns; columns=[…] picks them, and text columns become node fields) or long, one row per (id, feature, value): features="keyword", values="score" (values default to 1). method: "cosine" (default; profiles pointing the same way), "correlation" (same ups and downs), "jaccard" (shared nonzero features) or "euclidean" (close values; 1/(1+distance)). mutual=true keeps only pairs that are in each other's top k. Edges get `weight` = similarity. Typical uses: co-expression networks, documents by term profile, people by votes or answers.

`makeFromText(filename, terms=None, by='paragraph', window=None, minCount=3, maxTerms=60, caseSensitive=False, stopwords=None)`

makeFromText("story.txt", terms) builds a co-occurrence network: one node per term, linked when two terms occur in the same paragraph (by="sentence" or "line" for smaller units; window=k for within k words of each other, in the same paragraph). terms is a list of names, or a dict from each node's name to its aliases, e.g. {"Holmes": ["Holmes", "Sherlock"], "Adler": ["Irene Adler", "Miss

Adler", "the woman"]} (matching ignores case unless caseSensitive=true). Without terms, the maxTerms most frequent words (at least minCount times, minus common words and stopwords=[...]) are used. Nodes get `count` (occurrences), `units` (paragraphs etc. containing them) and `first` (the first unit, for story order); edges get `weight`, the number of shared units (or windows).

`makeFromTwoMode(filename, a=None, b=None, weight=None)`

makeFromTwoMode("paper-authors.csv", "paper", "author") reads a table with one row per pair (a paper and one of its authors, a gene and one of its terms, a person and an event) as a two-mode network: a node for every value of each column, an edge for every row, and `kind` set to the column's name, ready for bipartiteLayout(kind) or project(kind, "author"). a= and b= name the columns (default: the first two); weight= names a numeric column to use as edge weight. Repeated rows add to the weight.

`makeNicheModel(species=30, connectance=0.15, seed=0)`

makeNicheModel(species, connectance) builds a food web with the niche model (Williams and Martinez 2000), which reproduces the structure of real food webs well: each species has a niche value (0..1) and eats every species whose niche falls in a range below its own. connectance (links / species ^ 2) is typically 0.05-0.3. Edges run from food to eater; nodes get `niche` and `range`. Species left without any link are dropped. seed= gives a different web.

`mergeNodes(*nodes, name=None)`

mergeNodes(a, b, ...) merges nodes that are the same entity into the first (or name=...): every edge of the others moves to it (parallel edges are combined, adding their weights; edges between the merged nodes disappear), empty fields of the kept node are filled from the others, `aliases` records the other names and labels, and `merged` counts the nodes it stands for. One undo step. Also takes a set or a list of pairs from findDuplicates (each pair merged).

`minCostFlow(source, target, amount, capacity='capacity', cost='weight')`

minCostFlow(a, b, 100) finds the cheapest way to send `amount` units from a to b: each edge can carry up to its `capacity` field (missing or empty = unlimited) at its `cost` field per unit (default: weight; missing = 1). Direction is followed for directed edges; an undirected edge can carry flow either way. Stores each edge's flow in `flow` and returns the total cost (an error if the amount can't get through).

`nestedness(field, rows=None)`

nestedness(kind) measures how nested a two-mode network is (NODF, Almeida-Neto et al. 2008; 0 = not at all, 100 = perfectly): in a nested plant-pollinator network, the specialists interact with subsets of the species the generalists interact with. kind is the field saying which side each node is on; rows= names one side (default: the first value). Returns (NODF, NODF of the rows, NODF of the columns). Compare with a randomised network (rewire()) to judge whether a value is high.

`ninverse(seq)`

All nodes not in seq.

nodeData(*args)

nodeData(field, …) or nodeData(nodes, field, …): one row per node for an Altair chart, with `name`, the fields given (all data fields when none are) and `_node`, the node's id, which links each mark to its node. alt.Chart(nodeData("degree", "dept")).mark_point().encode(x="degree", y="dept")

offspring(cases=None, show=True)

offspring() reads a transmission tree (directed edges from infector to infected) and counts how many each case infected (its outdegree). Returns them with R (the mean), the dispersion k of a negative binomial fitted to them (small k, below 1: a few superspreaders cause most transmission), the share who infected no one, and the share of transmissions caused by the top 20% of cases. offspring(cases) uses only those cases, e.g. offspring(onset < 28); their infections count wherever they lead. Stores the count in the field `offspring`.

outbreaks(runs=200, seeds=1, model='SIR', beta=0.3, gamma=0.1, threshold=0.5, immune=None, weight=None, steps=1000, large=0.1, seed=0, name='risk', show=True)

outbreaks(runs) repeats spread() many times, because one simulated outbreak is an anecdote. Each run starts from `seeds` random nodes (a number) or from the given nodes. immune=nodes (vaccinated, isolated) can't be infected or start an outbreak; weight=field makes long contacts riskier (see spread). Stores each node's share of runs it was reached in `risk` (name=…), charts the final sizes (hover a bar to see where those outbreaks started), and returns them (.mean, .median, .large = share of runs reaching more than `large` of the nodes).

readSets(filename, item=None, set=None, where=None)

readSets("go.csv") reads a table with one row per (item, set) pair, e.g. gene,term, into {set: {items}} for enrichment(). item= and set= name the columns (default: the first two); where={"aspect": "process"} keeps only matching rows. A row's set may list several, separated by ";" or "|".

remove(seq)

Remove nodes and/or edges (a node takes its edges). Undo with add(…). As in the original, a removed node's name still refers to it, so add(name) or addNode(name) puts it back.

removeComplete(seq)

Remove nodes and/or edges permanently (they cannot be re-added).

removeDisconnected()

Remove (and return) nodes with no edges other than self loops.

removeEdge(e)

Remove one edge.

removeLeaves()

Repeatedly remove nodes of total degree <= 1. Returns what was removed.

removeMap()

removeMap() removes the map drawn by geoLayout (the positions stay).

removeNode(node)

Remove one node and its edges (undo with add(g.deleted) or undo()). Its name still refers to it, so addNode(n) puts it back.

removeSelfLoops()

Remove (and return) all edges whose endpoints are the same node.

report(filename=None, top=5)

report() writes a one-page summary of the graph: size, density, components, degree distribution, path lengths, clustering, assortativity, the `top` nodes by degree, betweenness and PageRank, and the communities if any have been found (louvain_class, leiden_class, infomap_class). report("summary.html") saves it as a web page (offered as a download in the browser) and report("summary.md") as Markdown; with no name it's printed. Returns the Markdown.

resizeLinear(field, smallest, largest)

Size nodes (or edge widths) by rank of field value, from smallest to largest.

resizeRandom(field, smallest, largest)

Give each distinct value of field a random size (or edge width) between smallest and largest.

richClub(normalized=True, trials=5, seed=0, show=True)

richClub() asks whether the well-connected nodes are densely linked to each other: for each degree k, the rich-club coefficient is the density of the links among nodes with degree > k. normalized=True divides it by the same coefficient in degree-preserving random networks (averaged over `trials`): above 1 = a rich club (hubs prefer each other, as in airline or scientific elites), below 1 = hubs avoid each other. Prints a table and returns {k: value}.

shortcutNodes()

Repeatedly replace degree-2 nodes by an edge between their neighbours. Returns [removed, added].

showChart(chart, title=None)

showChart(chart, title=None) shows an Altair chart in the dock (a cell whose value is a chart does this by itself). Hovering a mark highlights its nodes, clicking selects them, right-click opens the element menu; interval selections (brushes) select what they cover.

sizeBy(field, smallest, largest, scale='linear')

sizeBy(field, smallest, largest) sizes nodes (or edge widths, for an edge field) in proportion to a numeric field's value, from smallest to largest. scale may be "linear", "log", "sqrt" or "rank" (as in Gephi's ranking). Compare resizeLinear, which spaces distinct values evenly.

skitter(field)

Nodes around a circle ordered by field; radius shrinks with outdegree.

sortBy(*d, descending=False)

sortBy(field) or sortBy(seq, field), smallest first; descending=true for largest first.

spread(seeds, model='SIR', beta=0.3, gamma=0.1, threshold=0.5, steps=100, seed=0, show=True, immune=None, weight=None)

spread([v1, v2], model="SIR") simulates something spreading from the seed nodes, one step at a time: "SI" each step, every infected node infects each susceptible neighbour with probability beta "SIR" the same, and each infected node recovers (and can't be infected again) with probability gamma "cascade" independent cascade: a newly active node gets one chance (probability beta) to activate each neighbour "threshold" a node becomes active when at least `threshold` of its neighbours are (complex contagion: ideas, adoption) Directed edges are followed in their direction. immune=nodes can't be infected (vaccinated, say: state V). weight=field makes long contacts riskier: beta is then the chance per unit of weight, so an edge of weight w transmits with chance 1 - (1 - beta)**w per step. Stores `infected_at` (the step a node was reached; -1 = never) and `state` (S, I, R or V at the end), plots how many are in each state at each step, and returns those counts. Replay it with showTime(t, infected_at); outbreaks() repeats it many times.

travelingSalesman(stops=None, weight='weight', cycle=True)

travelingSalesman(stops) finds a short route that visits every node in `stops` (default: all) and returns to the start (cycle=False: an open path), travelling along the graph's edges with `weight` as the length (missing = 1); a stop can be passed through more than once. It's an approximation (Christofides' algorithm on undirected graphs, a greedy method otherwise), fine for planning, not a proof of optimality. Returns the route's nodes and edges in order; .length holds its total length.

trophicLevels()

trophicLevels() computes each species' prey-averaged trophic level (`trophic`: 1 for species that eat nothing in the web, otherwise 1 + the mean level of what it eats), `omnivory` (the spread of its food's levels: 0 = eats on one level) and `role` (basal: eats nothing here; top: nothing eats it; else intermediate). Prints a summary.

wordCloud(field, top=80, stopwords=None, minLength=3)

wordCloud(field) shows the most frequent words in a text field as a word cloud in the dock (as Gephi's Word Cloud plugin), sized by how many times each occurs. Hovering a word lights up the nodes (or edges) whose text contains it; clicking selects them. Common English words are left out; stopwords=[…] adds more, top= sets how many words, minLength= the shortest word kept.

Undo, redo, history and logging

history()

history() lists every command run this session, in order.

log(filename='session.py')

Start recording every successful command (save with stoplog()).

redo()

redo() re-applies the last command undone with undo().

saveHistory(filename='history.py')

saveHistory("work.py") downloads every command run this session as a script (run it again with execfile).

stoplog()

Stop recording and save the log (offered as a download); replay with execfile.

undo()

undo() reverses the most recent command that changed the graph, its fields, states, hulls, legends or display settings (Ctrl/Cmd+Z outside the code cell). Repeat to go further back (up to 50 steps); redo() re-applies. Variables and downloaded files are not undone.

Statistics, communities, filters and paths

averageClustering()

The mean local clustering coefficient.

averagePathLength()

Mean shortest-path length (hops, ignoring direction) over all connected pairs.

diameter()

The longest shortest path (in hops, ignoring direction) between any two connected nodes.

egoNetwork(center, depth=1)

egoNetwork(node, depth=1): the node and everything within `depth` hops of it (ignoring direction).

giantComponent()

The nodes of the largest connected component (ignoring direction).

graphStats()

Print a summary: size, density, degree, components, diameter, average path length, clustering, transitivity, degree assortativity, reciprocity (directed edges) and (if computed) modularity.

groupsOfSize(field, smallest, largest=None)

groupsOfSize(field, min[, max]): the nodes (or edges) whose value of field is shared by at least min (and at most max) elements — e.g. drop tiny communities with showOnly(groupsOfSize(louvain_class, 5)).

heatMap(center, near='#d7191c', far='#2c7bb6', unreachable='#bdbdbd', middle='#8e5fbf', spread=True)

heatMap(node) colours every node by its distance (hops, ignoring direction) from node: red at the node, through violet, to blue for the farthest, grey if unreachable; the distance is stored in the hops field (-1 if unreachable). spread=True (default) spaces the colours by how many nodes are at each distance, so every band is visible; spread=False spaces them by distance alone. near, middle, far and unreachable change the colours (middle=None: a straight blend). Gephi's heat map tool.

interEdges(field)

interEdges(field): edges joining nodes with different values of the field.

intraEdges(field)

intraEdges(field): edges whose two endpoints share the field's value (e.g. edges inside each community).

kCore(k)

kCore(k): the nodes of the k-core — the largest subgraph in which every node has at least k neighbours. E.g. showOnly(kCore(3)).

louvainClusters(resolution=1.0, seed=0)

louvainClusters() finds communities by Louvain modularity optimisation (Gephi's Modularity statistic) using edge weights, stores each node's community in the louvain_class field (0 = largest) and returns the communities. Try colorize(louvain_class). resolution > 1 gives more, smaller communities; seed=None varies the result between runs.

modularity(field)

modularity(field): the modularity Q of the partition given by a node field (e.g. modularity(dept), modularity(louvain_class)).

mutualEdges()

Directed edges whose reverse edge also exists.

paint(center, color, depth=0)

paint(node, color, depth=0) colours a node, and with depth=k everything within k hops of it (Gephi's brush).

radius()

The smallest eccentricity among nodes with at least one neighbour.

recompute(*fields)

Computed fields (betweenness, pagerank, component, …) recalculate themselves when read after the graph has changed, so this is rarely needed: recompute(pagerank, …) forces it now (e.g. for a field you had overwritten, which then follows the graph again); recompute() redoes every computed field that has been used so far.

shortestPath(source, target, weighted=False)

shortestPath(a, b): the nodes and edges of a shortest path from a to b (following edge direction), or an empty set if there is none. weighted=true uses edge weights as lengths. E.g. shortestPath(a, b).color = red

showAll()

Make every node and edge visible again.

showOnly(seq, induced=False)

showOnly(seq) hides every node and edge not in seq: a set of nodes keeps the edges among them; a set with edges keeps just those edges and their endpoints, plus any nodes listed (showOnly(shortestPath(a, b)) shows the path and nothing else). induced=true keeps every edge among the shown nodes instead. Undo with showAll(). As subgraph does.

Clustering and generators

biComponentClusters()

Biconnected components (ignoring edge direction).

edgeBetweennessClusters(count)

Split the graph into `count` communities with Girvan-Newman.

makeBarabasiAlbertRandom(*args)

makeBarabasiAlbertRandom(vert, edges, evolve): preferential attachment; starts from `vert` nodes and adds `evolve` nodes with `edges` edges each.

makeEppsteinRandom(*args)

makeEppsteinRandom(nodes, edges, r): power-law graph by r rounds of degree-biased rewiring of a uniform random graph (Eppstein & Wang).

makeErdosRenyiRandom(*args)

makeErdosRenyiRandom(nodes, p): each edge present with probability p.

makeKleinbergRandom(*args)

makeKleinbergRandom(size, clust): Kleinberg small world on a size x size torus with one long-range edge per node, distance exponent clust.

makeLattice1DRandom(*args)

makeLattice1DRandom(nodes, toroidal): a path (or ring if toroidal).

makeLattice2DRandom(*args)

makeLattice2DRandom(size, toroidal): a size x size grid (torus if toroidal).

makeSimpleDirectedRandom(*args)

makeSimpleDirectedRandom(nodes, edges): uniform random directed graph.

makeSimpleRandom(*args)

makeSimpleRandom(nodes, edges): uniform random undirected graph.

weakComponentClusters()

Connected components (ignoring edge direction).

More measures: Katz, prestige, bridging, cliques, MDS, k-brace; field tools and generators

castField(field, type)

castField(field, INTEGER) changes a field's type (INTEGER, DOUBLE, BOOLEAN or VARCHAR) and converts its values: "12" becomes 12, 3.7 becomes 3, "yes" becomes true. Values that can't be converted are counted and left empty. One undo step.

cliques(minSize=3, limit=1000)

cliques(minSize=3) finds every maximal clique (a group where everyone is linked to everyone, that no other node could join) with at least minSize nodes, ignoring edge direction; largest first, up to `limit`. largestCliques() gives just the biggest ones.

copyColors(fromGraph, field='name')

copyColors("other") gives each node the colour of the node with the same name (or the same value of field=...) in another graph of this session, so the same people look the same in both (Gephi's Node Color Manager). Nodes with no match keep their colour. One undo step.

evaluateLinks(method='adamic_adar', hide=0.1, seed=0, show=True)

evaluateLinks() tests how well predictLinks would have done: it hides a share of the edges (hide=0.1), scores the unlinked pairs from what's left, and reports precision (of the top-scored pairs, as many as were hidden, how many are hidden edges) and AUC (how often a hidden edge outscores a pair that was never linked; 0.5 is chance). The graph isn't changed. method as in predictLinks, or "all" to compare them.

geoVectors(edges=None, lat=None, lon=None)

geoVectors() stores each edge's compass bearing (degrees from north, from node1 to node2) in `bearing` and its great-circle length in `km`, and summarises the edges (all, or the given ones): their mean length and its standard deviation, and the resultant vector (the edges added up as arrows, as east and north kilometres) with its size and bearing. Latitude and longitude fields are found as geoLayout finds them, or named: geoVectors(lat=y, lon=x). Returns the summary as a dict.

kBrace(k=2)

kBrace(k) is the k-brace: what's left after repeatedly removing every edge whose two ends have fewer than k neighbours in common (an edge's embeddedness), ignoring direction. Weakly embedded ties go; the dense core stays. showOnly(kBrace(2)) shows it. (It equals the (k+2)-truss: each remaining edge lies in at least k triangles.) Returns the edges.

makeBarabasiAlbertAging(n, m, aging=1.0, seed=0)

makeBarabasiAlbertAging(n, m, aging=1) grows a preferential-attachment network in which old nodes lose their appeal: each new node links to m existing ones, chosen with probability proportional to degree x age^-aging (age in steps since the node arrived). aging=0 is plain Barabási-Albert; larger values favour recent nodes (Dorogovtsev and Mendes' model).

makeGeometricRandom(n, radius, seed=0)

makeGeometricRandom(n, radius) scatters n nodes in the unit square and links every two closer than radius (a random geometric graph: sensor and contact networks). Nodes are drawn where they fell.

makePowerlawCluster(n, m, p, seed=0)

makePowerlawCluster(n, m, p) grows a scale-free network with clustering (Holme and Kim): preferential attachment with m edges per new node, each followed with probability p by a triangle-closing edge.

makeWattsStrogatz(n, k, p, seed=0)

makeWattsStrogatz(n, k, p) replaces the graph with a Watts-Strogatz small world: n nodes on a ring, each linked to its k nearest neighbours, then each edge rewired with probability p (0 keeps the ring lattice; 1 is close to random). Nodes are drawn on the ring.

mds(dims=2, weight=None, name='mds')

mds(dims=2) places the nodes in `dims` dimensions by classical multidimensional scaling of their shortest-path distances (hops; with weight=… an edge field read as a distance), storing the coordinates in fields mds1, mds2, … (or name=…). Unreachable pairs count as one more than the longest distance. scatterLayout(mds1, mds2) draws them; mdsLayout() does both in one go.

readNames(filename, field='name')

readNames("keep.txt") is the nodes whose name (or field=…) appears in a text file: one per line, or separated by commas, semicolons, tabs or spaces. showOnly(readNames("keep.txt")) keeps just them (Gephi's FilterFromFile). Says how many listed names matched no node.

weightedBridging(weight='weight', name='wbridging')

weightedBridging() is bridging centrality for weighted networks: a strong tie (big weight) counts as a short one (distance 1/weight), so shortest paths prefer strong ties. Stores the weighted betweenness in `wbetweenness` and weighted betweenness x bridging_coef in `wbridging` (or name=…). Edges with weight 0 or less are left out.

More clustering: Chinese Whispers, MCL, DBSCAN, MCODE, overlapping communities

chineseWhispersClusters(iterations=20, weighted=True, seed=0)

chineseWhispersClusters() finds communities by Chinese Whispers (Biemann 2006): every node starts in its own class and repeatedly takes the class with the most (weighted) ties among its neighbours. Fast and parameter-free; the number of communities isn't chosen in advance. Stores cw_class (0 = largest) and returns the communities. weighted=False ignores edge weights; seed=None varies the result.

cliquePercolationClusters(k=3)

cliquePercolationClusters(k=3) finds overlapping communities by clique percolation (Palla et al. 2005): a community is a chain of k-cliques that share k-1 nodes, so a node can be in several (and many nodes are in none). Stores cpm_classes (e.g. "0;2"; "" for none) and cpm_count; returns the communities, largest first. showOverlaps(…) draws them.

dbscanClusters(fields=('x', 'y'), eps=None, minPts=4)

dbscanClusters() groups nodes that lie close together on screen with DBSCAN (Ester et al. 1996): a cluster is a dense region (at least minPts nodes within eps of each other), and nodes in sparse areas are noise (dbscan_class -1). fields: numeric node fields to measure distance in, by default the positions (x, y); e.g. dbscanClusters((degree, pagerank)). Fields aren't rescaled, so give them comparable ranges. eps=None picks it from the knee of the sorted k-distance curve and prints it.

linkCommunities(threshold=None, minEdges=2)

linkCommunities() finds overlapping communities by clustering edges (Ahn, Bagrow & Lehmann 2010): two edges sharing a node are similar when their other ends have similar neighbourhoods; edges are merged by single linkage and the dendrogram is cut where partition density is highest (or at similarity `threshold`, 0-1). Each edge gets link_class; a node belongs to the communities of its edges (link_classes "0;3", link_count). Communities of fewer than minEdges edges are dropped (link_class -1). Returns node communities, largest first.

mclClusters(inflation=2.0, expansion=2, loops=True, iterations=100)

mclClusters() finds clusters by the Markov Cluster algorithm (MCL, van Dongen 2000): random walks are expanded and then "inflated" until flow stays inside clusters. inflation (1.2-5, default 2) sets the granularity: higher gives more, smaller clusters. Uses edge weights; loops=True adds self loops (recommended). Stores mcl_class. Works on connected pieces of up to 3000 nodes.

mcodeClusters(degreeCutoff=2, scoreCutoff=0.2, haircut=True, fluff=False, fluffDensity=0.1, kCore=2, maxDepth=100)

mcodeClusters() finds densely connected regions with MCODE (Bader & Hogue 2003), made for protein complexes in interaction networks. Each node is weighted by the density of the highest k-core of its neighbourhood; complexes grow from the heaviest seeds through neighbours weighing at least (1 - scoreCutoff) of the seed's. Defaults are the paper's: degreeCutoff 2, scoreCutoff 0.2, haircut (keep only the 2-core) on, fluff (add dense neighbours, fluffDensity) off, kCore 2, maxDepth 100. Stores mcode_class (0 = best scoring, -1 = in none) and mcode_score (the node weight); returns the complexes, best first, and prints their scores (density x size).

showOverlaps(communities, hulls=False, none='lightgray')

showOverlaps(communities) shows overlapping communities: each gets its own colour, a node in one takes that colour, a node in several is drawn as a pie with a slice per community, and nodes in none are lightgray (none=...). communities: the result of cliquePercolationClusters()/linkCommunities(), any set of sets, or a field listing them (cpm_classes, link_classes, or a class field). hulls=True also shades each community. removePies() goes back to plain nodes.

Structure: bridges, cut nodes, backbones, directed graphs, mixing, comparing partitions

`ancestors(node)`

ancestors(node): every node with a directed path to node (upstream: what it depends on or cites, transitively). Undirected edges count both ways.

`assortativity(field=None)`

assortativity() is the degree assortativity of the graph: positive when hubs link to hubs, negative when hubs link to small nodes. assortativity(dept) measures homophily by a field: near 1 = ties stay within the same value, 0 = random mixing, negative = ties cross values. Numeric fields use the correlation of values across edges.

`blockmodel(count=4, show=True)`

blockmodel(count) groups the nodes into count positions of structurally equivalent nodes: nodes tied to the same others (in the same directions), whether or not they're tied to each other, which is a role (teacher, broker, follower), not a community. Profiles are compared by correlation and merged by average-linkage clustering. Stores each node's position in `position` (0 = largest), prints the image matrix (the density of ties from each position to each other; a dense diagonal means cohesive positions, dense off-diagonals mean roles that serve each other) and returns the positions. Try colorize(position), then collapseBy(position) for the reduced graph. Fine up to a couple of thousand nodes.

`bridges()`

bridges(): the edges whose removal would split a connected part of the graph in two (direction ignored; a pair joined by parallel edges is never a bridge). bridges().color = red shows the weak ties.

`compareClusters(field1, field2, method='nmi')`

compareClusters(louvain_class, dept) says how well two partitions of the nodes agree, over the nodes that have both values: method "nmi" (normalized mutual information, 0..1), "ari" (adjusted Rand: 0 = chance, 1 = identical), "rand" or "vi" (variation of information: 0 = identical). Works for edge fields too.

`cutNodes()`

cutNodes(): the nodes whose removal would split a connected part of the graph (articulation points; direction ignored).

`descendants(node)`

descendants(node): every node reachable from node along directed edges (downstream: what depends on it, or what it leads to).

feedbackEdges()

feedbackEdges(): directed edges that close cycles (back edges of a depth-first search, plus self-loops); removing or reversing them leaves the directed part acyclic. A small set, not necessarily the smallest.

mainPath()

mainPath() finds the main path of a citation (or other flow) network: the chain of directed edges that the most source-to-sink paths run through (Hummon and Doreian's main path analysis, key-route variant). Each directed edge gets `spc`, its search path count as a share of all source-to-sink paths (0..1). The path starts at the edge with the highest spc and extends both ways, always along the highest-spc edge, to a source and a sink. Edges closing cycles (feedbackEdges()) are ignored. Returns the path's nodes and edges. In a citation network with edges citing -> cited, it runs from recent work back to the field's foundations.

mixingMatrix(field, show=True)

mixingMatrix(dept) counts the edges between each pair of values of a node field (dept1-dept1, dept1-dept2, …), printed as a table and returned as {value: {value: count}}. Direction is ignored. With many values only the 20 most common are printed.

reciprocity()

reciprocity(): the share of directed edges whose reverse edge also exists (1 = every tie returned). Undirected edges are left out.

robustness(orders=('random', 'degree'), trials=5, seed=0, show=True)

robustness() asks how the network falls apart as nodes fail: for each way of removing nodes it gives the share of nodes still in the largest connected piece after removing each share of them, and plots the curves. orders: "random" (failures; averaged over `trials` random orders), "degree" or "betweenness" (attacks on the biggest hubs or bridges first, ranked once at the start), "adaptive" (the current highest-degree node each time), or a node field (highest first). The graph isn't changed. Returns {order: [(removed share, largest-piece share), …]} and prints the share removed at which the largest piece drops below half. Many real networks shrug off random failures but break quickly under attack.

spanningTree(maximum=False, weighted=True)

spanningTree() returns the edges of a minimum spanning tree (a forest if the graph is in pieces) by edge weight: the cheapest backbone that keeps everything connected. spanningTree(maximum=True) keeps the strongest ties instead, a good backbone when weight means strength: showOnly(spanningTree(maximum=True)). weighted=False ignores weights.

transitiveReduction()

transitiveReduction(): in a directed acyclic graph, the edges implied by longer paths (a -> c when a -> b -> c exists, and repeated edges). They are marked in the edge field `redundant` and returned,

so transitiveReduction().visible = false shows the backbone of a citation or dependency graph. Only directed edges count; cycles are an error (see feedbackEdges).

`transitivity()`

transitivity(): the share of connected triples that close into triangles (global clustering coefficient; direction ignored).

`triadCensus(show=True)`

triadCensus() counts the 16 kinds of directed triads (Holland and Leinhardt): how often three nodes form a cycle, a transitive triple, a star and so on. Undirected edges count as mutual. Prints the types that occur and returns {type: count}.

networkx bridge and sample graphs

`fromNetworkX(G, name=None)`

fromNetworkX(G) replaces the graph with networkx graph G: its node and edge attributes become fields (types from the values), `pos` or x/y give positions (otherwise a spring layout), and direction is kept. String keys become node names; other keys become n0, n1, … with the original in `nxkey`. fromNetworkX(G, "name") opens it in a new tab instead and switches to it (use("main") goes back).

`makeSample(name=None)`

makeSample("karate") loads a classic network: karate, lesmis, florentine, davis, petersen, kite, tutte, dodecahedron, caveman. makeSample() lists them. They come with their attributes (karate's `club`, lesmis' `weight`, davis' `bipartite`) and a spring layout.

`setField(values, name, kind=None)`

setField(values, "name") stores a result as a field: a dict from nodes (or node names) to values, as networkx centralities return; a dict from node pairs (a, b) to values gives an edge field; a list of node sets (as community algorithms return) gives each node its set's index, largest first. The type is taken from the values. Returns the field. setField(nx.core_number(toNetworkX()), "core") setField(nx.community.greedy_modularity_communities(toNetworkX()), "greedy")

`toNetworkX(directed=None, visibleOnly=False)`

toNetworkX() returns the graph as a networkx graph, to use any networkx algorithm: node keys are node names, and every field (dept, color, x, y, pagerank once used, …) is an attribute; positions are also `pos`. directed=True/False forces the type (a mixed graph becomes directed, with undirected edges both ways); visibleOnly=True leaves out hidden elements. Bring results back with setField: import networkx as nx setField(nx.katz_centrality(toNetworkX()), "katz")

igraph: communities, large-graph layouts, similarity, patterns, cuts and flows

`bipartiteLayout(field, hgap=1, vgap=1)`

bipartiteLayout(field) draws a two-mode network in two rows (by a field with two values, e.g. bipartite in makeSample("davis")), ordering each row to reduce edge crossings.

`cohesiveBlocks()`

cohesiveBlocks() finds nested groups that stay connected after removing any k-1 nodes (Moody and White's cohesive blocking): a structural view that k-cores miss. Each node's highest cohesion is stored in `cohesion`; the blocks are returned, most cohesive last. Slow beyond a few thousand nodes.

`cutDendrogram(count)`

cutDendrogram(n) re-cuts the hierarchy of the last walktrapClusters or fastGreedyClusters into n communities (same field), e.g. driven by a Slider to explore coarser and finer groupings.

`dhLayout(seed=0)`

dhLayout() is the Davidson-Harel layout: simulated annealing that spreads nodes evenly and avoids edge crossings and nodes on edges. Slow but tidy: for figures of up to a few hundred nodes.

`dominatorTree(root)`

dominatorTree(root) follows directed edges from root and finds, for every node it reaches, its immediate dominator: the last node every path from root must pass through (who controls access to whom). Stored in `idom` (a node name); returns the nodes root reaches.

`drlLayout(preset='default', seed=0)`

drlLayout() is a force-directed layout for large graphs (DrL / OpenOrd): tens of thousands of nodes in seconds, with clusters pulled well apart. preset: "default", "coarsen" (more separated clusters), "coarsest", "refine" (polish an existing layout) or "final".

`eigenvectorClusters(count=None)`

eigenvectorClusters() splits the graph by the leading eigenvector of the modularity matrix (Newman), repeatedly; count limits the number of communities. Stored in eigen_class.

`fastGreedyClusters(count=None)`

fastGreedyClusters() merges communities greedily by modularity (Clauset-Newman-Moore), building a hierarchy; count as in walktrapClusters. Stored in fastgreedy_class; cutDendrogram(n) re-cuts.

findPattern(pattern, induced=False, limit=100)

findPattern(pattern) finds every place where a small pattern occurs in the graph (subgraph isomorphism, LAD algorithm). The pattern is a list of pairs, e.g. findPattern([("a","b"), ("b","c"), ("c","a")]) for triangles, or a networkx graph (nx.cycle_graph(4), nx.star_graph(3)). induced=True requires no extra edges among the matched nodes. Returns up to `limit` matches, each a set of nodes; the same nodes matched in another order count once.

fluidClusters(count, seed=0)

fluidClusters(count) splits a connected graph into exactly count communities (fluid communities). On a graph in pieces run it on one part, e.g. in subgraph(giantComponent()). Stored in fluid_class.

infomapClusters(trials=10, seed=0)

infomapClusters() finds communities by Infomap (how a random walker's flow stays inside groups), following edge direction and weights; a good choice for directed networks such as citations or web links. Stored in infomap_class.

labelPropagationClusters(field=None, seed=0)

labelPropagationClusters() finds communities very fast (linear time) by letting each node adopt its neighbours' most common label; good for very large graphs, rougher than Leiden. labelPropagationClusters(field) starts from the values of a field and keeps those nodes fixed, filling in the rest (semi-supervised). Stored in lpa_class.

largestCliques()

largestCliques() returns the largest cliques (groups where everyone is linked to everyone), each a set of nodes. Exponential in the worst case, fast on sparse graphs.

leidenClusters(resolution=1.0, objective='modularity', iterations=-1, seed=0)

leidenClusters() finds communities with the Leiden algorithm (better than Louvain: every community is internally connected) and stores them in leiden_class (0 = largest). resolution > 1 gives more, smaller communities. objective="CPM" (constant Potts model) avoids modularity's resolution limit; its resolution is a density, typically 0.001-0.1. iterations=-1 runs until stable; seed=None varies the result.

lglLayout(root=None, seed=0)

lglLayout() is the Large Graph Layout: grows the drawing outward from a root node (default: a random one, or lglLayout(v1)), suited to big, tree-like networks.

maxFlow(a, b, capacity='weight')

maxFlow(a, b) computes the maximum flow from a to b with edge weights as capacities (capacity=None: 1 per edge), following edge direction when the graph has directed edges. Each edge's flow is stored in the edge field `flow`; returns the total.

maxMatching(field=None)

maxMatching() returns a largest set of edges with no node in two of them (pairs everyone it can); maxMatching(field) matches across the two values of a field in a two-mode network, using weights.

minCut(a, b, capacity=None)

minCut(a, b) returns the fewest edges whose removal separates a from b (direction ignored); minCut(a, b, weight) the cut of least total weight. Prints the cut's size.

optimalClusters()

optimalClusters() finds the partition with the highest possible modularity (exact, by integer programming): only for small graphs (up to about 100 nodes), as a baseline for the other methods. Stored in optimal_class.

personalizedPagerank(seq, damping=0.85)

personalizedPagerank(seq) ranks every node by how easily a random walk restarting at the given nodes reaches it: relevance to a chosen set (e.g. personalizedPagerank(sel) or personalizedPagerank(v1)). Stored in the field `ppr`.

predictLinks(top=20, method='adamic_adar', show=True, add=False)

predictLinks() lists likely missing edges: pairs of nodes that aren't linked but share neighbours, scored by "adamic_adar" (shared neighbours, rare ones counting more), "jaccard", "common" (count of shared neighbours) or "resource" (resource allocation). Returns the `top` pairs as (node, node, score), best first. add=true also adds them as edges, marked with the method's name in the edge field `predicted` (showOnly(predicted != None) shows them; undo() takes them away). evaluateLinks() tests how well a method would have predicted.

rewire(times=None, name=None, seed=0, keep=None)

rewire() makes a randomised copy of the graph in a new tab keeping every node's degree (repeated swaps of edge ends, 10 x edges by default): the null model for "is this structure more than chance?". Compare e.g. transitivity() or modularity here and there. Edge weights travel with the swapped ends. keep=field only swaps ends that share that field's value, so the edges between each pair of values stay the same in number: rewire(keep=kind) keeps a two-mode network two-mode (plants still only link to pollinators).

similarTo(node, method='jaccard', top=10)

similarTo(v1) scores every node by how much its neighbourhood overlaps v1's, stores the score in the field `similarity` and returns the `top` most similar nodes. method: "jaccard" (shared neighbours / all neighbours of either), "dice" (2 x shared / sum of degrees), or "adamic_adar" (shared neighbours, rare ones counting more). Direction is ignored.

simplify(combine='sum', loops=True)

simplify() merges parallel edges between the same two nodes (and the same direction) into one, whose weight combines theirs: combine="sum" (default), "max", "min", "mean" or "first"; loops=True also removes self-loops. One undo step.

spinglassClusters(spins=25, gamma=1.0, seed=0)

spinglassClusters() finds communities by simulated annealing of a spin model (Reichardt-Bornholdt); handles negative edge weights (signed networks). Slow: for graphs up to about 2,000 nodes. spins is the most communities allowed, gamma > 1 favours smaller ones. Stored in spinglass_class. Needs a connected graph.

umapLayout(minDist=0.01, epochs=500, seed=0)

umapLayout() places nodes by UMAP on the graph's connections: tight, well-separated islands for clusters. Edge weights count as similarity. minDist (0..1) is how tightly points may pack; more epochs, more polish.

useIgraph(on=True)

useIgraph(false) makes the statistics use networkx / numpy instead of igraph (to compare results or work around a problem); useIgraph() turns igraph back on. Prints which engine is active.

voronoiClusters()

voronoiClusters() picks central "generator" nodes automatically and gives every other node to its nearest one (by weighted distance); the communities are stored in voronoi_class and the generators returned.

walktrapClusters(count=None, steps=4)

walktrapClusters() groups nodes that short random walks (steps long) tend to stay among, building a hierarchy; count=None cuts it where modularity is highest, walktrapClusters(8) gives 8 communities. Stored in walktrap_class; cutDendrogram(n) re-cuts it.

Graphviz layouts: dot, neato, sfdp, fdp, circo, twopi, osage; overlap removal

circoLayout()

circoLayout() puts the biconnected parts of the graph on circles, chained together: shows rings and cycles (pathways, loops).

dotLayout(rankdir='TB', clusters=None, nodesep=0.25, ranksep=0.5, route=True)

dotLayout() draws the graph in layers with Graphviz's dot: edges point down (rankdir="LR": left to right; "BT", "RL"), crossings are minimised and long edges are routed around nodes as smooth bends (route=False: straight edges). clusters=dept keeps each value's nodes together in a box. nodesep and ranksep are spacings in inches. Best for hierarchies, dependency and citation graphs.

fdpLayout(clusters=None, overlap='prism')

fdpLayout() is a force layout that can keep clusters together: fdpLayout(clusters=louvain_class) gives each community its own box.

graphvizLayout(engine='dot', **attrs)

graphvizLayout("neato") lays the graph out with any Graphviz engine: dot, neato, fdp, sfdp, circo, twopi, osage or patchwork. Keyword arguments become Graphviz graph attributes, e.g. graphvizLayout("neato", mode="sgd", overlap="prism") or graphvizLayout("dot", rankdir="LR", ranksep=1). Pinned (fixed) nodes stay put in neato and fdp.

neatoLayout(mode='major', model='shortpath', field=None, overlap='prism')

neatoLayout() places nodes so their distances on screen follow their distances in the graph (stress majorization: the best-quality layout up to a few thousand nodes). mode: "major" (default), "sgd" (faster), "KK" (Kamada-Kawai); model="circuit" uses resistance distance; field=weight makes an edge field the preferred edge length; overlap="prism" (default) then removes node overlaps (overlap="true" leaves them).

osageLayout(clusters)

osageLayout(dept) packs each value's nodes into a rectangle and packs the rectangles together: a compact overview of the groups.

removeOverlap(method='prism', sep=4)

removeOverlap() moves nodes apart until none overlap, keeping the shape of the current layout (Graphviz's PRISM). Works after any layout. method: "prism" (default), "voronoi", "scale" (spread evenly), "compress"; sep is the extra margin in points.

sfdpLayout(overlap='prism', K=None)

sfdpLayout() is Graphviz's multilevel force layout: fast on large graphs (thousands to tens of thousands of nodes) with good global structure. K is the ideal edge length in inches; overlap="prism" removes overlaps.

twopiLayout(root=None)

twopiLayout(v1) places nodes on rings by their distance from a root (Graphviz's twopi; default root: chosen by Graphviz).

Layouts

binPackLayout(rescale=True)

binPackLayout() packs the connected components next to each other, first rescaling them to similar densities; binPackLayout(false) packs them without rescaling.

centerAfterLayout(state)

centerAfterLayout(state): whether the view re-centres after each layout (default true). Disable it to control the view yourself.

circleLayout(field=None, center=None, x=None, y=None)

circleLayout() places all nodes evenly on a circle (radius 450 around 500,500).

circleLayout(field, centerNode) puts centerNode at (500, 500) and its neighbours on a circle around it, ordered by `field`, at a radius proportional to `field` (a node field of the neighbours, or an edge field of the connecting edges). Other nodes are left alone. circleLayout(field, centerNode, x, y) uses (x, y) as the centre.

dagLayout()

dagLayout() lays out a directed acyclic graph with a spring layout, keeping each node on a horizontal line for its level (sources at the top), for 500 iterations (fewer on graphs over 1000 nodes). Cycles are broken first.

forceAtlas2Layout(iterations=None, scaling=None, gravity=1.0, strongGravity=False, linLog=False, dissuadeHubs=False, preventOverlap=False, edgeWeightInfluence=1.0, gravitySources=None)

forceAtlas2Layout() runs Gephi's ForceAtlas2 from the current positions (animated; Stop ends it early). Options, by keyword: iterations steps to run (default 500, fewer on big graphs) scaling repulsion strength (default 2, or 10 under 100 nodes) gravity pull toward the centre (default 1) strongGravity=true gravity grows with distance (keeps components close) linLog=true LinLog mode: tighter, more separated clusters dissuadeHubs=true pushes hubs to the periphery (authorities central) preventOverlap=true keep node discs apart (best run after a first pass) edgeWeightInfluence 0 ignores weights, 1 uses them, >1 exaggerates gravitySources nodes that act as extra centres of gravity: every node is pulled toward the nearest of them (in hops) instead of the centre, so the graph forms a cluster around each (MultiGravity ForceAtlas2) E.g. forceAtlas2Layout(linLog=true, preventOverlap=true)

frLayout(width=None, height=None)

frLayout() runs Fruchterman-Rheingold (as in Pajek) starting from the current positions and fits the result into a 1000 x 1000 grid with cells the size of the largest node; frLayout(width, height) uses that area.

gemLayout(seed=None)

gemLayout() runs the GEM force-directed layout (Frick et al.); gemLayout(seed) seeds the random generator for a repeatable layout.

hiveLayout(axis=None, position=None, scale='rank', axes=None, inner=60, length=440, curved=True)

hiveLayout(axis, position) draws a hive plot (Krzywinski et al.): one radial axis per value of the node field `axis` (the most common values, at most `axes` of them, the rest merged into "other"), each node placed along its axis by the numeric field `position` (default totaldegree), and edges drawn as curves between axes. scale is "rank" (default; evenly spread), "linear" or "log". Without `axis`: for directed graphs the classic source / hub / sink axes (only outgoing, both, only incoming edges), otherwise leaves / connectors / hubs by degree. The axes are drawn with their labels; any later layout removes them. E.g. hiveLayout(dept, pagerank) or hiveLayout(louvain_class, axes=5)

isomLayout(iterations=None)

isomLayout() runs the JUNG ISOM (self-organising map) layout for 2000 epochs; isomLayout(n) runs n epochs.

jfrLayout()

jfrLayout() runs the JUNG Fruchterman-Rheingold layout (random start in a 1000 x 1000 area, up to 700 cooling iterations).

jkkLayout1()

jkkLayout1() runs the JUNG Kamada-Kawai layout: unweighted graph distances, random start, 2000 iterations each moving the worst-placed node.

jkkLayout2()

jkkLayout2() runs the JUNG Kamada-Kawai layout with integer distances (same algorithm as jkkLayout1).

jSpringLayout(iterations=None)

jSpringLayout() runs the JUNG spring layout from random positions until it settles or for 500 iterations (fewer on graphs over 1000 nodes); jSpringLayout(n) runs exactly n iterations.

kkLayout(width=None, height=None)

kkLayout() runs Kamada-Kawai on each connected component, starting from a circle, then fits the result into 1000 x 1000; kkLayout(width, height) uses that area instead.

labelAdjustLayout(margin=2)

labelAdjustLayout() moves nodes just enough that nodes and their visible labels stop overlapping (Gephi's Label Adjust).

mdsLayout()

mdsLayout() places nodes by classical multidimensional scaling of the edge weights (an edge's weight is the distance between its endpoints, every other pair is at distance 1). Nodes without edges are not moved.

mirrorLayout(horizontal=True)

mirrorLayout() flips the layout left-right; mirrorLayout(false) flips it upside down.

moveLayout(x, y)

moveLayout(x, y) moves the layout so its top-left corner is at (x, y).

noverlapLayout(margin=5, ratio=1.2)

noverlapLayout() nudges nodes apart until none overlap (Gephi's Noverlap): each node is treated as a disc of ratio x its size plus margin.

physicsLayout(iterations=None)

physicsLayout() runs the graphopt physics layout (charged nodes, springs) for 500 steps from the current positions (fewer on graphs over 1000 nodes); physicsLayout(n) runs n steps.

radialLayout(center, edges=None)

radialLayout(centerNode) lays the graph out as a radial tree around centerNode (placed at 0,0), following a breadth-first spanning tree (Eades' radial drawing). Nodes not reachable from the centre stay put. radialLayout(centerNode, edges) restricts the tree to the given edges.

randomLayout(width=None, height=None)

randomLayout() places nodes at random in a 1000 x 1000 area; randomLayout(width, height) uses a width x height area.

rescaleLayout(*d)

rescaleLayout(percent) scales the layout by percent (0.5 = half size) about its top-left corner; rescaleLayout(xpercent, ypercent) scales each axis separately; rescaleLayout(width, height) with integers fits the layout into a width x height box at the origin. rescaleLayout(nodes, percent) or rescaleLayout(nodes, xpercent, ypercent) spreads out (or draws together) just those nodes about their centre, leaving the rest where they are (Gephi's Scale layout).

ringMove(node, index)

ringMove(node, k) moves a node to position k around the circle made by circleLayout() and re-spaces the ring (what dragging a node around the circle does).

rotateLayout(degrees=90)

rotateLayout(degrees) turns the layout clockwise about its centre.

scatterLayout(x, y, width=1000, height=1000)

scatterLayout(x, y) places nodes by two numeric fields (x to the right, y upwards) in a width x height area. Dragging a node then edits those two values.

springLayout(iterations=None)

springLayout() runs the GUESS spring layout from the current positions until it settles or for 1000 iterations (fewer on graphs over 1000 nodes: 1000 * 1000 / nodes, at least 100); springLayout(n) runs exactly n iterations. Stop it early with the Stop button.

sugiyamaLayout(bends=False)

sugiyamaLayout() draws the graph in layers (Sugiyama): edges are made acyclic, nodes put on levels so every edge points down, crossings reduced by barycentres, on a 250 x 150 grid. sugiyamaLayout(true) also routes long edges through dummy positions.

sugiyamaLayout2()

sugiyamaLayout2() is a simpler layered layout: levels 200 apart, each node given horizontal room in proportion to its children.

translateLayout(dx, dy)

translateLayout(dx, dy) moves every node by (dx, dy).

treeLayout(root=None, horizontal=False, spacing=None)

treeLayout() draws each connected component as a tidy tree (a breadth-first spanning tree from its best-connected node, or from root), parents centred over their children; horizontal=true grows it left to right; spacing sets the distance between neighbouring leaves.

weightedKKLayout()

weightedKKLayout() runs the JUNG Kamada-Kawai layout using shortest paths where each edge's `weight` is its length.

weightedSpringLayout(minLength, maxLength, iterations=None)

weightedSpringLayout(minLength, maxLength[, iterations]) is springLayout with each edge's natural length mapped linearly from its `weight` (lowest weight -> minLength, highest -> maxLength). Iterations as for springLayout.

Structure-aware layout (motifLayout, after LUMINA)

`motifLayout(spacing=None, minStar=3, minCycle=3, maxCycle=40, minChain=2, minParallel=2, leavesAround=True, gap=2.0, iterations=300, outline=False, seed=0)`

motifLayout() is a structure-aware layout (a port of LUMINA, the IntAct team's layout for protein networks). It finds four shapes and gives each its own arrangement: cycles become circles, chains (paths ending in a leaf) hang off their anchor as small rings, a node with leaves becomes a star with the leaves in rings around it, and nodes sharing exactly the same neighbours (parallel) line up in a grid between them. Each shape is treated as one big node while the whole is laid out by stress majorization, so the drawing keeps graph distances; components are turned to fit and packed. Each node's shape is stored in the field `motif` ("star 3", "cycle 1", …), so colorize(motif) or groupBy(motif) work.

spacing= the distance between neighbouring nodes in a shape (default: from node sizes); gap= the room between shapes, in spacings; minStar= the leaves a node needs to be a star (leavesAround=false leaves smaller groups of leaves to the layout); minCycle/maxCycle and minChain set the sizes of rings and chains; outline=true draws an ellipse or box around each shape (removeConvexHulls() clears them). Fixed nodes keep their positions.

More layouts: multipartite, concentric, ordered, circle packing, isometric layers

`circlePackLayout(*fields, padding=4, outlines=True)`

circlePackLayout(field) packs the nodes as circles inside one circle per value of a field; circlePackLayout(field1, field2, …) nests them (each value of field2 a circle inside its field1 circle, and so on). Each node keeps its size, so nothing overlaps; padding is the gap between circles. The group circles are drawn and labelled (outlines=false leaves them out); any later layout removes them. Without a field all nodes are packed into one circle. (Bostock's d3 circle packing.)

`concentricLayout(field, rings=None)`

concentricLayout(field) puts the nodes on rings by their value of a field, the highest value in the centre (degree, a score, a level). A numeric field with many values is cut into `rings` equal ranges (default 8); a text field gets one ring per value. Each ring is ordered so neighbours of the ring inside sit near them, and spaced to fit the nodes' sizes. Compare twopiLayout(root), rings by distance from one node.

`dualCircleLayout(count=None, field='degree')`

dualCircleLayout(count) puts the `count` nodes with the highest value of field (default degree) on an inner circle and everyone else on an outer circle, each outer node near the inner nodes it links to (Gephi's Dual Circle Layout). count defaults to the square root of the number of nodes.

`hairballLayout(neighbors=True, logX=False, logY=False, width=1000, height=1000)`

hairballLayout() draws a Hairball Buster plot (node-neighbour centrality, Matties et al., JHU/APL): nodes sorted by degree, the busiest on the left, each at x = its rank and y = its degree (upwards), with a short tick, level with each node, under the rank of every one of its neighbours, so each row of ticks shows who a node is tied to. Hubs tied mostly to hubs, to leaves, or to a clique stand out at

a glance, and two graphs can be compared plot to plot. Edges are left as they are: g.edges.visible = false shows the plot alone. neighbors=false leaves out the ticks (drawn for up to 5000 edges); logX/logY use log scales. Any later layout removes the axes and ticks.

isometricLayout(field, layers=None, angle=30, gap=None)

isometricLayout(field) stacks the graph in layers by a node field and shows them at an angle, as planes one above the other: each layer keeps the current layout's shape, squashed into a diamond, the lowest value at the bottom (Gephi's Isometric Layout and Network Splitter 3D). Edges between layers run between the planes. A text field gives one layer per value; a numeric field with many values is cut into `layers` equal ranges (default 5). angle tilts the planes (30 = isometric; smaller is flatter); gap is the distance between layers. The layer number goes in the zlayer field, and the planes are outlined; run it again to change the field or angle without compounding. Any other layout removes the outlines.

multipartiteLayout(field, gap=None, vertical=False)

multipartiteLayout(field) puts the nodes in one column per value of a node field (in value order: kind, year, level, ...), ordering each column to reduce edge crossings; vertical=true makes rows instead. gap is the distance between columns (default: they fill 1000 units). The multipartite version of bipartiteLayout.

orderedLayout(field, axis='x', iterations=None, inverted=False, length=1000, gravity=1.0)

orderedLayout(field) fixes each node's position on one axis by a numeric field (a year, a step, an order) and runs a ForceAtlas2 force on the other axis only, so connected nodes come together without breaking the order (Gephi's Ordered Layout and EventGraphLayout). axis="y" orders from top to bottom; inverted=true reverses; length is the span of the ordered axis. Nodes without a value go in the middle.

Display: hulls, legends, background, edges, labels, selection, export

constantLabelSize(on=True)

constantLabelSize() keeps labels the same size on screen as you zoom (they grow with the graph otherwise); each label appears once there is room for it and stays as you zoom further in, so zooming in reveals more labels. Labels inside nodes keep fitting their nodes. constantLabelSize(false) turns it off.

createConvexHull(seq, color='200,200,255,90', shape='hull', rotate=False, padding=6)

createConvexHull(seq) shades the convex hull around the nodes in seq (edges contribute their endpoints). shape="box" or "ellipse" draws a box or an ellipse around them instead; rotate=true turns the box or ellipse to fit the group (the smallest box at any angle) rather than keeping it upright; padding= is the room left around the nodes. encircle(seq) is the same with an ellipse.

curvedEdges(on=True)

curvedEdges() draws every edge as a circular arc running clockwise from source to target, as in Gephi's preview, so a->b and b->a separate and direction reads without arrows. curvedEdges(false) goes back to straight lines; curvedEdges(k) sets the curvature: the arc radius is edge length / k (default 1.2, as in Gephi; 2 gives semicircles).

edgeLabelsAlong(on=True)

edgeLabelsAlong() turns edge labels to run along their edges (kept upright, so they read left to right); edgeLabelsAlong(false) keeps them horizontal (the default).

encircle(seq, shape='ellipse', color=None, rotate=True, padding=10)

encircle(seq) draws a shaded ellipse around a group of nodes, turned to fit it; encircle(seq, "box") a box, encircle(seq, "hull") the convex hull. rotate=false keeps the shape upright. Without a color, each group gets its own (light) colour. Returns the shape: .color, .shape, .rotate and .padding change it; removeConvexHull removes it.

encircleBy(field, shape='ellipse', rotate=True, padding=10, smallest=2)

encircleBy(field) draws one shape around each group of nodes that share a value of a node field (with at least `smallest` members), each in its own colour: encircleBy(louvain_class), encircleBy(dept, "box"). Returns the shapes.

excentricLens(on=True, radius=None, autoSize=None, maxLabels=None)

excentricLens() turns on the lens tool (the toolbar's Lens): a circle that follows the pointer and lists the labels of the nodes under it beside it, each joined to its node by a line (excentric labelling). The wheel resizes it, Shift+wheel scrolls long lists; a click pins it, and then clicking a label selects its node (Shift: add or remove), "Select" takes everything under the lens, and the colour bars filter the list. radius in pixels; autoSize=true sizes the lens to the density under it; maxLabels per side. excentricLens(false) turns it off.

exportEPS(filename='guess.eps')

Save the display as EPS (downloaded by the browser): an image of the current view inside the file. For vector output use exportSVG.

exportGIF(filename='guess.gif')

Save the display as GIF (downloaded by the browser): an image of the current view inside the file. For vector output use exportSVG.

exportJPG(filename='guess.jpg')

Save the display as JPG (downloaded by the browser).

exportPDF(filename='guess.pdf')

Save the display as PDF (downloaded by the browser): an image of the current view inside the file. For vector output use exportSVG.

exportPNG(filename='guess.png')

Save the display as PNG (downloaded by the browser).

exportPS(filename='guess.ps')

Save the display as PS (downloaded by the browser): an image of the current view inside the file. For vector output use exportSVG.

exportSVG(filename='guess.svg')

Save the display as SVG (downloaded by the browser). Vector: the whole graph, not just the visible area.

getConvexHulls()

All hulls currently shown.

getDisplayBackground()

The display background color, or None when it follows the light/dark theme.

GradientLegend(start, end, lo=0, hi=1, ticks=None)

A color ramp legend: GradientLegend(startColor, endColor, min, max[, tickEvery]) shows the ramp from min to max, with a tick mark every tickEvery units.

hideOverlappingLabels(on=True)

hideOverlappingLabels() hides labels that would overlap a label of a bigger node, rechecked as you zoom; hideOverlappingLabels(false) shows all.

hoverFields(*fields, nodes=None, edges=None)

hoverFields(dept, degree) sets the fields the tooltip shows as soon as the pointer is on a node or edge; by default it shows only the label. Waiting a moment shows every field. Node fields go to nodes' tooltips and edge fields to edges'; a kind you don't mention keeps its fields. hoverFields(nodes=["dept"], edges=["weight"]) names them by kind (for a field both have, such as color); hoverFields() goes back to labels only. View → Hover fields… picks them from a list.

labelOutlines(on=True)

labelOutlines() draws a halo in the background colour around labels so they stay readable over edges; labelOutlines(false) turns it off.

`labelPlacement(method='candidates')`

labelPlacement("candidates") or labelPlacement("particles") chooses how labels with labelpos "outside" (and edge labels, which slide along their edge) find room. "candidates" (default) tries 16 positions around each node, biggest nodes first, then re-places labels that still collide; "particles" also searches further out, ring by ring, and draws a leader line to labels it places away from their node. Both avoid other labels, nodes and edges.

`Legend()`

A legend box on the display. `l = Legend(); l.add(bob, "managers")`.

Entries snapshot the look of the node, edge or hull at the time they are added.

`optimizeLabels(seconds=2)`

optimizeLabels() improves the positions of labels placed "outside" (and of edge labels) by simulated annealing for a few seconds, trading overlaps with labels, nodes and edges against the preferred positions; the result stays until the graph is replaced, labelPlacement changes or you run it again. Good before exporting a figure.

`overrideArrowLength(length)`

Fix the arrowhead length (graph units) for all directed edges; None resets.

`overrideArrowWidth(width)`

Fix the arrowhead width (graph units) for all directed edges; None resets.

`raiseChanged(on=True)`

raiseChanged() (the default) draws a node or edge on top of the others once its appearance changes (colour, size, shape, label, visibility), so what you just changed isn't hidden under its neighbours; edges stay under nodes. raiseChanged(false) keeps the drawing order fixed.

`removeBackgroundImage()`

Remove the background image set with setBackgroundImage.

`removeConvexHull(hull)`

Remove one hull.

`removeConvexHulls(seq=None)`

Remove the given hulls, or all hulls.

`select(seq=None)`

select(seq) selects these nodes on the display (edges select their endpoints) and binds them to `sel`, as a box, lasso or shift-click does; select() clears the selection. On the display: shift-drag draws a box, alt-drag a lasso, shift-click adds or removes one node.

`setBackgroundImage(source, x=0, y=0)`

Show an image (a file you opened, or a URL) behind the graph with its top-left corner at (x, y) in graph coordinates.

`setDisplayBackground(color)`

Set the display's background color; setDisplayBackground(None) goes back to the theme's (white, or dark in dark mode).

`setEdgeColorMode(mode='original')`

setEdgeColorMode("mixed") draws each edge in the blend of its endpoints' colours; "source" / "target" use one endpoint's colour; "original" (the default) uses the edge's own color field. As in Gephi's preview.

`startMovie(fps=30, filename='guess.webm')`

Record the display (graph canvas) as a WebM video until stopMovie(). Everything shown — layouts, morphs, edits, panning — is captured.

`stopMovie()`

Stop recording and download the video.

Styles and visual mappings

`applyStyle(name)`

applyStyle(name) runs a style's commands (one undo step).

`defineStyle(name, commands)`

defineStyle(name, [command, …]) saves a named style: Gython lines that set colours, sizes, labels, legends or display settings. Styles are saved with the session; the style panel builds them for you.

`getStyles()`

The saved styles: {name: [commands]}.

`labelBy(field, show=True)`

labelBy(field) sets each node's (or edge's) label to its value of field and shows the labels; labelBy(field, false) sets them without showing.

legendFor(field, title=None)

legendFor(field) adds a legend explaining the current colours by field: one entry per value (for categories, as after colorize), or a gradient from the lowest to the highest value (for numbers, as after colorBy). The legend follows the colours: run colorize or colorBy again and it updates (until you add or remove entries yourself).

polygon(sides)

polygon(sides) is the node style for a regular polygon with that many sides (3 to 64), a corner at the top: (dept == 'dept1').style = polygon(5). polygonBy(field) sets one per node from a field.

polygonBy(field, smallest=3, largest=12)

polygonBy(field) draws each node as a regular polygon whose number of sides comes from a node field (as Gephi's Polygon Shaped Nodes): a whole number field gives the sides directly (kept within smallest..largest); any other field gives each value its own polygon, 3, 4, 5, … sides, the most common value first.

removeLegend(legend)

removeLegend(lg) or removeLegend(2) removes one legend (the number is the one shown as <legend 2: …>; the × on a legend runs this).

removeLegends()

removeLegends() removes every legend.

removeStyle(name)

removeStyle(name) deletes a saved style.

shapeBy(field)

shapeBy(field) gives each value of a node field its own shape (circle, square, rounded square, triangle, diamond, hexagon, octagon, then repeating), the most common value first.

table(seq=None)

table() opens the data table of all nodes; table(seq) shows just those nodes or edges (e.g. table(sel), table(freq > 10), table(g.edges)).

Pie nodes, borders and edge bundling

borderBy(field, width=2.5)

borderBy(field) gives node borders a distinct colour per value of a field (as colorize does for fills) and widens them.

bundleEdges(field=None, strength=0.85, method=None, neighbors=10, angleStrength=1.2, stiffness=0.1, compatibility=0.6, cycles=6)

bundleEdges() bundles edges that run alike, so the main flows stand out of a hairball. Three methods: bundleEdges() MINGLE (multilevel, saves "ink"): fast, tree-like bundles; good for large graphs bundleEdges(method="fdeb") force-directed edge bundling: smooth, organic curves; up to a few thousand edges bundleEdges(louvain_class) through the centres of the groups of a node field strength 0..1 blends toward straight lines (0 = straight). MINGLE: neighbors (edges compared with each), angleStrength (higher allows sharper turns). FDEB: stiffness (of each edge), compatibility (0..1, how alike edges must be to attract), cycles. Layouts and removeBundles() straighten them again.

pieChart(fields, palette=None)

pieChart([f1, f2, ...]) draws each node as a pie whose slices are the node's values of those numeric fields; palette=colors sets slice colours.

pieNeighbors(field)

pieNeighbors(field) draws each node as a pie of its neighbours' values of a categorical field (the most common values; the rest as "other").

removeBundles()

Draw edges straight again after bundleEdges.

removePies()

Draw nodes normally again (after pieChart / pieNeighbors).

Grouping: collapse and expand

collapse(seq, name=None)

collapse(seq) replaces those nodes with one meta-node (see module help); collapse(seq, "name") names it. Returns the meta-node.

collapseBy(field, smallest=2)

collapseBy(field) collapses each group of nodes sharing a value of field (with at least `smallest` members) into a meta-node named after the value.

expand(meta)

expand(metaNode) brings back the nodes a meta-node stands for (also: double-click it). Returns them.

expandAll()

Expand every collapsed group (repeatedly, for nested groups).

isGroup(node)

True if node is a collapsed group (meta-node).

Several graphs: subgraphs and tabs

closeGraph(name)

closeGraph(name) discards a graph that isn't the current one.

graphNames()

The graphs in this session; the current one first.

newGraph(name='untitled')

newGraph(name) starts an empty graph and switches to it.

project(field, keep, method='count', name=None)

project(kind, "woman") turns a two-mode network (people and the events, groups or papers they belong to) into a one-mode network of just the nodes whose `kind` is "woman": two are linked when they share a neighbour of the other kind. It opens as a new graph (the two-mode one stays; use() goes back), keeping fields and positions. Each edge gets `shared` (how many neighbours in common) and `weight`, by method: "count" the number shared (default) "jaccard" shared / neighbours of either (0..1; removes the size effect of very active nodes) "newman" each shared neighbour adds 1/(its degree - 1), so a small event counts more than a crowded one Each kept node also gets `affiliations`: its number of other-kind neighbours. E.g. project(kind, "event") gives the event network.

renameGraph(name)

renameGraph(name) renames the current graph.

subgraph(seq=None, name=None, induced=False)

subgraph(seq) makes a new graph of exactly what seq selects, keeping fields, positions and look, and switches to it: a set of nodes gives those nodes and the edges among them (subgraph(dept == "dept3")); a set with edges gives just those edges and their endpoints, plus any nodes listed (subgraph(freq > 100) is the heavy-tie network; subgraph(shortestPath(a, b)) is the path). induced=true keeps every edge among the selected nodes instead. subgraph(seq, "name") names the new graph; use(name) goes back. As showOnly does for the display.

use(name)

use(name) switches to another graph of this session (see graphNames()).

Time: ranges, years and the timeline

showTime(t, field=None, window=None)

showTime(t) shows only the nodes and edges whose ranges contain time t (elements without ranges stay visible); showTime(t, field) shows those whose numeric field is <= t (window=w: only within the last w). Edges are shown only when both ends are.

Presentations (slides)

addSlide(note='', camera=None, at=None)

addSlide(note) saves the current graph (as a state) and view as a slide; camera=[x, y, zoom] is filled in by the page; at=k inserts it at k.

moveSlide(index, to)

moveSlide(k, j) moves slide k to position j.

noteSlide(index, note)

noteSlide(k, text) changes slide k's note.

removeSlide(index)

removeSlide(k) deletes slide k.

showSlide(index, duration=1200)

showSlide(k) morphs the graph to slide k's state and moves the camera.

slides()

The slides' notes, in order.

Data from the web: URLs, SPARQL, NDEx, Neo4j, packages

installPackage(name)

installPackage("name") installs a Python package into this session (pure-Python wheels from PyPI, or Pyodide's own packages such as scipy), in the background; a note appears when it's ready to import.

makeFromCypher(query, url='http://localhost:7474', user=None, password=None, database='neo4j')

makeFromCypher(query, url, user, password) runs a Cypher query on a Neo4j server's HTTP API and builds the graph from the nodes and relationships it returns (properties become fields).

`makeFromFile(filename)`

makeFromFile(filename) opens a graph file in any format GUESS reads, choosing the reader from the file's extension or, when the extension doesn't say (.xml, .txt, .json, …), from its content: GraphML, XGMML, SBML, PSI-MI, KGML or GPML in .xml; PSI-MI TAB, SIF or CSV in .txt; Cytoscape.js JSON or CX in .json. Opening a file in the page (File → Open, or dropping it) does this.

`makeFromNDEx(uuid, server='https://www.ndexbio.org')`

makeFromNDEx(uuid) opens a public network from the NDEx repository (Cytoscape's network commons), with its attributes and layout.

`makeFromOpenAlex(search, limit=200, mode='citations')`

makeFromOpenAlex("graph drawing") searches OpenAlex (the open catalogue of scholarly works, CC0) for works with those words in the title, takes the `limit` most cited, and opens their citation network (citing -> cited) with `title`, `year`, `cites` (citations from anywhere), `venue` and `firstauthor`. mode="authors" opens a two-mode paper-author network instead (kind = paper or author), ready for project(kind, "author"). Ready for mainPath(), leidenClusters() and the Text cookbook's recipes.

`makeFromSPARQL(query, endpoint='https://query.wikidata.org/sparql', directed=True)`

makeFromSPARQL(query) builds a graph from a SPARQL query whose rows bind ?source and ?target (optionally ?sourceLabel / ?targetLabel, used as labels; any other variables become edge fields). E.g. a Wikidata query for "influenced by" links.

`makeFromSTRING(genes, species=9606, score=0.7, add=0, server='https://string-db.org')`

makeFromSTRING(["TP53", "MDM2", "ATM"]) opens the protein interactions among those genes from the STRING database (string-db.org): species is an NCBI taxon (9606 human, 10090 mouse, 4932 yeast, 7227 fly, 6239 worm, 3702 Arabidopsis, 511145 E. coli), score the minimum combined confidence (0.4 medium, 0.7 high, 0.9 highest), add=n also brings in the n proteins that interact most with the query. Genes may be a list or one string separated by spaces/commas. Nodes get `description`, `stringId` and `query` (true for the genes you asked for); edges get `score` (also `weight`) and the evidence behind it: `experiments`, `databases`, `textmining`, `coexpression`, `neighborhood`, `fusion`, `cooccurrence` (each 0..1). STRING data are CC BY 4.0: cite it.

`makeFromURL(url, format=None)`

makeFromURL(url) downloads a graph (GDF, GEXF, GraphML, GML, DOT, Pajek, DL, CSV, RDF (Turtle, N-Triples, RDF/XML), Cytoscape's SIF, NNF, Cytoscape.js JSON, CX/CX2 and XGMML, PSI-MI TAB or XML, SBML, BioPAX, KEGG KGML, WikiPathways GPML, a .guess session or a .py script) and opens it; format=".gexf" etc. when the URL doesn't end in the extension (the biological formats are also recognised by their content).

`makeFromWikipedia(title, limit=150, lang='en')`

makeFromWikipedia("Graph drawing") opens a page's link network: the page, up to `limit` articles it links to, and every link among them (directed: linking page -> linked page). Node `hop` is 0 for the start page and 1 for the others; `indegree` then shows which linked articles the neighbourhood itself points to most (identifier pages such as "Doi (identifier)", linked by citation templates, are left out). lang= picks the language edition ("de", "fr", ...). Wikipedia text is CC BY-SA; the link structure is what's loaded here.

`sparql(query, endpoint='https://query.wikidata.org/sparql')`

sparql(query[, endpoint]) runs a SPARQL query (Wikidata by default) and returns the rows as a list of {variable: value} dicts.

Standalone web pages (exportHTML)

`exportHTML(filename='graph.html', title=None, hidden=False, fields=True)`

exportHTML("graph.html") writes the graph as one self-contained web page: open it in any browser, from disk or a web server, with nothing else to install. It shows the graph as drawn here (colours, sizes, shapes, labels, legends, images) and lets the reader pan and zoom, search by name, hover for a node's fields and click to see its neighbours. hidden=true includes hidden nodes and edges; fields=false leaves the data fields out (a smaller, more private page).

Live graph streams (Gephi streaming protocol)

`stopStream()`

stopStream() stops every stream started with streamFrom.

`streamFrom(url)`

streamFrom(url) follows a live graph stream in the Gephi streaming format (Gephi's Graph Streaming plugin): an HTTP URL that sends one JSON event per line, such as Gephi's own "http://localhost:8080/workspace1?operation=getGraph", or a ws:// or wss:// WebSocket sending one event per message. Nodes and edges are added, changed and removed as events arrive (between commands, so a long command delays them); new nodes appear next to a neighbour. Streamed changes are not undo steps. stopStream() stops it. The server must allow cross-origin requests (CORS) from this page.

AI assistants (MCP bridge)

`connectAssistant(port=7777)`

connectAssistant() connects this page to an AI assistant through the GUESS MCP bridge (mcp/guess-mcp.mjs, which the assistant starts; see the manual). The assistant can then run Gython here, read a summary of the graph and the command help, and see the view. Everything it runs appears in the console marked "assistant" and can be undone. disconnectAssistant() ends it. port= must match GUESS_MCP_PORT.

disconnectAssistant()

disconnectAssistant() disconnects the AI assistant (see connectAssistant).

Cytoscape formats: SIF, NNF, Cytoscape.js JSON, CX/CX2, XGMML, .cys sessions

exportCYJS(filename='graph.cyjs')

exportCYJS("graph.cyjs") writes the graph as Cytoscape.js JSON, which Cytoscape (File → Import → Network from File) and Cytoscape.js read: every field as data, positions, and each node's colour, size and shape as data (color, width, height, shape) for a style to map.

makeFromCX(filename, directed=True)

makeFromCX("network.cx2") replaces the graph with a network in CX2 or CX (version 1), the formats of NDEx and Cytoscape's exchange format: attributes (typed as declared), node positions, and the default and per-node colours, sizes and shapes. Nodes are named by their `name` (a name two nodes share gets the id added). CX edges have a direction, as in Cytoscape; directed=false loads them undirected.

makeFromCYJS(filename)

makeFromCYJS("network.cyjs") replaces the graph with Cytoscape.js JSON, as Cytoscape exports for the web (File → Export → Network to File → Cytoscape.js JSON) and as Cytoscape.js uses: each node's and edge's data become fields, positions are kept, and compound (group) nodes become the field `parent` of their members. Nodes are named by their `name` (or label); a name two nodes share gets the node's id added. exportCYJS writes the format.

makeFromCYS(filename, network=None)

makeFromCYS("session.cys") opens a network from a Cytoscape session file: its attributes, and positions, colours, sizes and shapes from its view. A session can hold several networks: the first opens and the others are listed; makeFromCYS(file, "name") or makeFromCYS(file, 2) opens another, and makeFromCYS(file, "all") opens each as a graph of its own (tabs above the graph; use(name) switches). Styles (Cytoscape's visual mappings) aren't read.

makeFromNNF(filename)

makeFromNNF("nested.nnf") replaces the graph with a Cytoscape nested network file: lines `network node` (a node in a network) and `network source type target` (an edge), # comments. Every network's nodes and edges are loaded together: `network` says which network(s) each node and edge is in, and a node named after a network (one that stands for it, nested inside another) has that name in `nested`. So showOnly(network == 'M1') shows one network, and collapse(network == 'M1') folds a nested network into a node.

makeFromSIF(filename, directed=False)

makeFromSIF("network.sif") replaces the graph with a Cytoscape SIF file (simple interaction format): each line is `source type target [target …]`, or a lone node name. If the file has tabs,

fields are tab-separated and names may contain spaces; otherwise names are separated by spaces. The type goes in the edge field `interaction` (colorize(interaction) shows it). directed=true makes every edge run from source to target.

`makeFromXGMML(filename)`

makeFromXGMML("network.xgmml") replaces the graph with an XGMML file, Cytoscape's XML format: node and edge attributes (typed: string, integer, real, boolean; lists joined with ';'), and each node's graphics: position, size, fill and outline colour and shape. Files with bare '&' characters (a common fault) are repaired as they are read.

Biology formats: PSI-MI (MITAB, XML), SBML, BioPAX, KEGG KGML, WikiPathways GPML

`makeFromBioPAX(filename, mode='sif')`

makeFromBioPAX("pathway.owl") opens a BioPAX level 3 pathway (as Reactome, Pathway Commons and WikiPathways export them) as a simple network of molecules, after Pathway Commons' SIF rules: a reaction links what it uses to what it produces ("used-to-produce"); the controller of a reaction (a catalysis or other control) links to what it changes ("controls-state-change-of", or "controls-production-of" for small molecules) with the `sign` (activation, inhibition); the regulator of a gene's expression links to its product ("controls-expression-of"); molecular interactions link their participants ("interacts-with"); and each complex is a node linked to its components ("in-complex-with"). Forms of the same protein (states, places) are one node. mode="reactions" keeps the reactions as small nodes instead, with left, right, controller and participant edges.

`makeFromGPML(filename)`

makeFromGPML("WP707.gpml") opens a WikiPathways pathway (GPML 2013a or 2021), drawn as on WikiPathways: data nodes at their positions, with their colours, shapes and database references (`xref_db`, `xref_id`). Interactions become edges with their `arrow` (mim-conversion, mim-catalysis, mim-inhibition, Arrow, TBar, …). An interaction that ends on another one (a catalysis on a conversion) meets it at a small anchor node; groups that interactions point at become nodes, and members have the group in `group`.

`makeFromKGML(filename)`

makeFromKGML("hsa04115.xml") opens a KEGG pathway map (KGML), drawn as on KEGG: genes, compounds and linked maps at their positions, in their colours and shapes, labelled with the first name. Relations become directed edges with `kind` (PPrel, GErel, ECrel, PCrel) and `subtype` (activation, inhibition, phosphorylation, …); reactions become edges from substrates to products with the `enzyme`. Entries in a group (a complex) have the group in `group`.

`makeFromMITAB(filename, merge=True)`

makeFromMITAB("interactions.txt") opens molecular interactions in PSI-MI TAB (MITAB 2.5, 2.6 or 2.7), as IntAct, BioGRID, MINT and the IMEx databases export them. Each interactor becomes a node named by its gene name when the file gives one (else its short label or identifier), with `ident`, `taxid`, `species`, `kind` and `aliases`. Each pair becomes an edge with the detection `method`, `kind`

(interaction type), `pubmed`, `author`, `sourcedb`, `interaction` ids and IntAct's `miscore`; repeated evidence for a pair is merged into one edge with an `evidence` count (merge=false keeps one edge per line). An interactor paired with itself is a self-loop.

`makeFromPSIMI(filename, merge=True)`

makeFromPSIMI("interactions.xml") opens molecular interactions in PSI-MI XML (2.5 or 3.0), with the same nodes and fields as makeFromMITAB. An interaction with more than two participants is expanded into pairs: from the bait to each prey when a bait is named (spoke expansion), otherwise between every pair (matrix expansion); the edge field `expansion` says which.

`makeFromSBML(filename, reactions='nodes')`

makeFromSBML("model.xml") opens a systems biology model (SBML level 2 or 3). Species become nodes (`compartment`, `initial` amount or concentration, `sbo`); each reaction becomes a small grey node, with directed edges from its reactants and modifiers and to its products (`role`, `stoichiometry`). reactions="edges" draws each reaction as edges from reactants (and modifiers) to products instead. A species name used in two compartments gets the compartment added, as "ATP [cytosol]". Positions from the SBML layout package are kept.

Import and export: Excel, RDF, Bluesky, JSON, KML, shapefiles, standalone HTML

`exportJSON(filename='graph.json', format='graphology')`

exportJSON("graph.json") writes the graph for sigma.js and graphology (graph.import(data)): nodes with label, x, y, size, colour and every data field, edges with weight, size, colour and fields; hidden elements are marked hidden. format="sigma" writes the older sigma.js JSON ({nodes: [{id, label, x, y, size, color, attributes}], edges: [...]}) that Gephi's JSON Exporter makes. Offered as a download.

`exportKML(filename='graph.kml', lat=None, lon=None)`

exportKML("graph.kml") writes the graph for Google Earth and other maps: each node with coordinates a placemark (its colour, label and fields), each edge between two placed nodes a line (its colour and width). Coordinates come from the latitude/lat and longitude/lon/lng fields, or name them: exportKML("x.kml", lat="y", lon="x"). Offered as a download.

`exportSHP(filename='graph.zip', lat=None, lon=None)`

exportSHP("graph.zip") writes shapefiles for GIS (QGIS, ArcGIS) in a zip: nodes.shp (points, with name, label, colour and the node fields) and edges.shp (lines between placed nodes, with source, target, weight, colour and the edge fields), in WGS 84 longitude/latitude. Coordinates come from latitude/lat and longitude/lon/lng fields, or name them with lat= and lon=. Field names are cut to 10 characters, as the format requires. Offered as a download.

`makeFromBluesky(handle, depth=1, limit=100, mode='follows', maxNodes=500)`

makeFromBluesky("alice.bsky.social") opens a Bluesky account's network from the public API: the account and up to `limit` accounts it follows, with directed "follows" edges. mode="followers"

fetches who follows it instead, “both” both. depth=2 also fetches the same for each account found, adding the ties among them and new accounts, up to maxNodes accounts in all. Nodes get handle, did, followers, follows, posts and description; labels are display names.

`makeFromCooccurrence(filename, column=None, separator=';', sheet=None, keep=True)`

makeFromCooccurrence(“papers.csv”, “authors”) builds a network of the values that share a cell: each row’s list (split at separator, “;” by default; also “,” or “|”) links every pair in it, as co-authors on a paper or keywords on an article. Nodes get `count` (rows they appear in); edges `weight` (rows they share). Works on .csv, .tsv and .xlsx (sheet= picks a sheet); column= is a header name or a number (0 = first; default: the first column whose cells hold lists). keep=true also adds a node for a value that never shares a row.

`makeFromExcel(filename, sheet=None, nodes=None, directed=False, source=None, target=None, interaction=None, weight=None, header=True)`

makeFromExcel(“data.xlsx”) replaces the graph with an Excel workbook read as makeFromCSV reads tables: an edge sheet (Source, Target[, Type, Weight, Label, …]), an adjacency matrix or list; with a node sheet (Id[, Label, …]) too when the workbook has one. sheet= picks the edge sheet by name or number (0 = first); nodes= names the node sheet, or another file (.xlsx or .csv). Other columns become fields; directed=true makes edges without a Type directed. makeFromCSV(“data.xlsx”) does the same. source=, target=, interaction=, weight= and header= choose the edge sheet’s columns as in makeFromCSV.

`makeFromRDF(filename, format=None, types=True)`

makeFromRDF(“data.ttl”) replaces the graph with RDF data (Turtle, N-Triples or RDF/XML, by extension or content; format=“turtle”, “nt” or “xml” to say). Each resource becomes a node (labelled by rdfs:label, foaf:name, skos:prefLabel or its local name; its IRI in `uri`), each property linking two resources an edge (its name in `predicate`), and each literal property a node field (numbers and booleans typed). types=false keeps rdf:type as a `type` field instead of edges to class nodes. For SPARQL endpoints use makeFromSPARQL.

States and morphing

`getEdgeAppearAfter()`

The current edgeAppearAfter fraction (see setEdgeAppearAfter).

`getEdgeDisappearBy()`

The current edgeDisappearBy fraction (see setEdgeDisappearBy).

`getNodeAppearAfter()`

The current nodeAppearAfter fraction (see setNodeAppearAfter).

getNodeDisappearBy()

The current nodeDisappearBy fraction (see setNodeDisappearBy).

getStates()

Names of the saved states.

ls(state)

Load a saved state: restores which nodes/edges exist and all their values.

morph(state, duration=1000)

Animate from the current display to a saved state over `duration` ms.

scratch()

with scratch(): runs the block, then puts the graph back as it was (which nodes and edges exist, and every value): for trying things out, such as removing nodes and measuring the result. Fields made inside the block are removed too; print or keep in variables what you want to keep.

setEdgeAppearAfter(v)

Fraction (0..1) of a morph by which edgeAppearAfter takes effect.

setEdgeDisappearBy(v)

Fraction (0..1) of a morph by which edgeDisappearBy takes effect.

setNodeAppearAfter(v)

Fraction (0..1) of a morph by which nodeAppearAfter takes effect.

setNodeDisappearBy(v)

Fraction (0..1) of a morph by which nodeDisappearBy takes effect.

ss(state)

Save the current graph (membership and all field values) as a named state.

Ranges (existence intervals)

addRange(obj, spec)

Attach an existence range like “1,5-10,20” to nodes/edges (adds to any existing range).

getRange(obj)

The range of a node or edge as a string.

`searchContained(spec)`

Nodes and edges whose range lies inside the given range.

`searchContains(spec)`

Nodes and edges whose range contains the given range.

`searchExact(spec)`

Nodes and edges whose range equals the given range.

`searchOverlap(spec)`

Nodes and edges whose range overlaps the given range.

Charts

`liveCharts(on=True)`

liveCharts() makes every chart live, now and from now on: each is recomputed after any command that changes the graph or the display. liveCharts(False) turns that off. (The Live button on a chart does it for one chart.)

`plotDistrib(field, sortField=None)`

Each element's value of field, in rank order (or ordered by sortField).

`plotHistogram(field, bins=10)`

Histogram of a numeric field's values in `bins` equal-width bins.

`plotSizes(field)`

Bar chart of how many elements share each value of field (largest first).

`plotSizesPie(field)`

Pie chart of how many elements share each value of field.

Interface building (panels, widgets, timers, events)

`after(seconds, fn)`

Call fn() once after `seconds` seconds; returns a timer with .stop().

`Button(*a, **k)`

Button(text, onClick=fn): calls fn() when clicked; .enabled turns it on/off.

`Checkbox(*a, **k)`

Checkbox(text, checked=False, onChange=fn): calls fn(checked) when toggled.

`Dropdown(*a, **k)`

Dropdown(options, value=None, label="", onChange=fn): a choice list; calls fn(value) when the selection changes.

`every(seconds, fn)`

Call fn() every `seconds` seconds; returns a timer with .stop().

`Label(*a, **k)`

Label(text): a line of text in a panel; set .text to change it.

`openURL(url)`

Open a web page in a new browser tab.

`Panel(*a, **k)`

Panel(title, *widgets, position="bottom", vertical=False). Show with ui.dock(p).

`Slider(*a, **k)`

Slider(min, max, value=None, step=1, label="", onChange=None, onSlide=None). onChange fires on release, onSlide while dragging.

`stopTimers()`

Stop all timers.

`TextField(*a, **k)`

TextField(value="", placeholder="", onEnter=None): onEnter(text) on Return.

Sessions and apps

`exportApp(filename='app.guess', script=None)`

Save a session that opens as an app: with ?app=1 the console is hidden and `script` (Gython source, or the name of a .py file) runs on load — e.g. to dock panels and set graphevents handlers.

`makeFromSession(filename)`

Load a .guess session file, replacing the current graph.

saveSession(filename='session.guess')

Save the graph, fields, states, hulls, legends and view to a .guess file (downloaded by the browser). User variables are not saved.

SQL

sql(query)

Two tables are materialised in an in-memory SQLite database on each call:

```
nodes(name, <every node field>)
edges(__edgeid, node1, node2, directed, <every edge field>)

sql("SELECT dept, count(*) FROM nodes GROUP BY dept ORDER BY 2 DESC")
sql("SELECT name FROM nodes WHERE x > y")            -> a set of nodes
sql("SELECT __edgeid FROM edges WHERE freq > 100")   -> a set of edges
sql("UPDATE nodes SET color = 'red' WHERE dept = 'dept3'")
```

SELECTs whose only column is `name` (nodes) or `__edgeid` (edges) return graph elements, so they combine with the rest of Gython; other queries return rows. UPDATEs are written back through the normal field setters (the display updates). Inserting or deleting rows is not supported — use addNode/remove.

Colors

averageColor(c1, c2)

The color halfway between two colors.

distinctColors(count, seed=1)

distinctColors(count): count clearly different colours (as "r,g,b" strings), the most different first — the categorical palette colorize(field) uses.

generateColors(*args)

generateColors(start, end, n) or generateColors(start, middle, end, n).

randomColor(alpha=None)

A random color "r,g,b"; randomColor(alpha) adds an alpha (0..255).

Examples

The same examples are searchable in the app's Help panel and command palette (Cmd/Ctrl+K).

Basics

Colour nodes by a query

```
(freq > 100).color = red
```

Nodes with a field value

```
dept == 'dept3'
```

Pattern match names

```
name like 'v1%'
```

Combine queries

```
(dept == 'dept1') & (totaldegree > 3)
```

Loop over a set

```
for n in (dept == 'dept3'):
    n.labelvisible = true
```

Edges between two groups

```
(dept == 'dept1') <-> (dept == 'dept3')
```

Directed edges

```
v1 -> g.nodes
```

Any edges touching a node

```
v1 ? g.nodes
```

Neighbours of a node

```
v1.getNeighbors()
```

Add nodes and edges

```
addNode('zed', color=red)
addEdge(zed, v1)
```

Remove and restore

```
remove(totaldegree == 0)
add(g.deleted)
```

Undo the last command

```
undo()
```

Appearance

A colour per category

```
colorize(dept)
```

Gradient by value

```
colorBy(freq, yellow, red, 'log')
```

Size by importance

```
sizeBy(pagerank, 5, 40, 'sqrt')
```

Size by rank of a field

```
resizeLinear(totaldegree, 5, 30)
```

Labels from a field

```
labelBy(dept)
```

Shapes per category

```
shapeBy(dept)
```

Legend for a mapping

```
colorize(dept)
legendFor(dept)
```

Edge width by weight

```
sizeBy(freq, 1, 8)
```

Curved, mixed-colour edges

```
curvedEdges()
setEdgeColorMode('mixed')
```

Readable labels

```
labelOutlines()
hideOverlappingLabels()
```

Shade a group

```
createConvexHull(dept == 'dept1', '200,200,255,90')
```

Ellipse or box around a group

```
encircle(dept == 'dept1')
encircle(dept == 'dept2', 'box')
```

A shape around every group

```
encircleBy(dept)
```

Dark background

```
setDisplayBackground(black)
```

Save a named style

```
defineStyle('by dept', ['colorize(dept)', 'legendFor(dept)'])
applyStyle('by dept')
```

Polygon nodes

```
(dept == 'dept1').style = polygon(5)
polygonBy(dept, 3, 8)
```

Word cloud of a text field

```
wordCloud(dept)
```

Layouts

ForceAtlas2

```
forceAtlas2Layout()
```

ForceAtlas2, tighter clusters

```
forceAtlas2Layout(linLog=true, preventOverlap=true)
```

GEM layout

```
gemLayout()
```

Kamada-Kawai

```
kkLayout()
```

Hierarchy

```
sugiyamaLayout()
```

Hive plot

```
hiveLayout(dept, pagerank, axes=5)
```

Radial around a node

```
radialLayout(v1)
```

Remove overlaps

```
noverlapLayout()
```

Pack components

```
binPackLayout()
```

Rotate / mirror

```
rotateLayout(90)
mirrorLayout()
```

One column per group

```
multipartiteLayout(dept)
```

Rings by importance

```
concentricLayout(degree)
```

Hubs inside, the rest outside

```
dualCircleLayout(10)
```

Order by a field, untangle the rest

```
orderedLayout(degree)
```

Hairball Buster plot

```
hairballLayout()
g.edges.visible = false
```

Nested circles by group

```
circlePackLayout(dept)
```

Layers in 3D-style planes

```
isometricLayout(dept)
```

Two centres of gravity

```
forceAtlas2Layout(gravitySources=[v1, v73])
```

Spread out part of a layout

```
rescaleLayout(dept == 'dept1', 1.5)
```

Stars, rings and grids (structure-aware)

```
motifLayout(outline=true)
```

Analysis

Summary statistics

```
graphStats()
```

Communities

```
louvainClusters()
colorize(louvain_class)
```

Most central nodes

```
sortBy(pagerank)[-10:]
```

Label the hubs

```
(betweenness > 1000).labelvisible = true
```

Shortest path

```
shortestPath(v1, v14).color = red
```

Distance heat map

```
heatMap(v1)
```

Groups as charts

```
plotSizesPie(dept)
```

Distribution chart

```
plotHistogram(pagerank)
```

SQL over fields

```
sql('select dept, count(*) from nodes group by dept')
```

Weak points

```
bridges().color = red
cutNodes().size = 30
```

Backbone

```
showOnly(spanningTree(maximum=True))
```

Brokers

```
sortBy(constraint)[:5]
```

Homophily

```
assortativity(dept)
mixingMatrix(dept)
```

Compare partitions

```
louvainClusters()
compareClusters(louvain_class, dept)
```

Any networkx algorithm

```
import networkx as nx
setField(nx.community.greedy_modularity_communities(toNetworkX()), 'greedy')
colorize(greedy)
```

Classic networks

```
makeSample('karate')
colorize(club)
```

Leiden communities

```
leidenClusters()
colorize(leiden_class)
```

Communities in directed graphs

```
infomapClusters()
colorize(infomap_class)
```

Similar nodes

```
similarTo(v1).color = red
```

Likely missing links

```
predictLinks(10)
```

Find a pattern

```
findPattern([('a','b'), ('b','c'), ('c','a')])
```

Large-graph layout

```
drlLayout()
```

Layered drawing (Graphviz)

```
dotLayout(rankdir='LR')
```

Stress layout (Graphviz)

```
neatoLayout()
```

Remove overlaps, keep shape

```
removeOverlap()
```

Labels inside nodes

```
g.nodes.labelvisible = true
fitNodesToLabels()
```

Labels where there's room

```
g.nodes.labelvisible = true
g.nodes.labelpos = 'outside'
```

Labels off the edges

```
g.nodes.labelpos = 'outside'
labelPlacement('particles')
```

Polish label positions

```
optimizeLabels(2)
```

Labels that stay readable when zooming

```
constantLabelSize()
```

Excentric labelling lens

```
excentricLens(radius=80)
```

Label size by a field

```
labelSizeBy(pagerank, 0.8, 2)
```

Edge labels

```
labelBy(freq)
edgeLabelsAlong()
```

Katz and prestige

```
highest(katz, 5)
highest(domain_prestige, 5)
```

Bridging centrality

```
colorBy(bridging, yellow, red)
highest(bridging, 5)
```

Weighted bridging

```
weightedBridging()
highest(wbridging, 5)
```

Maximal cliques

```
cliques(4)
```

Test link prediction

```
evaluateLinks('all')
```

Coordinates from MDS

```
mds(2)
scatterLayout(mds1, mds2)
```

Chinese Whispers communities

```
chineseWhispersClusters()
colorize(cw_class)
```

Markov clustering (MCL)

```
mclClusters(inflation=2)
colorize(mcl_class)
```

Clusters of nearby nodes

```
dbscanClusters()
colorize(dbscan_class)
```

Dense complexes (MCODE)

```
mcodeClusters()
(mcode_class == 0).color = red
```

Overlapping communities

```
cs = cliquePercolationClusters(3)
showOverlaps(cs)
```

Communities of edges

```
linkCommunities()
showOverlaps(link_classes)
```

Bundle edges

```
bundleEdges()
```

Force-directed edge bundling

```
bundleEdges(method='fdeb')
```

Filtering and selection

Show only a core

```
showOnly(kCore(3))
```

Ego network

```
showOnly(egoNetwork(v1, 2))
```

Largest component

```
showOnly(giantComponent())
```

Edges inside communities

```
showOnly(intraEdges(louvain_class))
```

Show everything again

```
showAll()
```

The k-brace core

```
showOnly(kBrace(2))
```

Keep nodes listed in a file

```
showOnly(readNames('keep.txt'))
```

Select from code

```
select(dept == 'dept2')
```

Act on the selection

```
sel.color = orange
center(sel)
```

Table of a set

```
table(dept == 'dept3')
```

Files and sessions

Load Gephi's format

```
makeFromGEXF('graph.gexf')
```

Load spreadsheets

```
makeFromCSV('edges.csv', 'nodes.csv')
```

Export for Gephi

```
exportGEXF('graph.gexf')
```

Save a session

```
saveSession('work.guess')
```

Save what you ran

```
saveHistory('work.py')
```

Snapshot and morph

```
ss('before')
circleLayout()
morph('before', 2000)
```

Random graph

```
makeBarabasiAlbertRandom(5, 2, 300)
```

Small-world graph

```
makeWattsStrogatz(100, 4, 0.1)
```

Open an Excel workbook

```
makeFromExcel('network.xlsx')
```

Open any network file

```
makeFromFile('network.xml')
```

Choose the edge columns

```
makeFromCSV('ppi.tsv', source='gene_a', target='gene_b', interaction='type')
```

Open a Cytoscape SIF file

```
makeFromSIF('network.sif')
colorize(interaction)
```

Open Cytoscape.js JSON

```
makeFromCYJS('network.cyjs')
```

Export for Cytoscape

```
exportCYJS('graph.cyjs')
```

Open a CX or CX2 network

```
makeFromCX('network.cx2')
```

Open XGMML

```
makeFromXGMML('network.xgmml')
```

Networks from a Cytoscape session

```
makeFromCYS('analysis.cys', 'all')
```

Interactions from IntAct (MITAB)

```
makeFromMITAB('intact.txt')
```

PSI-MI XML interactions

```
makeFromPSIMI('interactions.xml')
```

A systems biology model (SBML)

```
makeFromSBML('model.xml')
```

A Reactome pathway (BioPAX)

```
makeFromBioPAX('pathway.owl')
```

A KEGG map

```
makeFromKGML('hsa04115.xml')
```

A WikiPathways pathway

```
makeFromGPML('WP707.gpml')
```

Co-authors from a list column

```
makeFromCooccurrence('papers.csv', 'authors', ';')
```

Open RDF / Turtle

```
makeFromRDF('foaf.ttl')
```

A Bluesky follow network

```
makeFromBluesky('bsky.app', limit=50)
```

Change a field's type

```
castField(year, INTEGER)
```

An interactive web page

```
exportHTML('graph.html')
```

Export for sigma.js

```
exportJSON('graph.json')
```

Export to Google Earth

```
exportKML('graph.kml')
```

Export shapefiles for GIS

```
exportSHP('graph.zip')
```

Follow a live stream

```
streamFrom('http://localhost:8080/workspace1?operation=getGraph')
```

Let an AI assistant help

```
connectAssistant()
```

Interface building

A slider that filters

```
s = Slider(freq.min, freq.max, label='min freq')
def update(v):
    (freq < v).visible = false
    (freq >= v).visible = true
s.onChange = update
ui.dock(Panel('Filter', s))
```

React to clicks

```
graphevents.clickNode = lambda n: center([n] + n.getNeighbors())
```

Record a movie

```
startMovie(30, 'run.webm')
gemLayout()
stopMovie()
```

Fields

Built-in node fields:

field	type	default
name	VARCHAR	None (read-only)
x	DOUBLE	500.0
y	DOUBLE	500.0
visible	BOOLEAN	True
color	VARCHAR	'cornflowerblue'
strokecolor	VARCHAR	None
labelcolor	VARCHAR	None
fixed	BOOLEAN	False
style	INTEGER	2
width	DOUBLE	10.0
height	DOUBLE	10.0
label	VARCHAR	None
image	VARCHAR	None
labelvisible	BOOLEAN	False
labelpos	VARCHAR	'right'
labelsize	DOUBLE	1.0
indegree	INTEGER	0 (read-only)
outdegree	INTEGER	0 (read-only)
totaldegree	INTEGER	0 (read-only)
degree	INTEGER	0 (read-only)

Built-in edge fields:

field	type	default
node1	VARCHAR	None (read-only)
node2	VARCHAR	None (read-only)
directed	BOOLEAN	False (read-only)

field	type	default
source	VARCHAR	None (read-only)
destination	VARCHAR	None (read-only)
color	VARCHAR	'dandelion'
labelcolor	VARCHAR	None
visible	BOOLEAN	True
width	DOUBLE	2.0
weight	DOUBLE	1.0
label	VARCHAR	None
labelvisible	BOOLEAN	False
labelsize	DOUBLE	1.0

Computed node fields (calculated the first time they are used, by name or as `n.<field>`; `recompute()` after editing the graph): `pagerank`, `hub`, `authority`, `eigenvector`, `closeness`, `harmonic`, `eccentricity`, `clustering`, `triangles`, `coreness`, `strength`, `component`, `strongcomponent`, `betweenness`, `degrank`, `hits`, `rwbetweenness`, `knn`; edge field `betweenness`.

Gython syntax

Gython is Python 3 plus these graph operators (see the CHI 2006 paper):

expression	meaning
`freq > 10`	elements whose field matches (`==`, `!=`, `<`, `>`, `<=`, `>=`, `like`)
`(freq > 10).color = blue`	setting a field on a set sets it on every element
`a & b`, `a | b`	intersection / union of element sets
`a -> b`, `a <- b`	directed edges from a to b / from b to a
`a <-> b`, `a - b`	undirected edges between a and b
`a ? b`	any edges between a and b
`s.difference(t)`	set difference
`bob[1999].color`	a value in a saved state
`Node rcontains 5`	range queries (`rcontains`, `roverlaps`, `rcontained`)
`g.nodes`, `g.edges`	all nodes / edges
`sel`	the current display selection